Adaptive Mesh Methods and a Schwarz Algorithm Implementation of a Smoothed Mesh Generating Function

by

© Andrew M. Rose

A thesis submitted to the

Faculty of Science

in partial fulfillment of the

requirements for the degree of

Bachelor of Science (Honours)

Department of Mathematics and Statistics

Memorial University of Newfoundland

April 2015

St John's Newfoundland & Labrador

Abstract

Employing an adaptive mesh for the purpose of solving PDEs is an effective, though complicated, method for improving the accuracy of numerical solutions. In this paper I will discuss the implementation of a classic Schwarz algorithm for a given, simple monitor function. We will study the impact that various aspects, such as the mesh discretization and the tolerance, have on the convergence and accuracy. This will initially be done on a single domain as a proof of concept before moving on naturally to examine the behaviour and convergence of domain decomposition implementation for our system for the purposes of parallelization. Finally, our classic Schwarz problem will be applied to a modified version of our differential equation and monitoring function in the framework of "smoothing" the solution. Numerical simulation results are provided to illustrate the successful application of these algorithms.

Contents

List of Figures

Acknowledgments

I would first like to thank Dr. Ronald Haynes for his guidance and supervision in this thesis. Thanks to his encouragement and patience I have learned and grown as both a student and as a researcher.

As well, I want to extend my thanks to Memorial University and the Department of Mathematics for the assistance and resources provided in performing this study and through my extensive time as a student at Memorial.

To my friends finishing their theses and degrees, I want to wish them the best in their future and thank them for providing an environment of support. Shared struggles and successes are so much more rewarding and I am glad we got to suffer and succeed together.

Finally, I thank my partner, Gina, my parents, and my siblings for pushing me in my academic career and always being there for me though the years. Without them, I never would have made it.

Chapter 1

Introduction

In different fields of mathematics, physics, and engineering, researchers are forced to analyze system models which are usually governed by complex differential equations. These equations may take years to find an analytic solution, or it may be possible that no solution exists. In these cases we must consider alternate methods for studying the solution to the differential equation.

Since no analytic solution exists or is too complicated to find, we instead will consider numerical (or discrete) methods for finding the solution. First, we can outline the standards for numerical differentiation and integration approximation techniques. Assuming some background knowledge of analysis, we can easily identify some numerical approximations through Taylor expansion. We perform a simple expansion

as a demonstration

$$f(x+\Delta x) = f(x) + \Delta x \frac{df(x)}{dx} + \frac{(\Delta x)^2}{2!}\frac{d^2 f(x)}{dx^2} + \cdots$$
$$f(x-\Delta x) = f(x) - \Delta x \frac{df(x)}{dx} + \frac{(\Delta x)^2}{2!}\frac{d^2 f(x)}{dx^2} - \cdots$$

These well known and well understood expansions are the backbone of numerical methods for differential equations. Thanks to their simple construction, if we choose a sufficiently small value for Δx then we may ignore the higher order terms in the Taylor expansion and consider only a finite truncation of these infinite series. For example, through a simple rearrangement

$$\frac{df(x)}{dx} = \frac{f(x+\Delta x) - f(x-\Delta x)}{2\Delta x} - \frac{(\Delta x)^2}{12}\frac{d^3 f(x)}{dx^3} + \cdots$$

is an expression for the expansion of $\frac{df(x)}{dx}$. We usually truncate this expansion to only the first term, $\frac{f(x+\Delta x)-f(x-\Delta x)}{2\Delta x}$. With this, we are able to approximate the derivative of a function by the surrounding function values. Hence, we may use this numerical form to approximate the function $f(x)$ over the whole domain of x.

Clearly, these approximations are more accurate as $\Delta x \to 0$. This limit recovers the exact solution. However, in doing this we would require to discretize the domain

of x into infinitely many Δx's. In fact, computational limitations exist which create a lower limit on how finely we may discretize our domain. Standard machine precision will fail to accurately perform calculations with floating point values less than $\approx 10^{-15}$. From this, it is clear that while the numerical method will approach convergence at the limit for $\Delta x \to 0$ this is not something which is not a feasible option in computational implementation. Thus, we may limit our discretization in terms of a finite number of points. This will define how many equations we need to solve.

The traditional method for discretization of a function $f(x)$ involves uniformly selecting $\Delta x = \frac{|x|}{N}$. In this case, we consider the domain of f, x, to be normalized for simplicity and thus $|x| = 1$. This means that all consecutive values for x are spaced apart by Δx. We will consider the domain of x to be $[0, 1]$ and the i^{th} value of x in the domain to be denoted x_i. Hence, $x_{i+1} - x_i = \Delta x$ for all $x \in [0, 1]$ and $i = 0, 1, 2, \ldots, N - 1$. These values of x_i are known as the "mesh points" of our domain and the general set of points $\mathcal{D} = \{x_i\}$ is known as the "mesh".

However, if a differential equation has regions of complicated behaviour then our choice of Δx may be insufficient to pick up its subtleties. The introductory approach toward fixing this issue is to simply increase the value of N. This reduces our value of Δx and thus comes closer to the exact solution and is more capable of handling this differential equation. For a simple forward marching scheme this approach is not

a bad plan. However, in the case of boundary value problems the process for solving the equation is not an iterative scheme but is instead equivalent to solving an $N \times N$ system of equations. This will be discussed in further detail when developing the methodology.

When solving $N \times N$ systems of equations the complexity of the solver scales as $\mathcal{O}(N^2)$. Thus, while we have increased our numerical accuracy we have increased our computational load.

Our particular interest comes from the cases where the differential equation is solvable with a low value of N in almost the entire domain except for a small number of points of complicated behaviour. This is to say, we are interested in the cases where the problem is approximately linear, up to a small number of localized non-linear activity.

In the standard way, the mesh points are constructed with uniform separation. This means that points are considered without biasing everywhere in the domain. This has the clear impact that if the problem is simple and easy to solve everywhere but for some finite number of points, then these points will dictate the total number of mesh points used. In theory, the total number of mesh points is dictated, at minimum, by the density required to accurately solve the differential equation in its most difficult region. For example, if we know a differential equation requires a spacing of δ in the

region $[a, b]$ then the number of points is $\frac{b-a}{\delta}$ and the overall number is $\frac{1}{\delta}$. It does not matter if this region is the only one which needs this density. For a uniform mesh this will become the definition of the spacings.

The natural following question is whether we are able to concentrate our values of x_i near these points of interest. If this is the case, then we cannot have the simple uniform spacing over the entire domain. We require the value of Δx to be variable so as to capture these points of difficulty.

This process is known as constructing an "adaptive mesh", so named for its ability to adapt to the specific problem and not be blind to the particular intricacies of its behaviour.

The use of adaptive meshes in the study of partial differential equations is an effective tool. When analyzing a PDE which has regions of highly erratic behaviour it is useful to cluster our mesh grid points in these regions to account for the more complicated dynamics of the solution. However, this has the clear difficulty in needing to know where the solution is varying.

Once we have considered the above processes we will extend this methodology into the world of domain decomposition (DD). While the solution to the single domain is the obvious and straight-forward method, a modification exists which may improve the efficiency and quality of the found solution. This method utilizes breaking up the

domain into several smaller subdomains. This is known as domain decomposition. There are many ways to do this, however we will explain the basic method used for the standard Schwarz Algorithm [5, 12] here.

Let us first consider the simple case of decomposing the domain in to two subdomains.

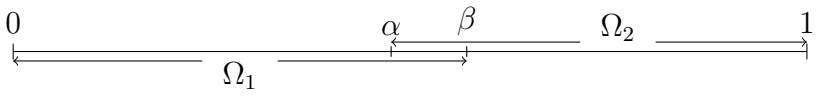


Figure 1.1: Decomposition of the standard normal domain into two subdomains.

By following through with the boundary value methods, as outlined in Chapter 2, we can solve our equations on each domain Ω_i. We do this by including modified boundary conditions which are outlined as follows:

$$x_1(0) = 0$$

$$x_2(1) = 1$$

$$x_1^k(\beta) = x_2^{k-1}(\beta)$$

$$x_2^k(\alpha) = x_1^{k-1}(\alpha).$$

Here, we have used the notation that x_i^k is the numerical approximation to the

solution on the i^{th} subdomain during the k^{th} iteration of the numerical scheme.

This is an effective way of partitioning the numerical scheme into two regions. As will be demonstrated later, this provides benefits to both the computational complexity and the ability to solve the numerical systems in a parallel method.

However, this approach suggests a possible generalization: Can we extend the problem's decomposition into two subdomains into more? Arbitrarily many subdomains? We will now investigate the case of some finite number of overlapping subdomains.

In doing this, we consider our domain Ω as defined before. In the above procedure we solve the equations over several smaller intervals at once. We will define these subintervals as follows,

$$\Omega_i = \{\alpha_i, \beta_i\} \tag{1.1}$$

$$\alpha_i, \beta_i \in \Omega \tag{1.2}$$

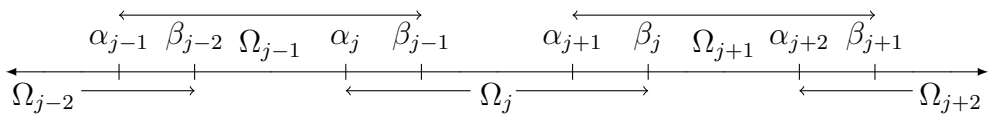
where for S subintervals

$$\alpha_1 = 0, \beta_S = 1$$

$$\alpha_{i+1} < \beta_i < \beta_{i+1}, \forall i = 1, 2, \ldots, S-1.$$

The result of this decomposition is that:

$$\Omega = \cup_{i=1}^{S} \Omega_i \tag{1.3}$$



Now, with this new construction in place, we must consider how our boundary conditions will be represented. In the case where we solve over the entire domain we know that both boundary values will be taken into consideration. However, in the case of domain decomposition this is not so clear. For example if S = 2, then the first subdomain possesses information regarding the boundary at $\xi = 0$, while the second subdomain has information about the boundary at $\xi = 1$.

How do we transmit this information from one domain to the other in order to accurately solve the system?

As in the case with the entire Ω domain, we require an iterative process. In this situation instead of a single $x^k(\xi)$ approximation at the k^{th} step, we have S different approximations, denoted $x_i^k(\xi)$ for $i = 1, 2, \ldots, S$. Each of these subdomains can be solved as their own single boundary value problem. Once all of the subdomains are solved for the k^{th} iteration, we will construct new boundary conditions for each of

them. This is done by taking the points interior to the neighboring subdomain which correspond to the boundary of the current subdomain.

For example, consider $\Omega_1 = \{0, \ldots, \beta_1\}$ and its neighbor $\Omega_2 = \{\alpha_2, \ldots, 1\}$. The value of the boundaries of Ω_1 are at $\xi = 0$ and $\xi = \beta_1$. We know that the value at $\xi = 0$ is correct as this is a true boundary. However, the value at $\xi = \beta_1$ is merely our approximation. What we will do is take the calculated value for the interior point $\xi = \beta_1$ which lies inside Ω_2, and make this our new value at boundary of thus subdomain. More accurately we will say

$$x_1^{k+1}(\beta_1) = x_2^k(\beta_1). \tag{1.4}$$

We would similarly say for Ω_2:

$$x_2^{k+1}(\alpha_2) = x_1^k(\alpha_2). \tag{1.5}$$

The new "right side" boundary for Ω_1 is constructed with influence from the true boundary at $\xi = 1$ and the new "left side" boundary for Ω_2 is constructed with influence from the true boundary at $\xi = 0$. These new boundaries are not immediately correct, but their approximation becomes better over time. This will be proven further in this text.

This method clearly will generalize to an arbitrary number of subdomains, though with a major issue. In the case of only 2 subdomains on the first iteration information from the first boundaries was used for the first subdomain solve, while information from the second boundary is used for solving on the second subdomain On the second iteration we can use this boundary information for both subdomains due to the transmission conditions. However, for the case of three subdomains it will not be until the third iteration. This trend will continue for any number, S, of subdomains. It takes S iterations for information from the boundaries to completely cover the domain. This is due to the information having to be used in each of the intermediate subdomains. In addition, we use only the interior calculated values from each pass. Thus, the information will degrade over successive iterations. Hence, more subdomains will result in poor transmission of boundary information. This begs the question, why would we use many subdomains?

The answer to the above question is two-fold. First, the complexity of solving systems of equations. Most methods which we will see for generating these meshes reduce to solving matrix-vector systems of equations. We know that standard methodology for solving these $N \times N$ systems through standard inversion scales as $\mathcal{O}(N^2)$ if the matrix is non-sparse. Thus, if for a given single domain we have an $N \times N$ system of equations our complexity is $\mathcal{O}(N^2)$. Then by breaking it into two subdomains, each

of approximately half the size of the original domain, we are left with a complexity of:

$$\mathcal{O}\left(\left(\frac{N}{2}\right)^2\right) + \mathcal{O}\left(\left(\frac{N}{2}\right)^2\right) \approx \frac{\mathcal{O}(N^2)}{4} + \frac{\mathcal{O}(N^2)}{4} \approx \frac{\mathcal{O}(N^2)}{2}. \tag{1.6}$$

Our new system is less computationally heavy as we have created smaller, more manageable systems to be solved on each iteration. While we may be required to do more iterations to accurately solve, each one is simpler.

As a note, first we will study a simple case where the system of linear equations corresponds to a solving a tridiagonal matrix. This particular type of matrix has efficient solving techniques which reduce the complexity to $\mathcal{O}(N)$. This is of benefit for the simple "proof of concept" calculations but as we transition to more complex methods this complexity factor will return to a similarly defined $\mathcal{O}(N^2)$.

Secondly, the power of parallelization within the implementation of this methodology. Thanks to neighboring systems requiring information about only the solution from the previous iteration, we are able to perform all the k^{th} iterations simultaneously on different computer cores or machines. This parallelization, coupled with the decreased complexity of the systems, suggest that domain decomposition is a powerful tool in numerical methods as it allows for more efficient use of computational power.

In fact, these above points underline the motivation for domain decomposition in

general. An ability to parallelize and solve multiple smaller systems clearly has a place in the world of numerical analysis of differential equations. Thanks to these benefits, we can attack more complicated problems than normal with the benefit of knowing that computational loads can be effectively managed and accuracy preserved.

In this study we will examine the well understood methods of mesh generation through a boundary value problem and its numerical solution. This process is at the forefront of adaptive mesh generation techniques and possess many properties which lend well to further analysis. We will also consider the famous and field standard method of de Boor's Algorithm for splines. This method is well known and is a classic introduction to numerical generations of meshes. Next, we will consider a modification to the mesh generating function which will attempt to create a more natural adaptation to the mesh. This mesh is known as a smoothed mesh. Its motivation lies in [11] which suggests that meshes which are smoother will provide more accurate results than meshes which do not transition between regions of complex behaviour to regions of simple dynamics in a "smooth" manner. Then, we will introduce the notion of domain decomposition for numerical schemes for both the standard methods and our "smoothed" meshes.

Moving from this initial set up we will begin our study of the Smoothed Mesh and general 4^{th} order equations. This study will consider domain decomposition

techniques for the smoothed problem, general 4^{th} order equations, and De Boor's Algorithm. Domain decomposition methods for 4^{th} order equations are not as well understood as domain decomposition methods for 2^{nd} order problems. Few examples of implementation exist and there is little theory developed on the subject. In this work we will attempt to develop a structure for the study of the convergence and effectiveness of domain decomposition when applied to 4^{th} order equations.

Section 4 will discuss the numerical results from the implementation of these mesh generation techniques. As well we will consider the relationship and effect of the number of subdomains, the overlap size of the subdomains, and the effect of a smoothing technique on the mesh generating function.

Finally, we consider the end results of our study and suggest future areas of research in this topic. Such possible topics include Linearized and Optimized Schwarz algorithms. As well, we will introduce the idea of applying the smoothed mesh to the de Boor algorithm.

Chapter 2

Existing Methods and Schemes

In this section we introduce the standard methods and practices for generating an adapted mesh. This includes discussion of the boundary value problem method for constructing meshes. As well, we study de Boor's method for mesh construction which is the backbone of many techniques today. Finally, we will introduce a newer method for mesh generation which involves a modified monitor function. This new mesh type is known as the "smoothed" mesh.

2.1 Boundary Value Problem Approach

We will be considering Boundary Value Problems (BVPs) in all of our investigations thus we will be using a numerical scheme designed for BVPs [9]. In addition, this method lends well to extensions into higher dimensions which is often a concern in numerical schemes.

The key mathematical framework underlying our algorithm is the equidistribution principle [2,7]. For a given monitoring function, $M(x)$, we construct the a discretization of a domain, $\mathcal{D} = [0, 1]$, in a manner [9] such that,

$$\int_{x_0}^{x_1} M(x)\,dx = \cdots = \int_{x_{N-1}}^{x_N} M(x)\,dx,\ x_i \in \mathcal{D} \tag{2.1}$$

This is to say that we desire the integral over each subinterval in $\mathcal{D}$ to be equal. This will be the "mesh" that will we later study. Therefore, we require

$$\int_{x_{i-1}}^{x_i} M(x)\,dx = \theta,\ \forall i \in \{1, \ldots, N\} \tag{2.2}$$

$$\int_{\mathcal{D}} M(x)\,dx = \sigma,\ \theta = \frac{\sigma}{N}. \tag{2.3}$$

where θ and σ are constants. Then, we can partition any integral through the mesh nodes x_i as:

$$\int_{x_0}^{x_i} M(x)\,dx = \frac{i}{N}\sigma,\ x_i \in \mathcal{D}. \tag{2.4}$$

We define a mapping between the domain $\mathcal{D}$ and a new computational domain Ω:

$$F : \mathcal{D} \to \Omega$$

$$x_i \mapsto \xi_i$$

More specifically, with reference to the computational domain,

$$\xi_i = \frac{i}{N}$$

$$\int_{x_0}^{x(\xi)} M(x)\,dx = \xi\sigma. \tag{2.5}$$

We differentiate (2.5) with respect to ξ and arrive at the following,

$$M(x)\frac{dx}{d\xi} = \sigma. \tag{2.6}$$

The above equation (2.6) becomes the major driving force behind the forth-coming discretization and iteration schemes used to generate our meshes.

We differentiate (2.6) with respect to our computational domain ξ to arrive at a non-linear, second order boundary value problem:

$$\frac{d}{d\xi}\left(M(x)\frac{dx}{d\xi}\right) = 0 \tag{2.7}$$

with boundary conditions

$$x(0) = 0,\ x(1) = 1.$$

The solution to this boundary value problem, $x(\xi)$, is a distribution of nodes which will be our new mesh. There are several known methods for solving non-linear, second order boundary value problems, the most well known is the Newton method which we will now investigate.

Newton's method is a well known and well understood method for solving non-linear systems equations. It is known for being both robust and accurate. We will make use of the traditional Newton Solver as described by LeVeque, [10], to solve our system. First, we must define a vector function, denoted $\mathcal{G}$. Here, $\mathcal{G}$ is the system of discrete equations which arise from the discretization of the equation (2.7). We may simply discretize (2.7) as:

$$M\left(\frac{x_{i+1}+x_i}{2}\right)\frac{x_{i+1}-x_i}{\Delta\xi} - M\left(\frac{x_i+x_{i-1}}{2}\right)\frac{x_i-x_{i-1}}{\Delta\xi} = 0. \tag{2.8}$$

where the values i will iterate from 1 to N-1. Thus, $\mathcal{G}$ will accept a "guess" at the solution $x(\xi)$. We will denote the k^{th} guess at $x(\xi)$ as x^k. Thus we are now defining $\mathcal{G}$ as

$$\mathcal{G}(x^k) = \begin{pmatrix} M\left(\frac{x_2+x_1}{2}\right)(x_2 - x_1) - M\left(\frac{x_1+x_0}{2}\right)(x_1 - x_0) \\ M\left(\frac{x_3+x_2}{2}\right)(x_3 - x_2) - M\left(\frac{x_2+x_1}{2}\right)(x_2 - x_1) \\ \vdots \\ M\left(\frac{x_N+x_{N-1}}{2}\right)(x_N - x_{N-1}) - M\left(\frac{x_{N-1}+x_{N-2}}{2}\right)(x_{N-1} - x_{N-2}) \end{pmatrix} \tag{2.9}$$

This is the $N-2$ by 1 vector of values representing the discretization of our governing differential equation evaluated at the approximation, x^k, to the solution, $x(\xi)$. For consistency of notation, we will denote the $\mathcal{G}(x^k)$ as $\mathcal{G}^k$, the system of discretization at the k^{th} approximation of $x(\xi)$.

For the purposes of our implementation we use a midpoint scheme for evaluating the monitor function. Standard definition of the error bounds in numerical integration yields the well known estimates [1] for the error of the midpoint approximation,

$$|E_{Mid}| \leq \frac{K|x|^2}{24N^2}$$

and for the trapezoidal approximation,

$$|E_{Trap}| \leq \frac{K|x|^2}{12N^2}.$$

Where we have used $|x|^2$ to be the length of the integral's domain, here $|x|^2 = 1$. This result implies that our choice of a midpoint scheme is the best option for accurate and efficient calculations.

We next define the Jacobian matrix corresponding to the vector $\mathcal{G}$. This will be denoted as $\mathcal{J}$. $\mathcal{J}$ is constructed using the standard method which is to say that $\mathcal{J}$ evaluated at the k^{th} approximation is:

$$\mathcal{J}(x^k) = \begin{pmatrix} \frac{\partial \mathcal{G}_1}{\partial x_1} & \frac{\partial \mathcal{G}_1}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_1}{\partial x_{N-1}} \\ \frac{\partial \mathcal{G}_2}{\partial x_1} & \frac{\partial \mathcal{G}_2}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_2}{\partial x_{N-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{G}_{N-1}}{\partial x_1} & \frac{\partial \mathcal{G}_{N-1}}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_{N-1}}{\partial x_{N-1}} \end{pmatrix}. \tag{2.10}$$

Once again, we will denote the Jacobian $\mathcal{J}$ evaluated at the approximation x^k as $\mathcal{J}^k$. This is the Jacobian corresponding to $\mathcal{G}^k$. A sample of the Jacobian entries are computed below to describe the implementation.

$$\frac{\partial \mathcal{G}_1}{\partial x_1} = \frac{1}{2}\frac{\partial M(x)}{\partial x}\Big|_{\frac{x_2+x_1}{2}}(x_2-x_1) - M\left(\frac{x_2+x_1}{2}\right) - \frac{1}{2}\frac{\partial M(x)}{\partial x}\Big|_{\frac{x_1+x_0}{2}}(x_1-x_0) - M\left(\frac{x_1+x_0}{2}\right)$$

$$\frac{\partial \mathcal{G}_1}{\partial x_2} = \frac{1}{2}\frac{\partial M(x)}{\partial x}\Big|_{\frac{x_2+x_1}{2}}(x_2 - x_1) + M\left(\frac{x_2+x_1}{2}\right)$$

$$\frac{\partial \mathcal{G}_2}{\partial x_1} = \frac{-1}{2}\frac{\partial M(x)}{\partial x}\Big|_{\frac{x_2+x_1}{2}}(x_2 - x_1) + M\left(\frac{x_2+x_1}{2}\right)$$

This completes the mathematical language which we will use to solve our system over our entire computational domain.

First we define a standard uniform mesh over the computational domain ξ. After discretizing our governing equation (2.7) we apply it to all points of the computational domain. This may involve modifications near boundaries, however these are simply different discretizations of (2.7) at the appropriate points. This system is only defined for the points interior to the domain. When considering the method to follow, we simply only update the "solution" points in the interior to the computational domain. This is because the boundary points are defined and known exactly. We construct the system of non-linear equations,$\mathcal{G}$, and the canonical Jacobian, $\mathcal{J}$, as defined previously. For notation, we will indicate the system $\mathcal{G}_k$ as the system of non-linear equations evaluated using our approximation of the solution at the k^{th} iteration. Similarly we define $\mathcal{J}_k$ as the Jacobian evaluated using the approximation of the solution at the k^{th} iteration. Then our iterative scheme is:

$$x_{k+1} = x_k - \mathcal{J}_k^{-1}\mathcal{G}_k. \tag{2.11}$$

We must now define a stopping criterion for the iteration. The Newton solver defines the stopping condition such that for some defined tolerance, denoted tol:

$$\|\mathcal{J}_k^{-1}\mathcal{G}_k\|_\infty < tol. \tag{2.12}$$

For our purposes we will choose the norm to be the infinity norm. This is to say that the infinity norm of the difference between consecutive Newton iterations is less than our defined acceptable tolerance. Then the final $x_k = x_*$ is our numerical solution to the equation (2.7).

2.2 De Boor's Algorithm

Perhaps the first major algorithm used for solving adaptive meshes is de Boor's Algorithm. This procedure [9] was developed due to the work of Carl de Boor [2] in a methodology for approximating functions by piecewise polynomial interpolants. This algorithm takes a function, denoted $\rho(x)$, and constructs a piecewise–linear interpolant which corresponds to the equidistribution principle's requirements of computational load balancing. We will follow through the derivation as outlined in [9] to provide a suitable background for future reference.

First, we will define an initial mesh (most likely a uniform distribution of k points

from 0 to 1) denoted $X = \{x_i^o \mid i = 0, \ldots, N \text{ where } x_i^o < x_{i+1}^o\}$. This mesh may be of arbitrary non-trivial size as it is only needed for the initialization of our mesh iteration. The following piecewise-constant function is then defined on this mesh.

$$p(x) = \begin{cases} \frac{1}{2}\left(\rho\left(x_0^o\right) + \rho\left(x_1^o\right)\right) & \text{for } x \in [x_0^o, x_1^o] \\ \frac{1}{2}\left(\rho\left(x_1^o\right) + \rho\left(x_2^o\right)\right) & \text{for } x \in (x_1^o, x_2^o] \\ \vdots & \\ \frac{1}{2}\left(\rho\left(x_{N-1}^o\right) + \rho\left(x_N^o\right)\right) & \text{for } x \in (x_{N-1}^o, x_N^o] \end{cases}$$

We will note the integral function of this is

$$P(x) = \int_{x_0^o}^{x} p(x)dx. \tag{2.13}$$

Thus, we may express this as the finite sum

$$P(x_j^o) = \sum_{i=1}^{j} (x_i^o - x_{i-1}^o)\frac{\rho(x_i^o) - \rho(x_{i-1}^o)}{2} \tag{2.14}$$

for $j = 1, \ldots, k$. By the equidistribution principle we require that

$$P(x_j) = \frac{j}{N}P(1),\ j = 1, \ldots, N-1 \tag{2.15}$$

for equal distribution of the computational load. Next, we find consecutive mesh values so that

$$P(x_{k-1}^o) < \frac{j}{N}P(1) < P(x_k^o). \tag{2.16}$$

By a simple algebraic rearrangement we may express the new value for the j$^{\text{th}}$ mesh point as

$$x_j^1 = x_{k-1}^o + \frac{2(\frac{j}{N}P(1) - P(x_{k-1}^o))}{\rho(x_{k-1}^o) + \rho(x_k^o)}. \tag{2.17}$$

Thus, after solving for $j = 1, \ldots, N-1$ this will be our new mesh approximation with $x_0^1 = 0, x_N^1 = 1$. This is the case for the first iteration of de Boor's algorithm. This simple scheme may be repeated until convergence is reached. The general h$^{\text{th}}$ mesh approximation is described as

$$x_j^h = x_{k-1}^{h-1} + \frac{2(\frac{j-1}{N-1}P(1) - P(x_{k-1}^{h-1})}{\rho(x_{k-1}^{h-1}) + \rho(x_k^{h-1})}. \tag{2.18}$$

This convergence is usually defined by the following simple stopping condition

$$||x^h - x^{h-1}||_\infty < tolerance \tag{2.19}$$

where $||\cdot||_\infty$ is the usual infinity-norm, $max\{|x^h - x^{h-1}|\}$. Thus, when the above condition is met, $x^h = x^* = X$ is the converged de Boor mesh.

2.3 Smoothed Domain Solver

The following analysis is performed on a modified version of our original differential equation (2.7). This new version will henceforth be referred to as the "Smoothed" version of our original mesh generating process. The final form [8] is seen below. In the following sections we will discuss the derivation, motivation, and discretization of the differential equation

$$\frac{d}{d\xi}\left(\frac{1}{M(x)}\left(I-\beta^{-2}\frac{d^2}{d\xi^2}\right)\left(\frac{1}{\frac{dx}{d\xi}}\right)\right)=0. \tag{2.20}$$

2.3.1 Derivation of Smoothed Equation

In [11] it was demonstrated that for a mesh generated by a weight function the truncation error depends on the weight function and its derivative. Specifically, if the weight function is smooth in its variable then the iteration method will converge faster than coarser and less regular functions. This is our major motivating factor for implementation. Additionally, as will be seen in numerical examples to follow, smoothed meshes are particularly adept at capturing points between dense regions of mesh points. This so that the transition from dense to sparse is without abrupt changes in a natural way which preserves the structure of the mesh.

We first studied the original differential equation, (2.7). As outlined in [11], smooth meshes provide more accuracy when applied. We make the assumption that there exists a new generating function $\widetilde{M}(x)$ as follows. Consider the case, outlined in [9], where

$$\left(I - \beta^{-2}\frac{d^2}{d^2\xi}\right)\widetilde{M}(x) = M(x) \tag{2.21}$$

$$\frac{d\widetilde{M}}{d\xi}(0) = \frac{d\widetilde{M}}{d\xi}(1) = 0.$$

For simplicity we will denote $S = \left(I - \beta^{-2}\frac{d^2}{d^2\xi}\right)$. Then by applying (2.7) to $\widetilde{M}(x)$ we arrive at,

$$\frac{d}{d\xi}\left(\widetilde{M}(x)\frac{dx}{d\xi}\right) = 0.$$

Substituting (2.21) we have,

$$\frac{d}{d\xi}\left(S^{-1}M(x)\frac{dx}{d\xi}\right) = 0.$$

Now we have the new constant function,

$$S^{-1}M(x)\frac{dx}{d\xi} = \theta,$$

which can be rearranged into the convenient form,

$$\frac{1}{\theta} = \frac{1}{M(x)}S\frac{1}{\frac{dx}{d\xi}}.$$

Hence, we now have the governing differential equation,

$$\frac{d}{d\xi}\left(\frac{1}{M(x)}S\frac{1}{\frac{dx}{d\xi}}\right) = 0$$

which can be rewritten as

$$\frac{d}{d\xi}\left(\frac{1}{M(x)}\left(I-\beta^{-2}\frac{d^2}{d\xi^2}\right)\left(\frac{1}{\frac{dx}{d\xi}}\right)\right) = 0.$$

As we can see, by considering the extreme limits of (2.21) in terms of the β value we are able to recover the original monitor function, $M(x)$, as $\beta \to \infty$. As $\beta \to 0$ the higher order differential term becomes dominant and we lose the original monitor function and the new monitor function becomes approximately cubic in x.

2.3.2 Discretization

Here, we will walk through the process of discretizing the differential equation (2.20). First, we consider a simple expansion of the system for visualization. This begins as

$$\left(\frac{1}{M(x)}\left(I-\beta^{-2}\frac{d^2}{d\xi^2}\right)\left(\frac{1}{\frac{dx}{d\xi}}\right)\right) = C \tag{2.22}$$

for some arbitrary constant C. This constant is meaningless and will be dismissed soon enough. Thus we may take $C = 0$ without loss of generality. Thus

$$\frac{1}{M(x)}\left(\frac{1}{\frac{dx}{d\xi}}-\beta^{-2}\frac{d^2}{d\xi^2}\frac{1}{\frac{dx}{d\xi}}\right)=F(\xi)=0 \tag{2.23}$$

is our expanded interior term. Since we know this must be a constant in ξ we know we may take any difference and arrive at $F(\xi_1)-F(\xi_2)=0$ for any $\xi_i\in\Omega$. We return to the discretization of the Ξ domain as defined in chapter 1 and proceed under the same notation. In our particular case, we will consider specifically the half-step method for the purposes of numerical accuracy. Hence, we arrive at the expression:

$$F(\xi_{j+\frac{1}{2}})-F(\xi_{j-\frac{1}{2}})=0$$

$$\begin{aligned}\frac{1}{M_{j+\frac{1}{2}}}&\left[\frac{1}{\frac{dx_{j+\frac{1}{2}}}{d\xi}}-\frac{1}{\beta^2\Delta\xi^2}\left(\frac{1}{\frac{dx_{j+\frac{3}{2}}}{d\xi}}-2\frac{1}{\frac{dx_{j+\frac{1}{2}}}{d\xi}}+\frac{1}{\frac{dx_{j-\frac{1}{2}}}{d\xi}}\right)\right]\\&-\frac{1}{M_{j-\frac{1}{2}}}\left[\frac{1}{\frac{dx_{j-\frac{1}{2}}}{d\xi}}-\frac{1}{\beta^2\Delta\xi^2}\left(\frac{1}{\frac{dx_{j+\frac{1}{2}}}{d\xi}}-2\frac{1}{\frac{dx_{j-\frac{1}{2}}}{d\xi}}+\frac{1}{\frac{dx_{j-\frac{3}{2}}}{d\xi}}\right)\right]=0\end{aligned} \tag{2.24}$$

The above is obtained using standard techniques in numerical differentiation. Now, define the discretization of $\frac{1}{\frac{dx_{j+\frac{1}{2}}}{d\xi}}\approx\frac{1}{\frac{x_{j+1}-x_j}{\Delta\xi}}=\frac{\Delta\xi}{x_{j+1}-x_j}$ as $y_{j+\frac{1}{2}}$. We can dis-

cretize the system in (2.20) simply as

$$\frac{1}{M_{j+\frac{1}{2}}}\left[y_{j+\frac{1}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{j+\frac{3}{2}} - 2y_{j+\frac{1}{2}} + y_{j-\frac{1}{2}}\right)\right] - \frac{1}{M_{j-\frac{1}{2}}}\left[y_{j-\frac{1}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{j+\frac{1}{2}} - 2y_{j-\frac{1}{2}} + y_{j-\frac{3}{2}}\right)\right] = 0. \quad (2.25)$$

This is now the discretized version of our non-linear system. This is known as our G equation(s) as discussed in section 1.

Our discretization can be simply implemented as before with a Newton Solver with the associated Jacobian matrix.

In chapter 4, we see many examples of adaptive meshes generated. However, to see a uniform mesh and an adapted mesh, refer to Figure 4.28. This demonstrates a monitoring function which is plotted using a standard uniform mesh, as well as an adapted mesh which concentrates points near the largest values of the mesh. This can be thought of as a mesh generated by a function whose peaks are the points where we desire more node points.

We will next consider the implementation of a domain decomposition method for the smoothed mesh generation method as described above. This will be for the purpose of preserving the advantages of the smoothed technique while benefiting from the parallelization advantages of domain decomposition. In chapter 4, we will

see numerical results which demonstrate the advantages of this implementation.

Chapter 3

Domain Decomposition Methods for 4^{th} Order linear BVPs and Smoothed Mesh Generating Functions

In this section we will apply a domain decomposition technique to the smoothed mesh generating function. We will demonstrate the iteration technique and describe the results found as evidence that the smoothed mesh equation lends well to domain decomposition.

3.1 Domain Decomposition for a model 4^{th} Order Boundary Value Problem

We start to prove the convergence of our numerical scheme by first considering a model problem.

$$u_{xxxx} = 0 \tag{3.1}$$

$$u(0) = 0,\ u(1) = 1$$

$$u_x(0) = 1,\ u_x(1) = 1$$

This is a general homogeneous fourth order differential equation in a single variable. Next, we consider the basic decomposition of our domain into two overlapping subdomains.

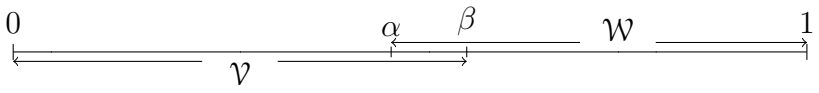


Figure 3.1: A simple decomposition of the standard normal domain into two subdomains for our fourth order problem.

This system has the following boundary value problem differential equation forms

when applied to the two subdomains:

$$\mathcal{V}^k_{xxxx} = 0 \qquad \mathcal{W}^k_{xxxx} = 0$$

$$\mathcal{V}^k(0) = 0 \qquad \mathcal{W}^k(1) = 1$$

$$\mathcal{V}^k_x(0) = 1 \qquad \mathcal{W}^k_x(1) = 1$$

$$\mathcal{V}^k(\beta) = \mathcal{W}^{k-1}(\beta) \qquad \mathcal{W}^k(\alpha) = \mathcal{V}^{k-1}(\alpha)$$

$$\mathcal{V}^k_x(\beta) = \mathcal{W}^{k-1}_x(\beta) \qquad \mathcal{W}^k_x(\alpha) = \mathcal{V}^{k-1}_x(\alpha)$$

This will be the general form for the two subdomain version of the 4^{th} order Schwarz Algorithm.

In preparation for an upcoming theorem, we will outline the following mathematical preliminaries.

Lemma 1. *The error on each subdomain satisfies* $e^k_{\mathcal{V},xxxx} = 0$ *and* $e^k_{\mathcal{W},xxxx} = 0$.

Proof. We can express the general differential equation of the for $u_{xxxx} = 0$. Thus,

for each subdomain we know that $\mathcal{V}^k_{xxxx} = 0$ and $\mathcal{W}^k_{xxxx} = 0$. Hence, the difference is

$$e^k_{\mathcal{V},xxxx} = u_{xxxx} - \mathcal{V}^k_{xxxx} = 0$$

$$e^k_{\mathcal{W},xxxx} = u_{xxxx} - \mathcal{W}^k_{xxxx} = 0$$

We have confirmed the error for our differential form. □

Lemma 2. *The error on each subdomain satisfies* $e^{\mathcal{V}}_k(\beta) = e^{\mathcal{W}}_{k-1}$.

Proof. We know that the error at $x = \beta$ can be written as

$$e^k_{\mathcal{V}}(\beta) = u(\beta) - \mathcal{V}^k(\beta).$$

Based on our definitions, it is also known that

$$\mathcal{V}^k(\beta) = \mathcal{W}^{k-1}(\beta).$$

Hence,

$$e^k_{\mathcal{V}}(\beta) = u(\beta) - \mathcal{W}^{k-1}(\beta).$$

But we also have that

$$e^k_{\mathcal{W}}(\beta) = u(\beta) - \mathcal{W}^k(\beta).$$

Thus, we arrive at the desired conclusion

$$e^k_{\mathcal{V}}(\beta) = e^{k-1}_{\mathcal{W}}(\beta).$$

The corresponding definition for $e^k_{\mathcal{W}}(\alpha)$is found similarly. □

Lemma 3. *The error on each subdomain satisfies* $e^k_{\mathcal{V},x}(\beta) = e^{k-1}_{\mathcal{W},x}$.

Proof. We know that the error at $x = \beta$ can be written as

$$e^k_{\mathcal{V},x}(\beta) = u_x(\beta) - \mathcal{V}^k_x(\beta).$$

Once again, we know by definition that

$$\mathcal{V}^k_x(\beta) = \mathcal{W}^{k-1}_x(\beta).$$

Then we have,

$$e^k_{\mathcal{V},x}(\beta) = u_x(\beta) - \mathcal{W}^{k-1}_x(\beta).$$

Additionally, we know

$$e^k_{\mathcal{W},x}(\beta) = u_x(\beta) - \mathcal{W}^k_x(\beta).$$

Hence,we have

$$e^k_{\mathcal{V},x}(\beta) = e^{k-1}_{\mathcal{W},x}(\beta).$$

The definition for $e^k_{\mathcal{W},x}(\alpha)$ is found via a similar method. □

With the above lemmas in mind, we present the following theorem:

Theorem 1. *Let the differential equation be* $u_{xxxx} = 0$ *with boundary conditions* $u(0) = 0,\ u(1) = 0, u_x(0) = 1,\ u_x(1) = 1$ *defined on* $x \in \mathcal{D} = [0, 1]$. *Decomposing*

into $\mathcal{D} = [0, \beta] \cup [\alpha, 1]$ *where* $\alpha < \beta$ *with* $\alpha,\ \beta \in \mathcal{D}$. *Under this construction the Schwarz Iteration converges for any initial guesses* x_1^0, x_2^0.

Proof. We have the previously mentioned conditions for the differential equation.

$$\mathcal{V}_{xxxx} = 0 \qquad \mathcal{W}_{xxxx} = 0$$

$$\mathcal{V}(0) = 0 \qquad \mathcal{W}(1) = 1$$

$$\mathcal{V}_x(0) = 1 \qquad \mathcal{W}_x(1) = 1$$

$$\mathcal{V}^k(\beta) = \mathcal{W}^{k-1}(\beta) \qquad \mathcal{W}^k(\alpha) = \mathcal{V}^{k-1}(\alpha)$$

$$\mathcal{V}_x^k(\beta) = \mathcal{W}_x^{k-1}(\beta) \qquad \mathcal{W}_x^k(\alpha) = \mathcal{V}_x^{k-1}(\alpha)$$

Now we define error terms on each subdomain. These errors measure the difference between the exact solution and the numerical solution at the iteration count. We also impose the condition that the first order derivatives must match at the overlapping points.

$$e^k_{\mathcal{V},xxxx} = 0 \qquad e^k_{\mathcal{W},xxxx} = 0$$

$$e^k_{\mathcal{V}} = u - \mathcal{V}^k \qquad e^k_{\mathcal{W}} = u - \mathcal{W}^k$$

$$e^k_{\mathcal{V},x}(0) = 0 \qquad e^k_{\mathcal{W}}(1) = 0$$

$$e^k_{\mathcal{V}}(0) = 0 \qquad e^k_{\mathcal{W},x}(1) = 0$$

$$e^k_{\mathcal{V}}(\beta) = e^{k-1}_{\mathcal{W}}(\beta) \qquad e^k_{\mathcal{W}}(\alpha) = e^{k-1}_{\mathcal{V}}(\alpha)$$

$$e^k_{\mathcal{V},x}(\beta) = e^{k-1}_{\mathcal{W},x}(\beta) \qquad e^k_{\mathcal{W},x}(\alpha) = e^{k-1}_{\mathcal{V},x}(\alpha)$$

These fourth order terms imply a cubic relationship between terms. We assume the following:

$$e^k_{\mathcal{V}}(x) = A^k x^3 + B^k x^2 + C^k x + D^k$$

The conditions that $e^k_{\mathcal{V}}(0) = 0$ indicate that,

$$e^k_{\mathcal{V}}(0) = D^k = 0.$$

Additionally, our first order derivative term states,

$$e^k_{\mathcal{V},x}(0) = C^k = 0.$$

Now, the general form for this term is,

$$e_{\mathcal{V}}^k(x) = A^k x^3 + B^k x^2$$

and it's first order derivative term is,

$$e_{\mathcal{V},x}^k(x) = 3A^k x^2 + 2B^k x.$$

In a similar manner to the above, we define the error on the $\mathcal{W}$ domain as

$$e_{\mathcal{W}}^k(x) = P^k(x-1)^3 + Q^k(x-1)^2 + R(x-1) + S.$$

The boundary conditions necessitate that $R = S = 0$. We now have the corresponding function and its derivative form,

$$e_{\mathcal{W}}^k(x) = P^k(x-1)^3 + Q^k(x-1)^2.$$
$$e_{\mathcal{W}}^k(x) = 3P^k(x-1)^2 + 2Q^k(x-1).$$

Thanks to the transmission conditions defined above, we can express these equations in matrix form at $x = \beta$ as,

$$\begin{pmatrix} \beta^3 & \beta^2 \\ 3\beta^2 & 2\beta \end{pmatrix} \begin{pmatrix} A^k \\ B^k \end{pmatrix} = \begin{pmatrix} (\beta-1)^3 & (\beta-1)^2 \\ 3(\beta-1)^2 & 2(\beta-1) \end{pmatrix} \begin{pmatrix} P^{k-1} \\ Q^{k-1} \end{pmatrix}.$$

As well can say that at $x = \alpha$,

$$\begin{pmatrix} (\alpha-1)^3 & (\alpha-1)^2 \\ 3(\alpha-1)^2 & 2(\alpha-1) \end{pmatrix} \begin{pmatrix} P^k \\ Q^k \end{pmatrix} = \begin{pmatrix} \alpha^3 & \alpha^2 \\ 3\alpha^2 & 2\alpha \end{pmatrix} \begin{pmatrix} A^{k-1} \\ B^{k-1} \end{pmatrix}.$$

Rearranging for each term we can see that,

$$\begin{pmatrix} A^k \\ B^k \end{pmatrix} = \begin{pmatrix} \beta^3 & \beta^2 \\ 3\beta^2 & 2\beta \end{pmatrix}^{-1} \begin{pmatrix} (\beta-1)^3 & (\beta-1)^2 \\ 3(\beta-1)^2 & 2(\beta-1) \end{pmatrix} \begin{pmatrix} P^{k-1} \\ Q^{k-1} \end{pmatrix},$$

and,

$$\begin{pmatrix} P^k \\ Q^k \end{pmatrix} = \begin{pmatrix} (\alpha-1)^3 & (\alpha-1)^2 \\ 3(\alpha-1)^2 & 2(\alpha-1) \end{pmatrix}^{-1} \begin{pmatrix} \alpha^3 & \alpha^2 \\ 3\alpha^2 & 2\alpha \end{pmatrix} \begin{pmatrix} A^{k-1} \\ B^{k-1} \end{pmatrix}.$$

Hence, in general we have the two major governing matrix vector equations,

$$\begin{pmatrix} A^k \\ B^k \end{pmatrix} =$$

$$\begin{pmatrix} \beta^3 & \beta^2 \\ 3\beta^2 & 2\beta \end{pmatrix}^{-1} \begin{pmatrix} (\beta-1)^3 & (\beta-1)^2 \\ 3(\beta-1)^2 & 2(\beta-1) \end{pmatrix} \begin{pmatrix} (\alpha-1)^3 & (\alpha-1)^2 \\ 3(\alpha-1)^2 & 2(\alpha-1) \end{pmatrix}^{-1} \begin{pmatrix} \alpha^3 & \alpha^2 \\ 3\alpha^2 & 2\alpha \end{pmatrix} \begin{pmatrix} A^{k-2} \\ B^{k-2} \end{pmatrix},$$

$$\begin{pmatrix} P^k \\ Q^k \end{pmatrix} =$$

$$\begin{pmatrix} (\alpha-1)^3 & (\alpha-1)^2 \\ 3(\alpha-1)^2 & 2(\alpha-1) \end{pmatrix}^{-1} \begin{pmatrix} \alpha^3 & \alpha^2 \\ 3\alpha^2 & 2\alpha \end{pmatrix} \begin{pmatrix} \beta^3 & \beta^2 \\ 3\beta^2 & 2\beta \end{pmatrix}^{-1} \begin{pmatrix} (\beta-1)^3 & (\beta-1)^2 \\ 3(\beta-1)^2 & 2(\beta-1) \end{pmatrix} \begin{pmatrix} P^{k-2} \\ Q^{k-2} \end{pmatrix}.$$

To check the convergence of this scheme, we simply need to check that the spectral radius, ρ, the of matrix,

$$\mathcal{M} = \begin{pmatrix} \beta^3 & \beta^2 \\ 3\beta^2 & 2\beta \end{pmatrix}^{-1} \begin{pmatrix} (\beta-1)^3 & (\beta-1)^2 \\ 3(\beta-1)^2 & 2(\beta-1) \end{pmatrix} \begin{pmatrix} (\alpha-1)^3 & (\alpha-1)^2 \\ 3(\alpha-1)^2 & 2(\alpha-1) \end{pmatrix}^{-1} \begin{pmatrix} \alpha^3 & \alpha^2 \\ 3\alpha^2 & 2\alpha \end{pmatrix}$$

is strictly less than 1. This is to say $\rho(\mathcal{M}) < 1$.

Due to the complex nature of this problem, we check this term numerically with data provided below. We consider the case where $0 < \alpha$, $\alpha < \beta$, and $\beta < 1$.

In Figure 3.2 we can see that the spectral radius is confined to be < 1 on the entire domain of interest and peaks at the line $\beta = \alpha$. As well, we consider the two Figures 3.3,3.4. These demonstrate that the spectral radius, rho is increasing toward the local extreme extreme on the centre diagonal line. In Figure 3.3 we see that the function is approximately 0 everywhere except near the diagonal where it is negative, meaning that as we move further "right" the contraction factor decreases. Similarly, in Figure 3.4 we can see the function is nearly 0 everywhere but for a positive value near the diagonal suggesting that the function increases as we move "up" in the alpha direction.

All of this indicates that the function ρ is locally maximum on $0 < \alpha$, $\alpha < \beta$, and $\beta < 1$ near $\beta = \alpha$, $\rho \approx 1$. However, since $\alpha < \beta$ then we have that $\rho < 1$ for all possible configurations in this region. This suggests that

$$\lim_{k\to\infty} \begin{pmatrix} A^k \\ B^k \end{pmatrix} = \lim_{k\to\infty} \begin{pmatrix} P^k \\ Q^k \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

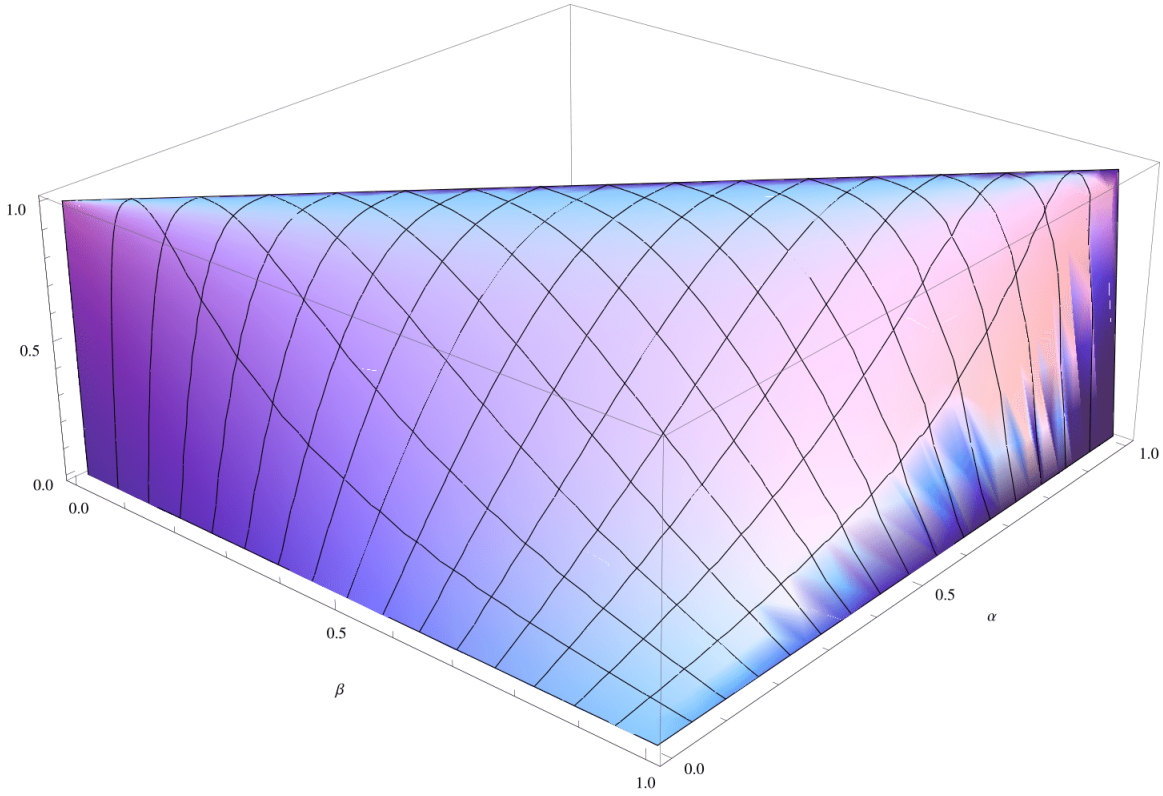


Figure 3.2: The Spectral Radius of the contraction matrix $\mathcal{M}$.

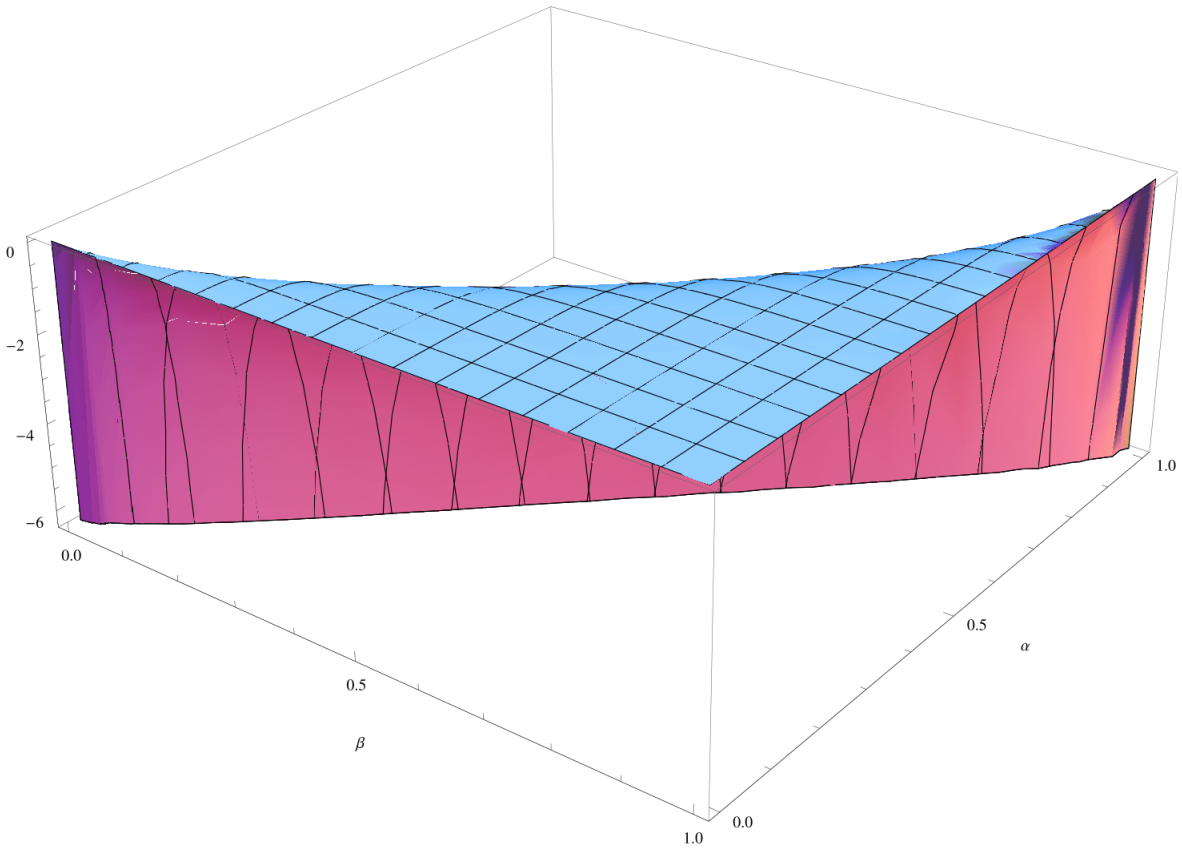


Figure 3.3: The derivative of the Spectral Radius of the contraction matrix $\mathcal{M}$ in the β direction.

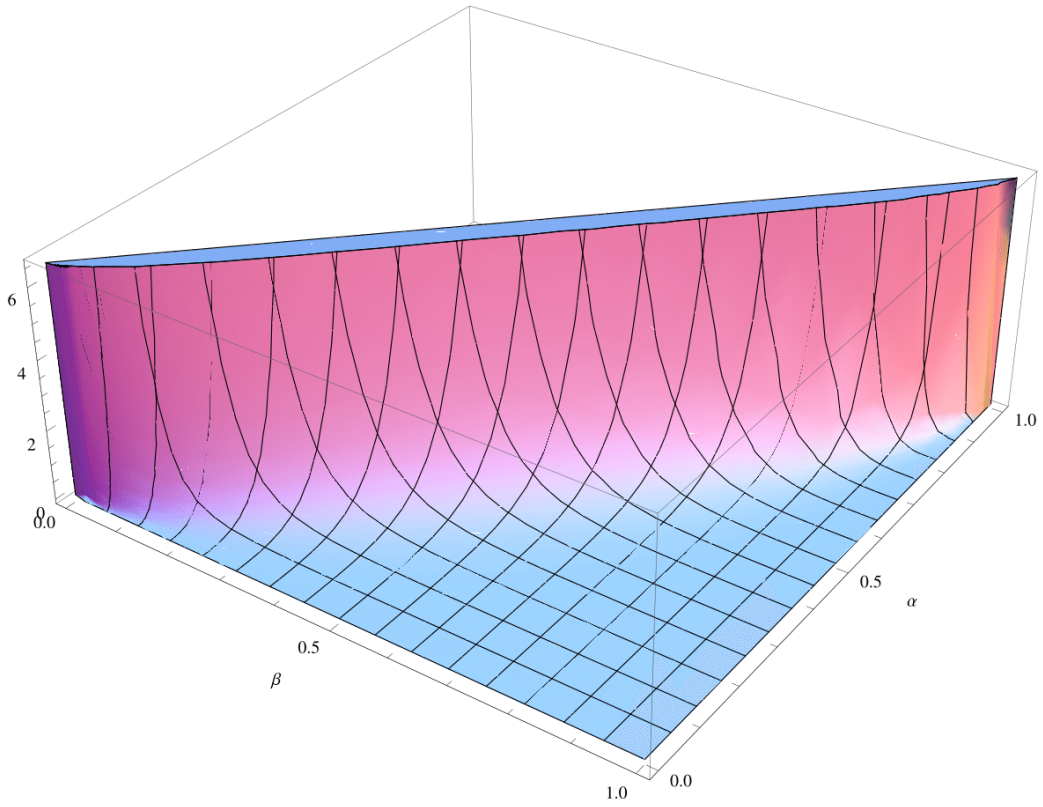


Figure 3.4: The derivative of the Spectral Radius of the contraction matrix $\mathcal{M}$ in the α direction.

Hence, we have

$$\lim_{k\to\infty} \begin{pmatrix} e_{\mathcal{V}}^k \\ e_{\mathcal{V},x}^k \end{pmatrix} = \lim_{k\to\infty} \begin{pmatrix} x^3 & x^2 \\ 3x^2 & 2x \end{pmatrix} \begin{pmatrix} A^k \\ B^k \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

and,

$$\lim_{k\to\infty} \begin{pmatrix} e_{\mathcal{W}}^k \\ e_{\mathcal{W},x}^k \end{pmatrix} = \lim_{k\to\infty} \begin{pmatrix} (x-1)^3 & (x-1)^2 \\ 3(x-1)^2 & 2(x-1) \end{pmatrix} \begin{pmatrix} P^k \\ Q^k \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

Thus we have a convergent method to numerical precision. □

3.2 Domain Decomposition of the Smoothed Mesh Generating Function

Since the basic structure of the smoothed mesh generation technique discussed is similar to the standard BVP method, we can attempt to define the domain decomposition of this method. For reference, the "G-Vector" and Jacobian terms for the standard BVP are as follows:

$$\mathcal{G}(x^k) = \begin{pmatrix} M\left(\frac{x_2+x_1}{2}\right)(x_2-x_1) - M\left(\frac{x_1+x_0}{2}\right)(x_1-x_0) \\ M\left(\frac{x_3+x_2}{2}\right)(x_3-x_2) - M\left(\frac{x_2+x_1}{2}\right)(x_2-x_1) \\ \vdots \\ M\left(\frac{x_N+x_{N-1}}{2}\right)(x_N-x_{N-1}) - M\left(\frac{x_{N-1}+x_{N-2}}{2}\right)(x_{N-1}-x_{N-2}) \end{pmatrix}$$

$$\mathcal{J}(x^k) = \begin{pmatrix} \frac{\partial \mathcal{G}_1}{\partial x_1} & \frac{\partial \mathcal{G}_1}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_1}{\partial x_{N-1}} \\ \frac{\partial \mathcal{G}_2}{\partial x_1} & \frac{\partial \mathcal{G}_2}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_2}{\partial x_{N-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{G}_{N-1}}{\partial x_1} & \frac{\partial \mathcal{G}_{N-1}}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_{N-1}}{\partial x_{N-1}} \end{pmatrix}.$$

Since the Jacobian relies only on the construction of the "G-Vector" we know it may be left the same. As defined in Chapter 2, (2.25), the new non-linear equations are defined as:

$$\frac{1}{M_{j+\frac{1}{2}}}\left[y_{j+\frac{1}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{j+\frac{3}{2}} - 2y_{j+\frac{1}{2}} + y_{j-\frac{1}{2}}\right)\right] - \frac{1}{M_{j-\frac{1}{2}}}\left[y_{j-\frac{1}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{j+\frac{1}{2}} - 2y_{j-\frac{1}{2}} + y_{j-\frac{3}{2}}\right)\right] = 0. \quad (3.2)$$

Thus our new "G-Vector" is defined for $j \in \{1, 2, \ldots, N-1\}$ and with the boundary conditions that $y_{\frac{-1}{2}} = y_{\frac{1}{2}}$ and $y_{N-\frac{1}{2}} = y_{N+\frac{1}{2}}$

$$\mathcal{G}_s(x^k) = \begin{pmatrix} \frac{1}{M_{\frac{3}{2}}}\left[y_{\frac{3}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{\frac{5}{2}} - 2y_{\frac{3}{2}} + y_{\frac{1}{2}}\right)\right] \\ -\frac{1}{M_{\frac{-1}{2}}}\left[y_{\frac{1}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{\frac{3}{2}} - y_{\frac{1}{2}}\right)\right] \\ \vdots \\ \frac{1}{M_{j+\frac{1}{2}}}\left[y_{j+\frac{1}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{j+\frac{3}{2}} - 2y_{j+\frac{1}{2}} + y_{j-\frac{1}{2}}\right)\right] \\ -\frac{1}{M_{j-\frac{1}{2}}}\left[y_{j-\frac{1}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{j+\frac{1}{2}} - 2y_{j-\frac{1}{2}} + y_{j-\frac{3}{2}}\right)\right] \\ \vdots \\ \frac{1}{M_{N-\frac{1}{2}}}\left[y_{N-\frac{1}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(-y_{N-\frac{1}{2}} + y_{N-\frac{3}{2}}\right)\right] \\ -\frac{1}{M_{N-\frac{3}{2}}}\left[y_{N-\frac{3}{2}} - \frac{1}{\beta^2\Delta\xi^2}\left(y_{N-\frac{1}{2}} - 2y_{N-\frac{3}{2}} + y_{N-\frac{5}{2}}\right)\right] \end{pmatrix}$$

Here, $y_{j+\frac{1}{2}} = \frac{\Delta\xi}{x_{j+1}-x_j}$ as defined in Chapter 2. Continuing on from this construction, we build the new "smooth" Jacobian matrix, $\mathcal{J}_s(x^k)$ as before:

$$\mathcal{J}(x^k) = \begin{pmatrix} \frac{\partial \mathcal{G}_{1,s}}{\partial x_1} & \frac{\partial \mathcal{G}_{1,s}}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_{1,s}}{\partial x_{N-1}} \\ \frac{\partial \mathcal{G}_{2,s}}{\partial x_1} & \frac{\partial \mathcal{G}_{2,s}}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_{2,s}}{\partial x_{N-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{G}_{N-1,s}}{\partial x_1} & \frac{\partial \mathcal{G}_{N-1,s}}{\partial x_2} & \cdots & \frac{\partial \mathcal{G}_{N-1,s}}{\partial x_{N-1}} \end{pmatrix}.$$

In the original case of the "un-smoothed" monitor function, the Jacobian matrix is usually of a tridiagonal form. In the matrix defined above the the smoothed case

will not be of this form, specifically it will be a penta-diagonal matrix. The tridiagonal form lends well for optimized methods for solving these systems. Penta-diagonal systems have fast solvers available, however they are very complex and require specifically definitions. This layer of complexity suggests that this scheme is inherently more difficult to solve and will require more computational load for numerical solvers.

From here, we progress in the same way as outlined in the above section on standard domain decomposition techniques defined on the usual monitor function. This includes the same transmission conditions being to match up the pseudo-boundary points that are imposed on the different subdomains.

In addition, the nature of the new $\mathcal{G}$ and $\mathcal{J}$ terms also require that we have larger overlaps on the subdomains. In the previous scheme we simply needed the overlap to be non-trivial that is the boundary of one subdomain must be proper interior to the neighbouring subdomain. In the smoothed case, we require more information about the neighbour points being considered. Thus, we provide a larger overlap of 3 points or more to ensure that the numerical scheme will properly execute.

The basic form of the iteration scheme for the "smoothed" mesh function follows a similar method as the "un-smoothed" version. This is expected as the function is simply a standard boundary value problem with a more complex mesh generating function. We solve the iteration as discussed in the form where our iterative scheme

is:

$$x_{k+1} = x_k - \mathcal{J}_k^{-1}\mathcal{G}_k.$$

This is using the above defined $\mathcal{J}$ and $\mathcal{G}$.

For the definition of this new mesh function, we want to match the boundaries, in the single domain case, as,

$$\left[y_{N-\frac{1}{2}}\right] = \left[y_{N+\frac{1}{2}}\right].$$

If we apply these conditions to the domain decomposition technique, then we implicitly get that the first order approximation of the derivatives must match as

$$\frac{\xi}{x_j - x_{j-1}} = \frac{\xi}{x_{j+1} - x_j}$$

is equivalent to matching their first order approximations. Thus, we proceed to perform the newton iteration 2.11 on each subdomain and to match the corresponding boundary values of the function and assume the derivative is implicitly matched via the boundary condition above:

$$\mathcal{V}^k(\beta) = \mathcal{W}^{k-1}(\beta)$$

$$\mathcal{V}^{k+1} = \mathcal{V}^k - \mathcal{J}_{\mathcal{V},k}^{-1}\mathcal{G}_{\mathcal{V},k}$$

$$\mathcal{W}^k(\alpha) = \mathcal{V}^{k-1}(\alpha)$$

$$\mathcal{W}^{k+1} = \mathcal{W}^k - \mathcal{J}_{\mathcal{W},k}^{-1}\mathcal{G}_{\mathcal{W},k}$$

This proceeds until the largest Newton iteration is less than some defined tolerance tol.

$$\|\mathcal{J}_k^{-1}\mathcal{G}_k\|_\infty < tol.$$

The above process represents the method to implement the Schwarz Algorithm we have used in all further "smoothed" mesh experiments.

In Chapter 4, we will see the numerical results obtained from the use of the previously outlined methods for standard mesh generation, (2.7), and the smoothed mesh generating function, (2.20). In addition to standard single domain solutions to each of these cases we will examine the effect of domain decomposition through increasing number of subdomains and changing the size of the overlap between consecutive subdomains.

Chapter 4

Numerics

In this Chapter we will outline and describe the process of studying each of the boundary value problem methods outlined in Chapter 2, for the standard,(2.7), and and smoothed, (2.20), mesh generating functions. We will consider simple "proof-of-concept" monitor functions, $M(x)$, as well as more difficult functions to test the robustness of the methods employed. Domain decomposition techniques are applied to each of the model problems as well to demonstrate the benefit of parallelization and multi-core processing.

4.1 Single Domain Solver

We first consider the simple case of a single domain. This study is used as a proof of concept and to demonstrate the effectiveness of this method to approximate the exact solution. We employ the discretization to the entire domain, $\mathcal{D}$. All numerical results provided are found using a grid spacing of $\delta\xi = 0.1$ unless otherwise stated.

In Figure 4.1 it can be seen that our computational results agree strongly with the known exact solution. This is a good qualitative check for the numerical results. For a more quantitative assurance, we reference Figure 4.2, which shows the difference between our numerical solution and the known exact solution. This once again provides a strong sense of confidence in the quality of our results.

To demonstrate the quality of our study, we have to examine the convergence of the solution to our desired result. As is clear in Figure 4.3, there is a quadratic decay of our consecutive iterations. Therefore, since our error is $e_k = \mathcal{O}(e_{k-1}^2)$ this implies a quadratic convergence for our solver. We choose to measure e_k as $e_k = ||\overline{e}_k||_\infty$. Here $\overline{e}_k$ is the error vector between our numerical solution and the numerical solution at the previous step.

Additionally, we examine the solver's effectiveness when varying the mesh refine-

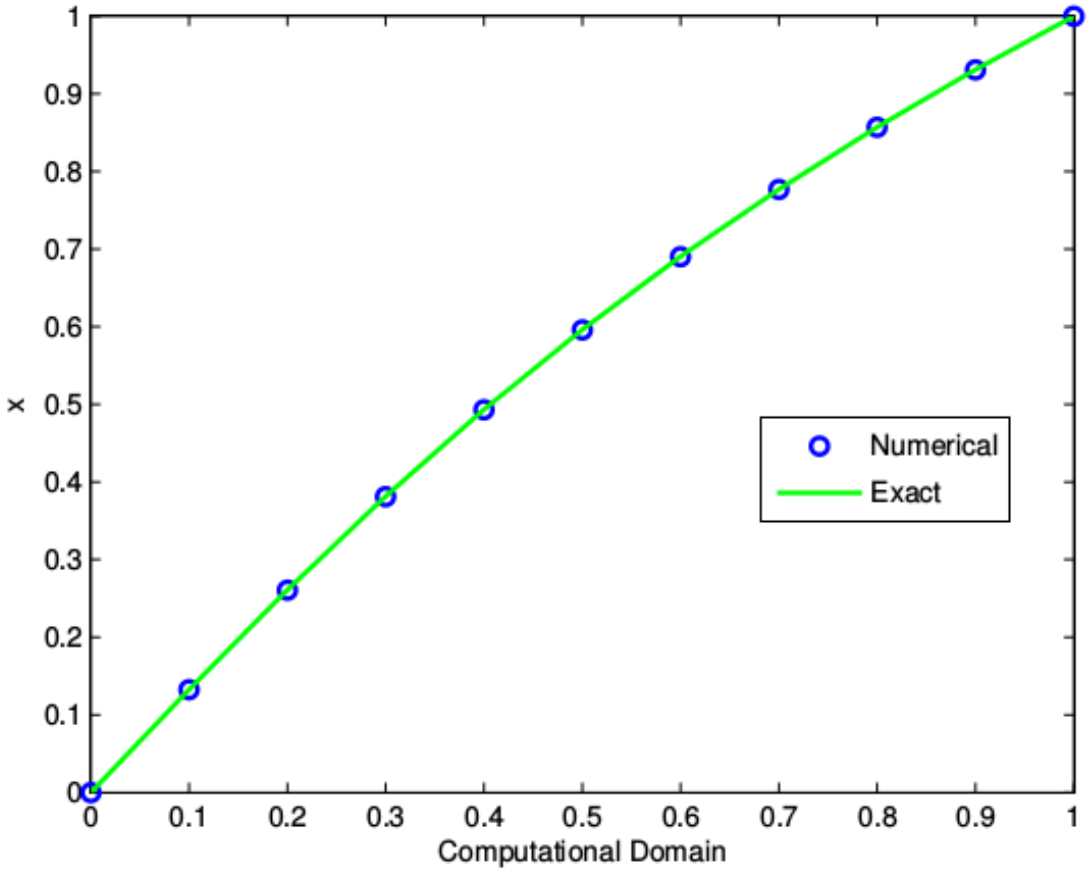


Figure 4.1: An overlay of the computed numerical solution over the known exact solution.

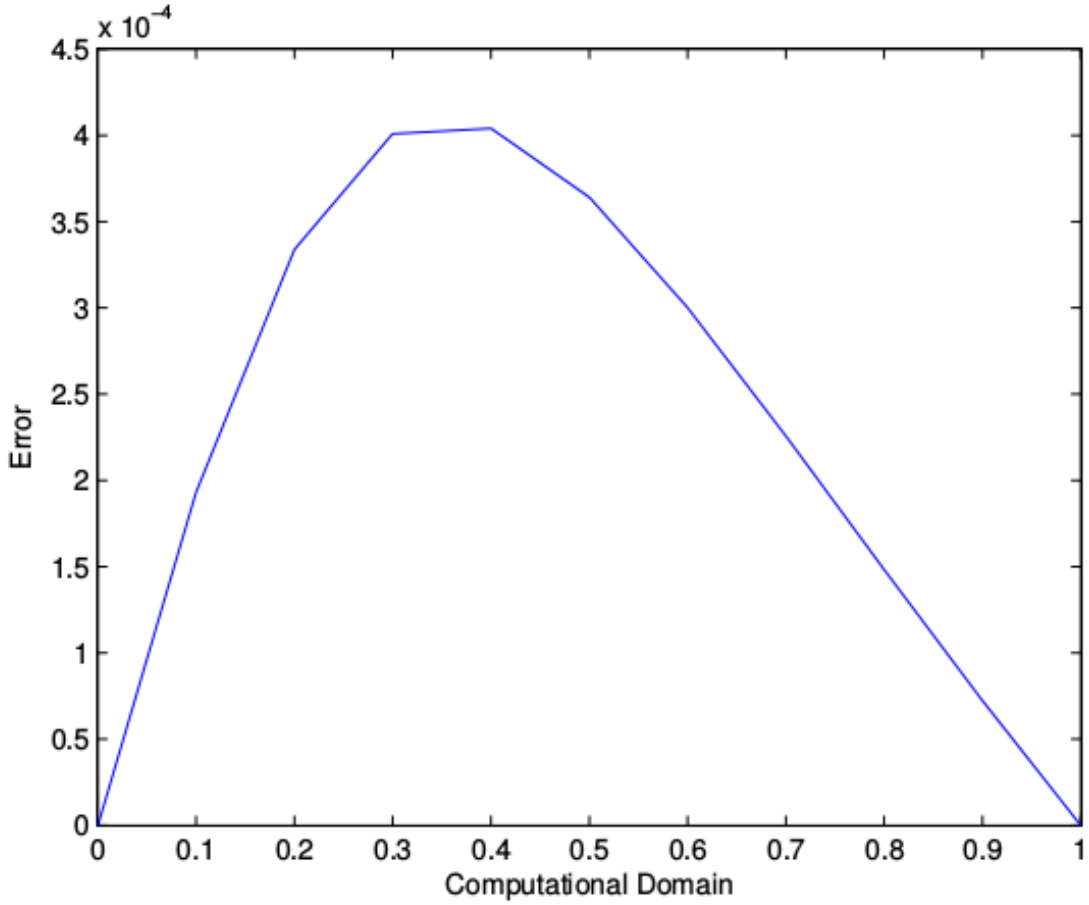


Figure 4.2: Plot of the absolute difference between the numerical solution and the exact solution for $\Delta\xi = 0.1$.

ment, Figure 4.4. We set our grid spacing at various values, $\Delta\xi$, where $\Delta\xi = 10^n$ and $n \in \{-1, \ldots, -6\}$. Through this we see a similar $\mathcal{O}(h^2)$ behaviour and can infer that as we further decrease the mesh size, we better approximate the continuous solution. This error is $||\overline{e}_k||_\infty$ at the convergence step.

4.2 Domain Decomposition

Moving on to a method of decomposing our domain, we subdivide our domain as discussed previously. For our testing purposes we will choose simple domain decompositions in order to perform our baseline calculations. We choose the subdomain decomposition $\alpha = (0.0, 0.2, 0.4)$ and $\beta = (0.4, 0.6, 1.0)$. Here, our α values indicate the left end points of our subdomain and the β values indicate the right end points, *i.e.* $\Omega_i = [\alpha_i, \beta_i]$.

Once again we begin by demonstrating the quality of our solution by overlaying our numerical results and the known solution, Figure 4.5. The convergence of our solver is not as efficient however, as can be seen in Figure 4.6. Where we previously saw quadratic convergence we now have linear in terms of the relative size of the consecutive newton iterations. This is an underlying drawback of the domain decomposition method.

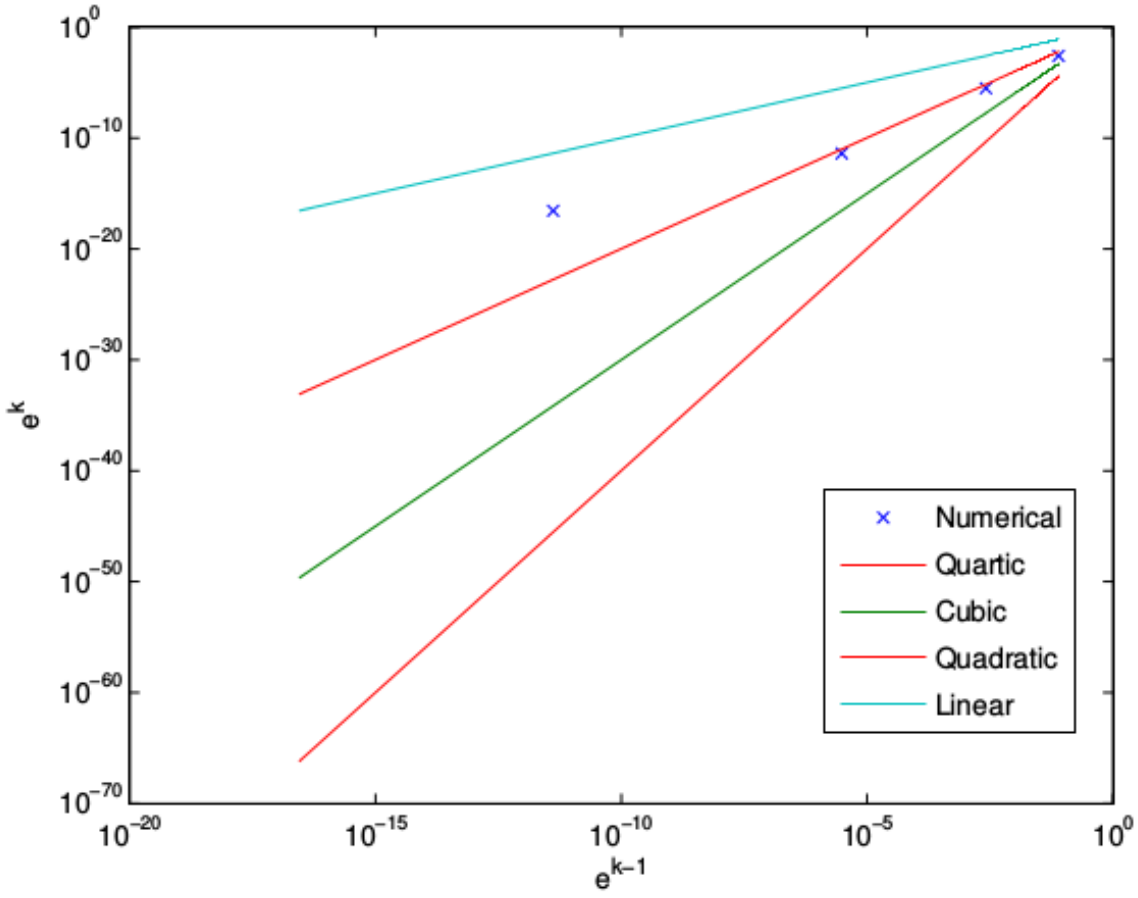


Figure 4.3: Plot of the error at the k^{th} Newton iteration, e_k, *vs.* e_{k-1}, the error at the $k-1^{th}$ Newton iteration.

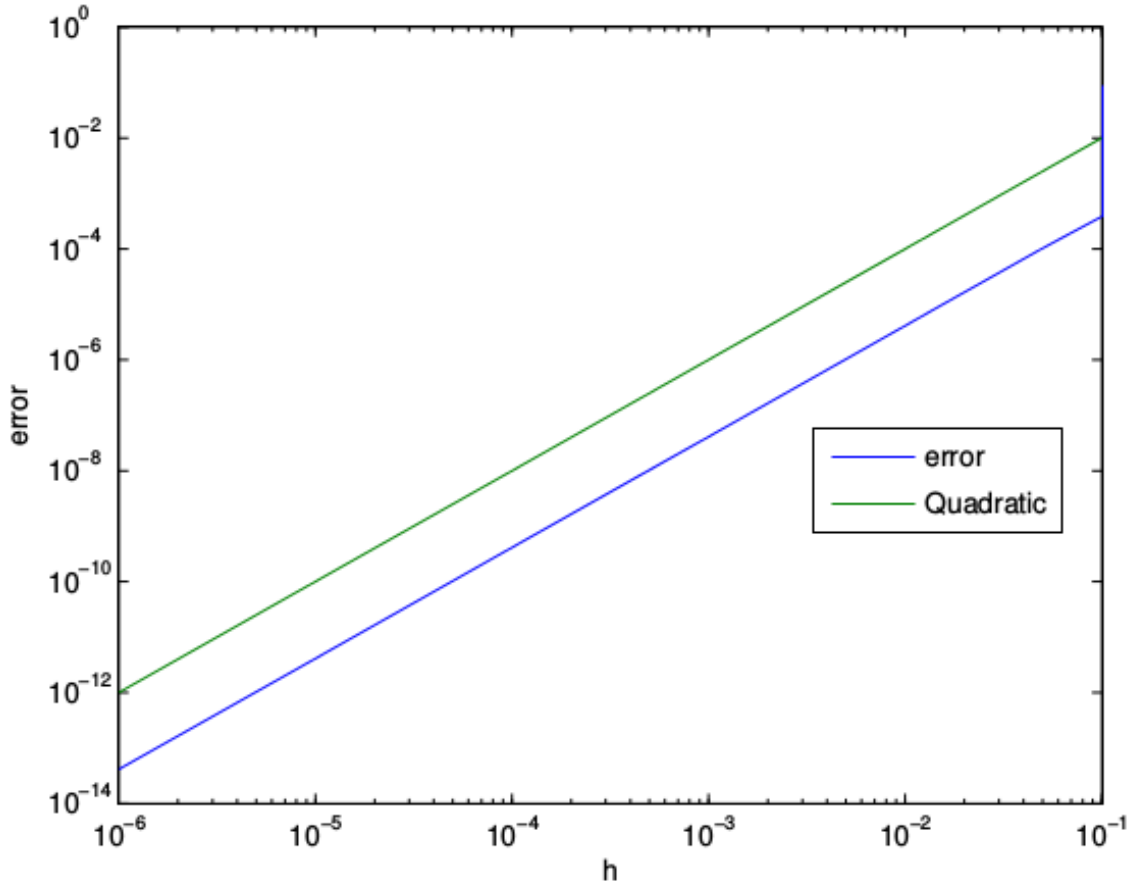


Figure 4.4: Demonstration of the convergence of the solver with e_k *vs.* h, the grid spacing. This is compared to a quadratic line.

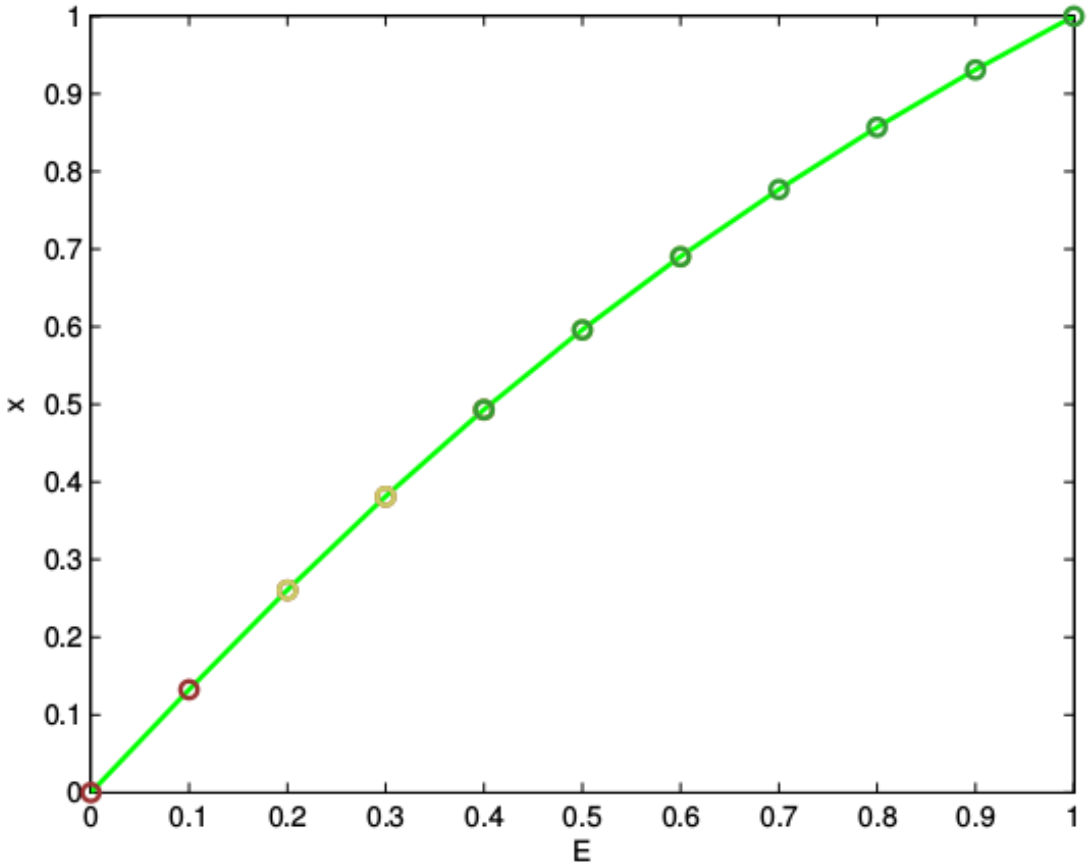


Figure 4.5: An overlay of the computed numerical solution over the known exact solution.

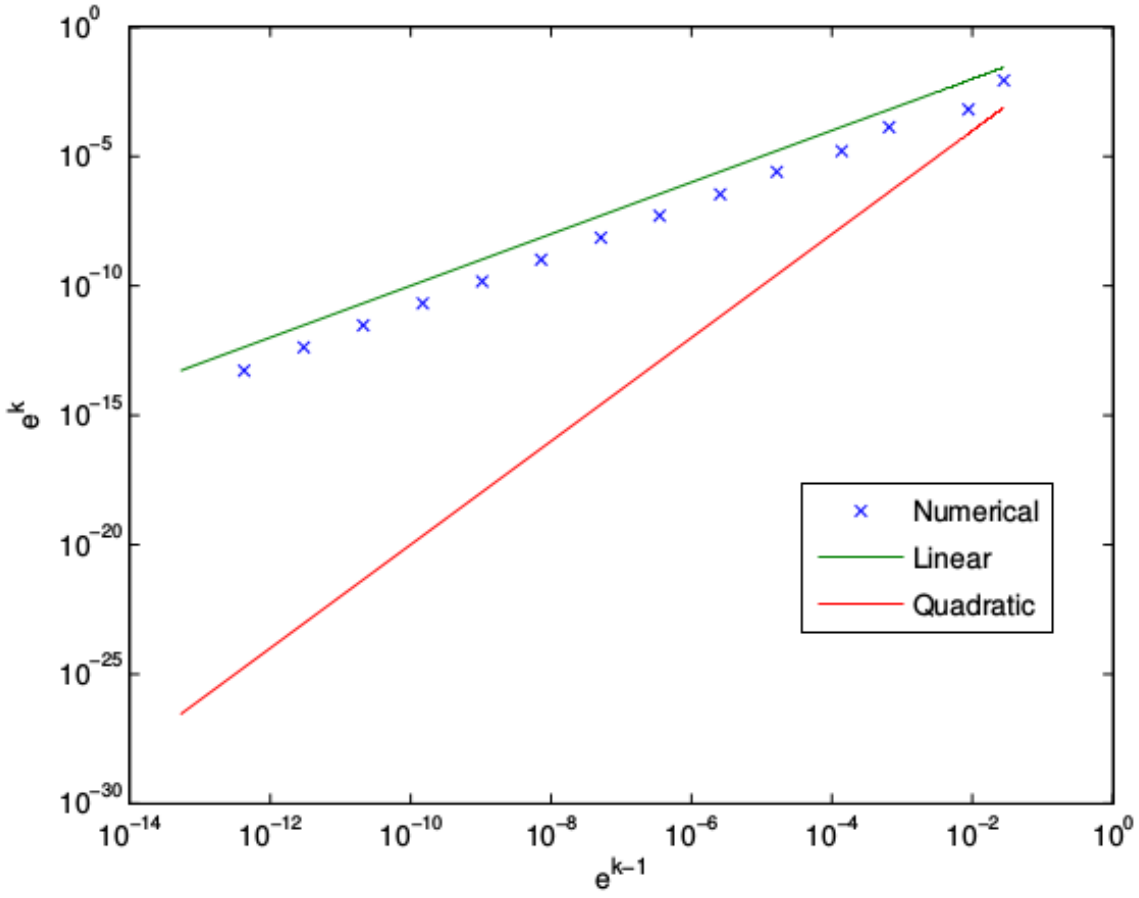


Figure 4.6: Plot of the absolute difference between the numerical solution and the exact solution.

As a point of interest, we modify the subdomain configuration in order to test the behaviour of convergence and the effect that subdomain configuration has upon this. In Figure 4.7 we have computed the solution for the subdomain configuration $\alpha = (0.0, 0.1)$ and $\beta = (0.9, 1.0)$. This provides a major overlap in the two subdomains. The resultant plot of our convergence shows a slightly higher order convergence than in the 3 subdomain case, which is the approximately quadratic convergence that we expect.

Next, we consider a "bad" decomposition of our subdomain. In this case we define the boundary points as $\alpha = (0.0, 0.7)$ and $\beta = (0.6, 1.0)$. Here we have provided little overlap between subdomains and the point of overlap occurs far from known boundary conditions. This implies that it will take several steps before information from the total domain, $\mathcal{D}$, can be used to solve on the adjacent subdomains. The outcome is a highly linear convergence suggesting very poor numerical efficiency. This method also required many more iterations to reach convergence, as can be seen in the corresponding plot, Figure [4.8].

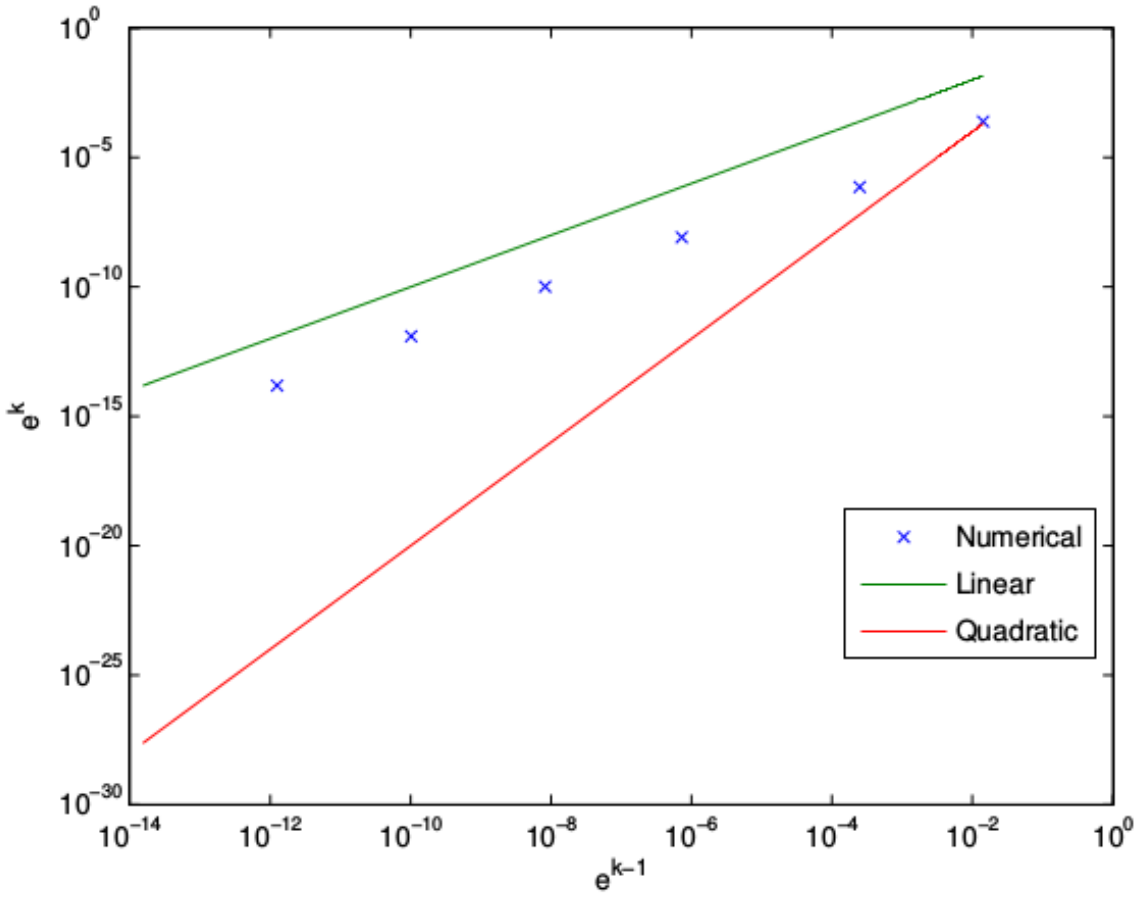


Figure 4.7: Plot of the error at the $k+1^{th}$ Newton iteration, e_{k+1}, *vs.* e_k, the error at the k^{th} Newton iteration.

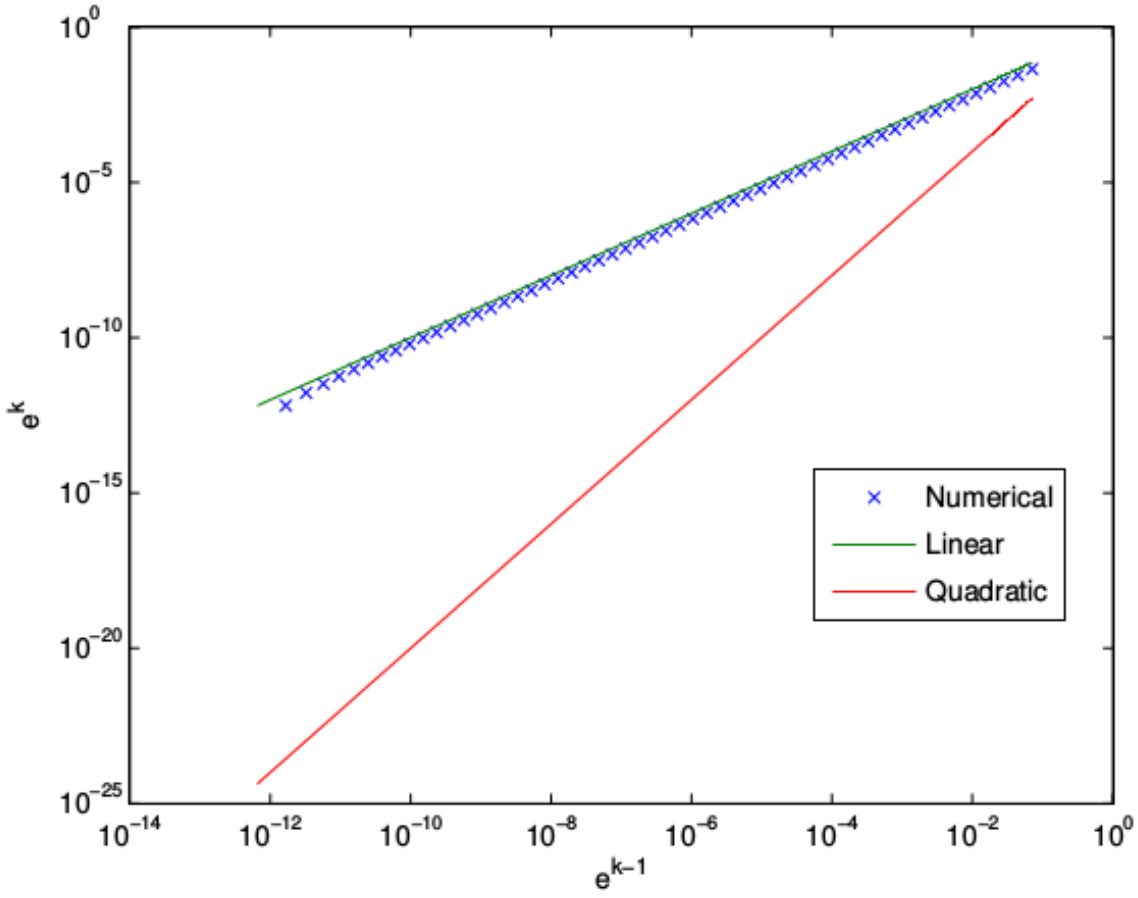


Figure 4.8: Demonstration of the convergence of the solver with e_k *vs.* h, the grid spacing. This is compared to a quadratic line.

4.3 Smoothed Mesh Density Function

4.3.1 Single Domain

For quality control purposes we examine the solver's results when applied to the simple single domain. In Figure 4.9 we have the results of our solver when modified to solve the smoothed mesh generator. Figure 4.10 is our solver when applied to our original non-smoothed case. This is a qualitative measure to confirm the accuracy of our solver's output.

Next, for the purposes of confirmation of our algorithm, we will employ the built in MATLAB solver `fSolve`. We have a qualitative sense of agreement between our solver, the smoothed solver solution, and the known exact solution. This can be seen in Figures 4.11, 4.12 which are overlays of these three solutions.

Additionally we compute the value of our system of equations, G, evaluated at the solution, x^*, which should be $G|_{x^*} = 0$.

These results suggest a high degree of quality in our results. As well as suggesting that our scheme is capable of achieving results on par with MATLAB's built in functionality.

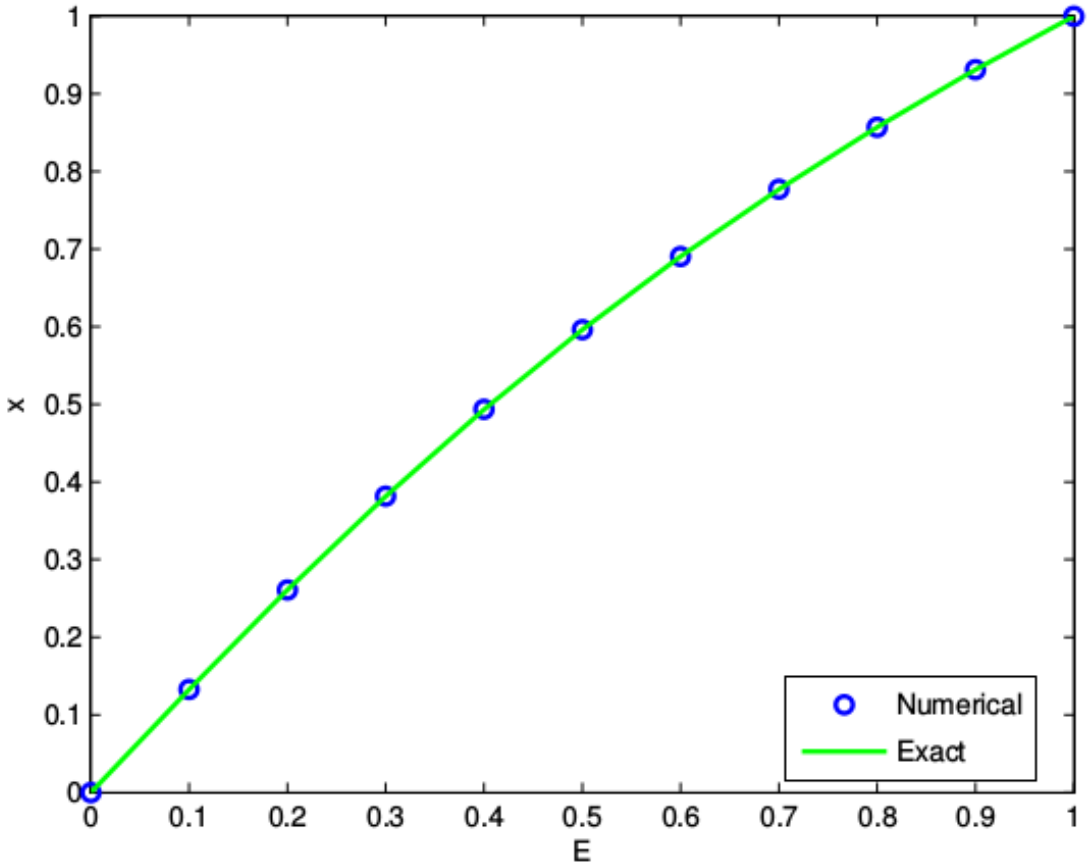


Figure 4.9: An overlay of the computed Smoothed numerical solution over the known exact solution.

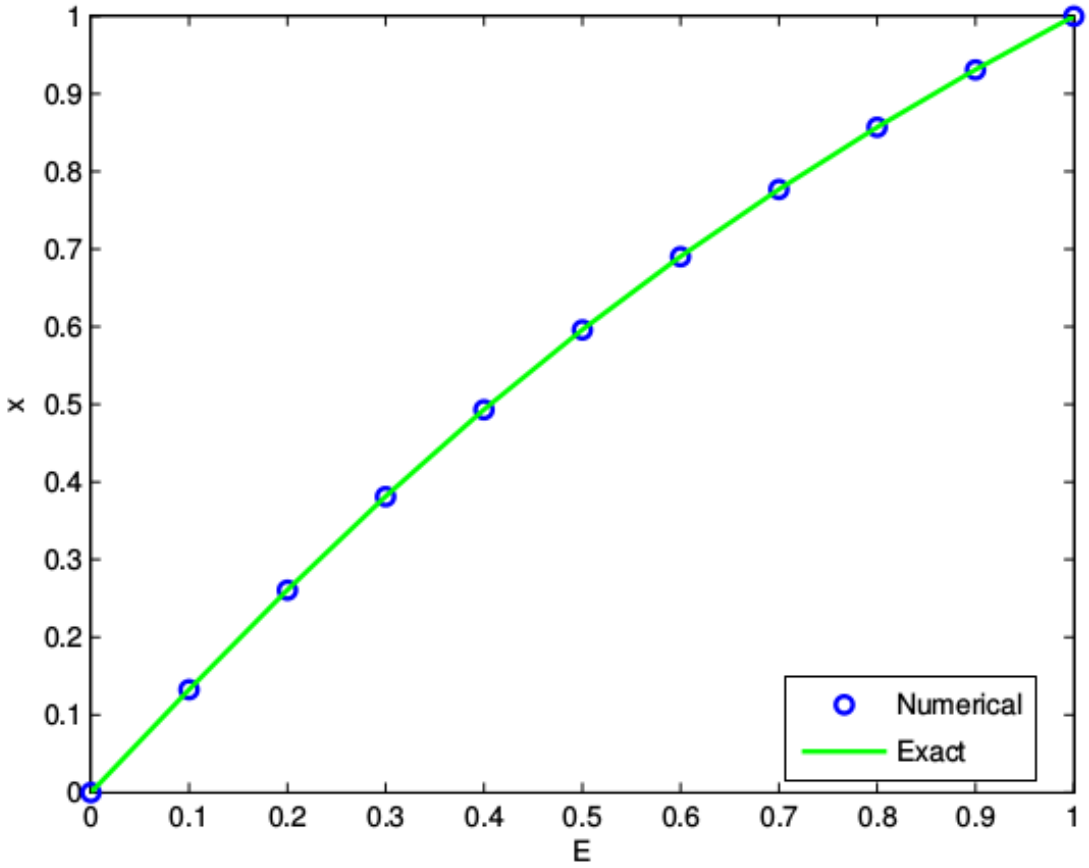


Figure 4.10: Plot of the computed numerical solution, without smoothing, over the known exact solution.

x^* Method	$G\|_{x^*}$
`fSolve`	$1.55431 \cdot 10^{-15}$
Smoothed Scheme	$1.44329 \cdot 10^{-15}$

Finally, as an additional method of proof, we examine the error between the solvers' used for our analysis. In Figure 4.13 we can see that, as seen in the original single domain case, we have a small error which tends to 0 near the boundaries. When viewing Figure 4.14 we can get a quantitative sense for the agreement between our `fSolve` and Smoothed solver results.

4.3.2 Domain Decomposition

As before, we extend our single domain solver to the multidomain system. We discretize the domains into $\alpha = (0.0, 0.2, 0.4)$ and $\beta = (0.4, 0.6, 1.0)$. Now, we evoke a similar solver as before onto each subdomain for both the Smoothed system and the non-smoothed system. We see a strong agreement between the two in Figures 4.15, 4.16.

We make use of MATLAB's built in `fSolve` once more to confirm our algorithm on these decomposed domains. Figure 4.18 shows `fSolve's` ability to handle domain decomposition when correctly employed. Figure 4.19 demonstrates the Smoothed Newton solver's ability to handle the decomposition as well. The agreement between

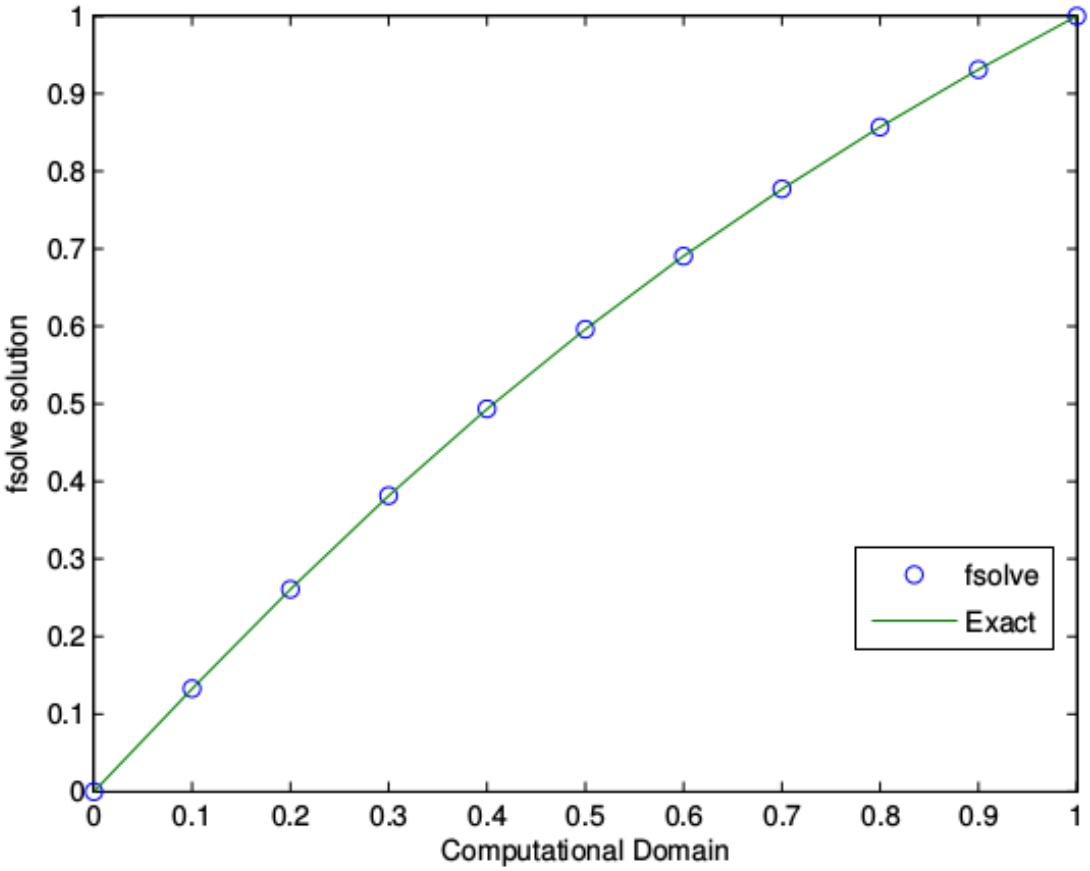


Figure 4.11: Overlay of the `fSolve` results on the known exact solution.

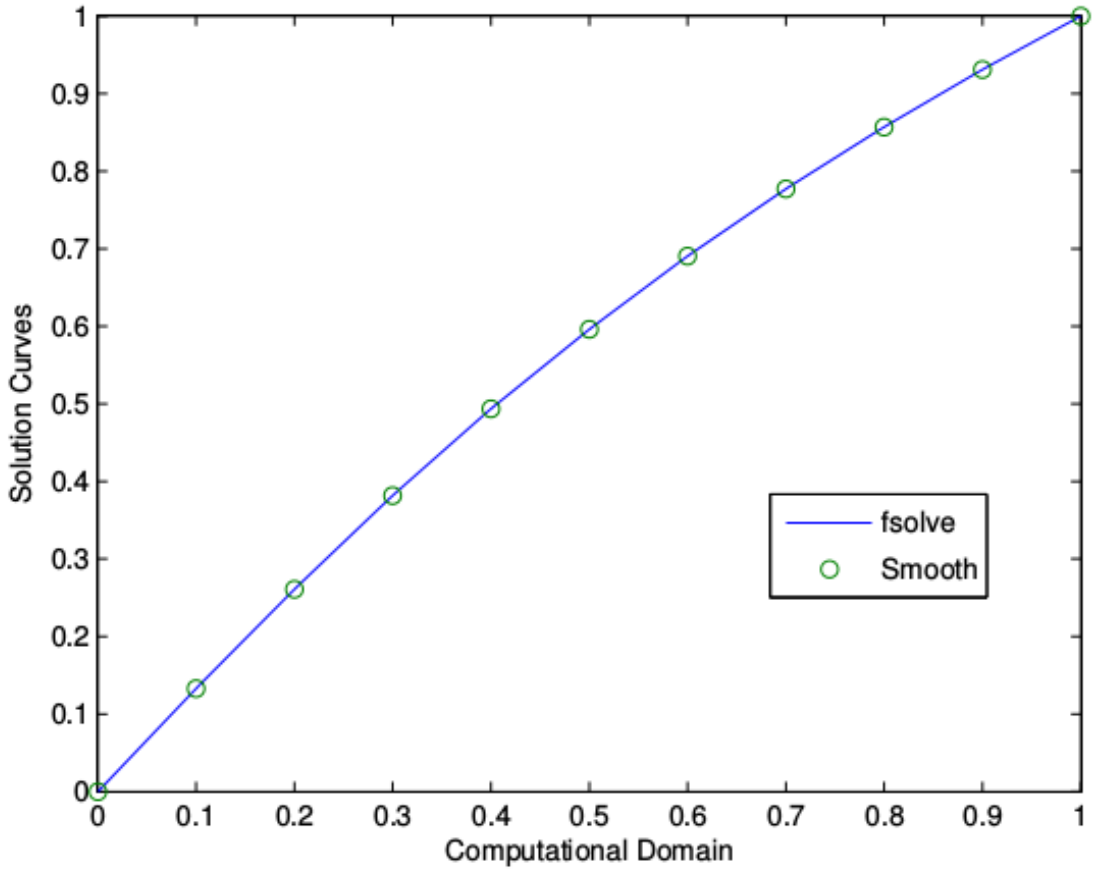


Figure 4.12: Overlay of the `fSolve` results on the Smoothed numerical solution.

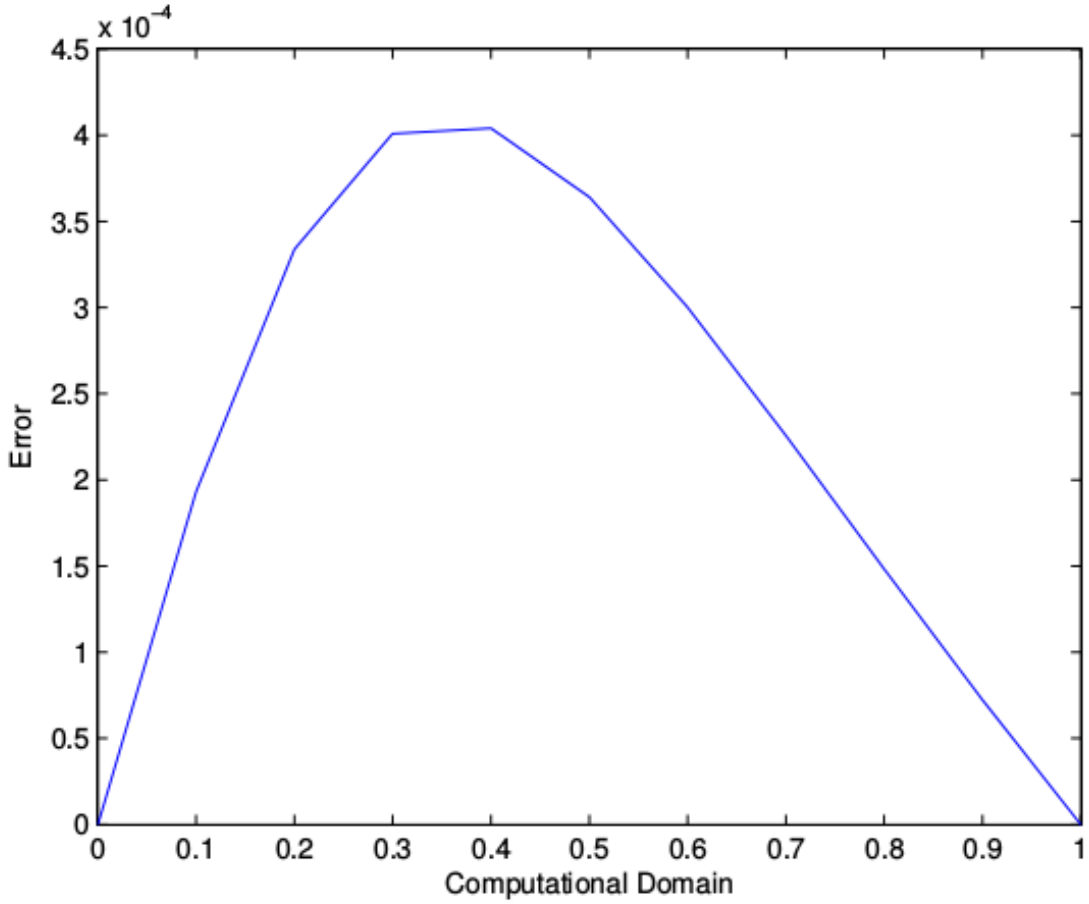


Figure 4.13: Error between Smoothed solution and known exact solution.

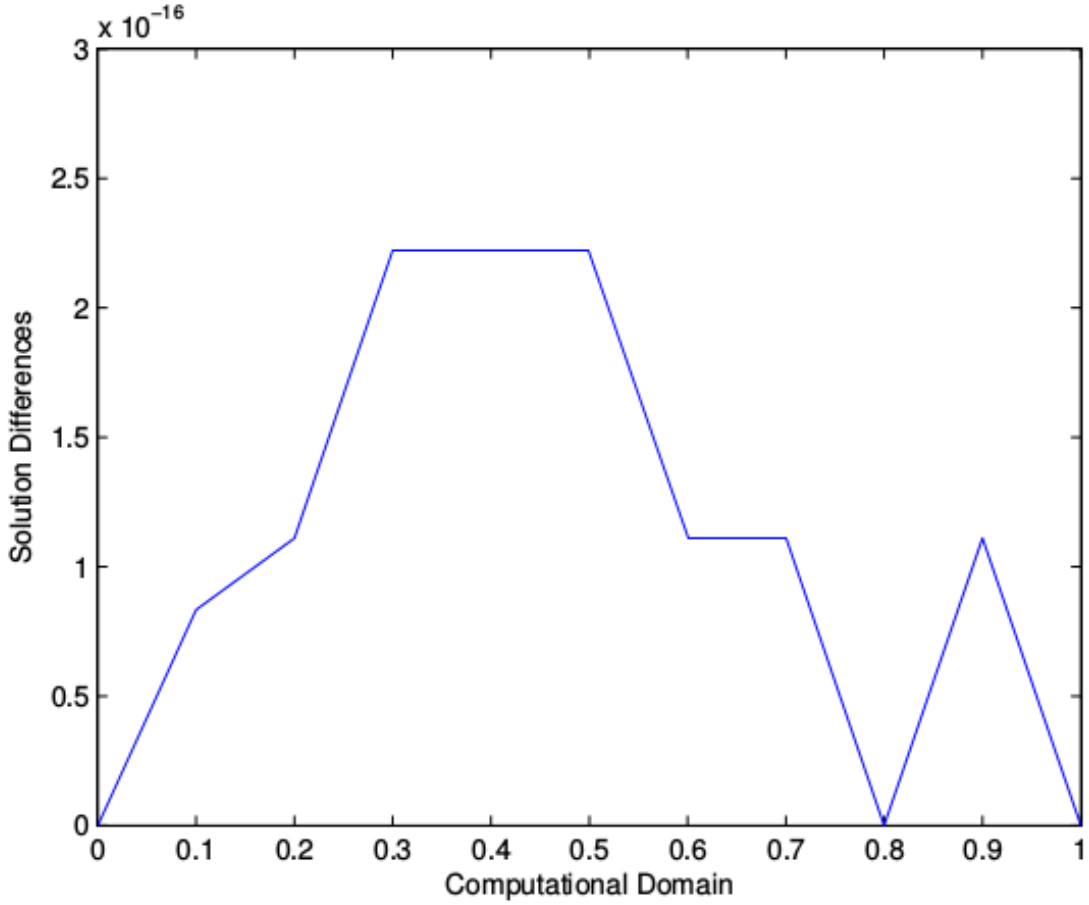


Figure 4.14: Absolute difference between MATLAB's built in fsolve function and numerically computed Smoothed solution.

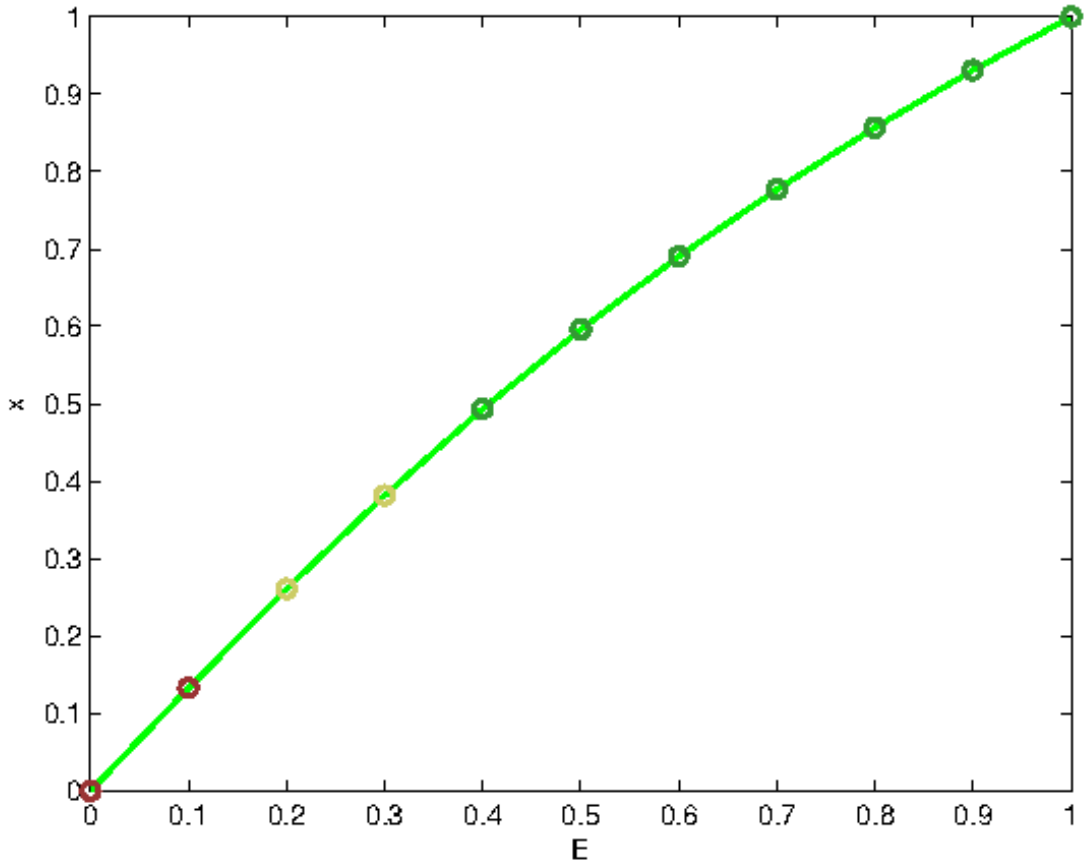


Figure 4.15: An overlay of the computed Smoothed numerical solution for all subdomains over the known exact solution.

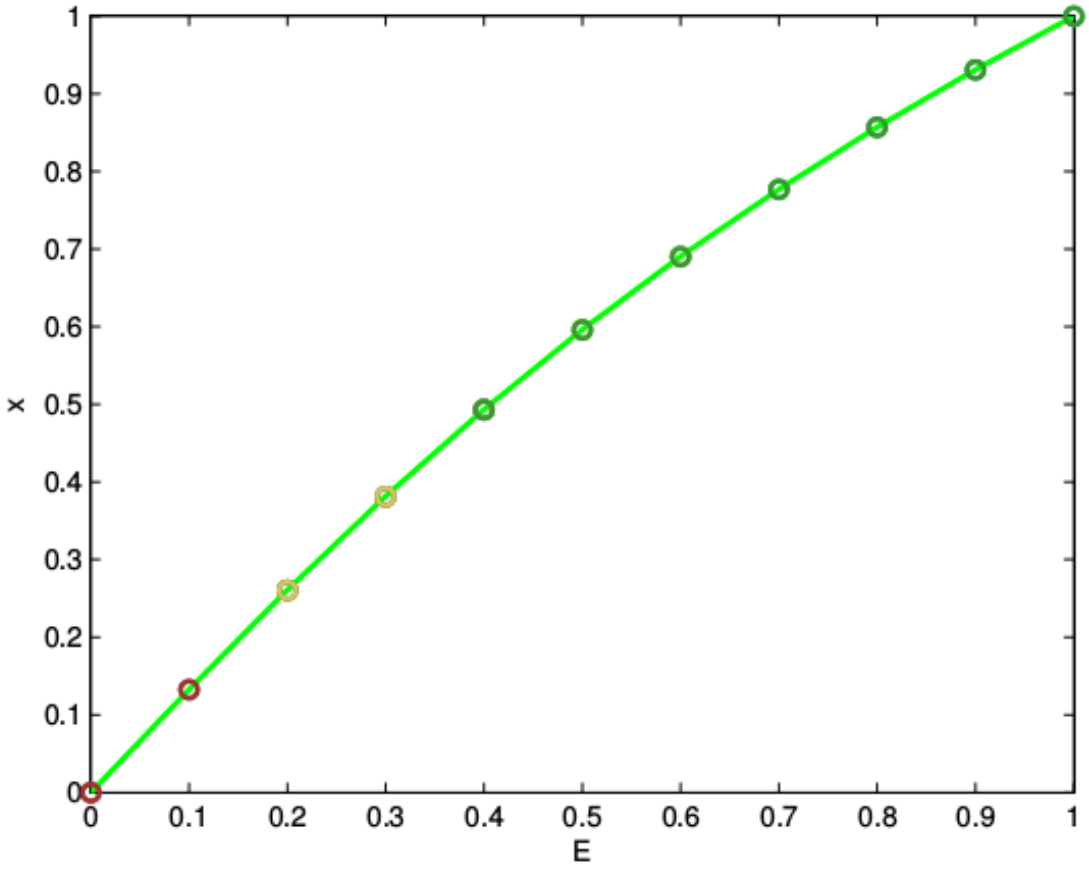


Figure 4.16: Plot of the computed numerical solution for all subdomains, without smoothing, over the known exact solution.

solves is seen in Figure 4.20. In this figure we can see that our difference between MATLAB's built in function and our own implementation strongly suggest convergence.

Once again, we compute the value of our system of equations, G, at our solutions. We do this in two ways now: Overlapping solutions on subdomains being averaged to behave like a single domain solution, and individual evaluations over each subdomain to preserve the uniqueness.

x^* Method	Type	$G\|_{x^*}$
`fSolve`	Average	$1.01888 \cdot 10^{-7}$
Smoothed Scheme	Average	$1.01887 \cdot 10^{-7}$
`fSolve`	Unique	$6.02851 \cdot 10^{-14}$
Smoothed Scheme	Unique	$4.29839 \cdot 10^{-12}$

The unique subdomain solutions are the most accurate evaluations and demonstrate the fact that these solutions are excellent solutions to our system as they are approaching machine precision. The averaged solutions are provided for simplicity and completeness.

The above data shows that while the method does not necessarily converge to the single domain solution of the problem, it does generate a very good mesh which is "close" to the single domain solution. This is a more heuristic perspective of the

generated mesh but does show that "good" meshes can be created with only simple assumptions of transmission.

Next, we examine the behaviour of the smoothed solver when applied to different numbers of subdomains. In Figure 4.17 we see cases applied to the same monitor function as before. However, we now vary the number of subdomains. We have taken the value of $\Delta\xi = 0.03$ to ensure that the domain overlaps are sufficiently large for large numbers of subdomains.

Figure 4.17 clearly demonstrates the same type of convergence that was seen for the standard BVP mesh generation technique. As the number of subdomains increases, the number of steps needed for convergence increases as well. The smoothed mesh generation technique relies on a similar transmission conditions as the standard method, that is matching the value of the "boundary" and its derivative. Thus, for more subdomains, it takes more iterations for information from the boundaries to reach points not in the subdomain containing that specific boundary. There exist methods of "coarse" correction which can reduce this issue and improve the convergence rate for large numbers of subdomains [3].

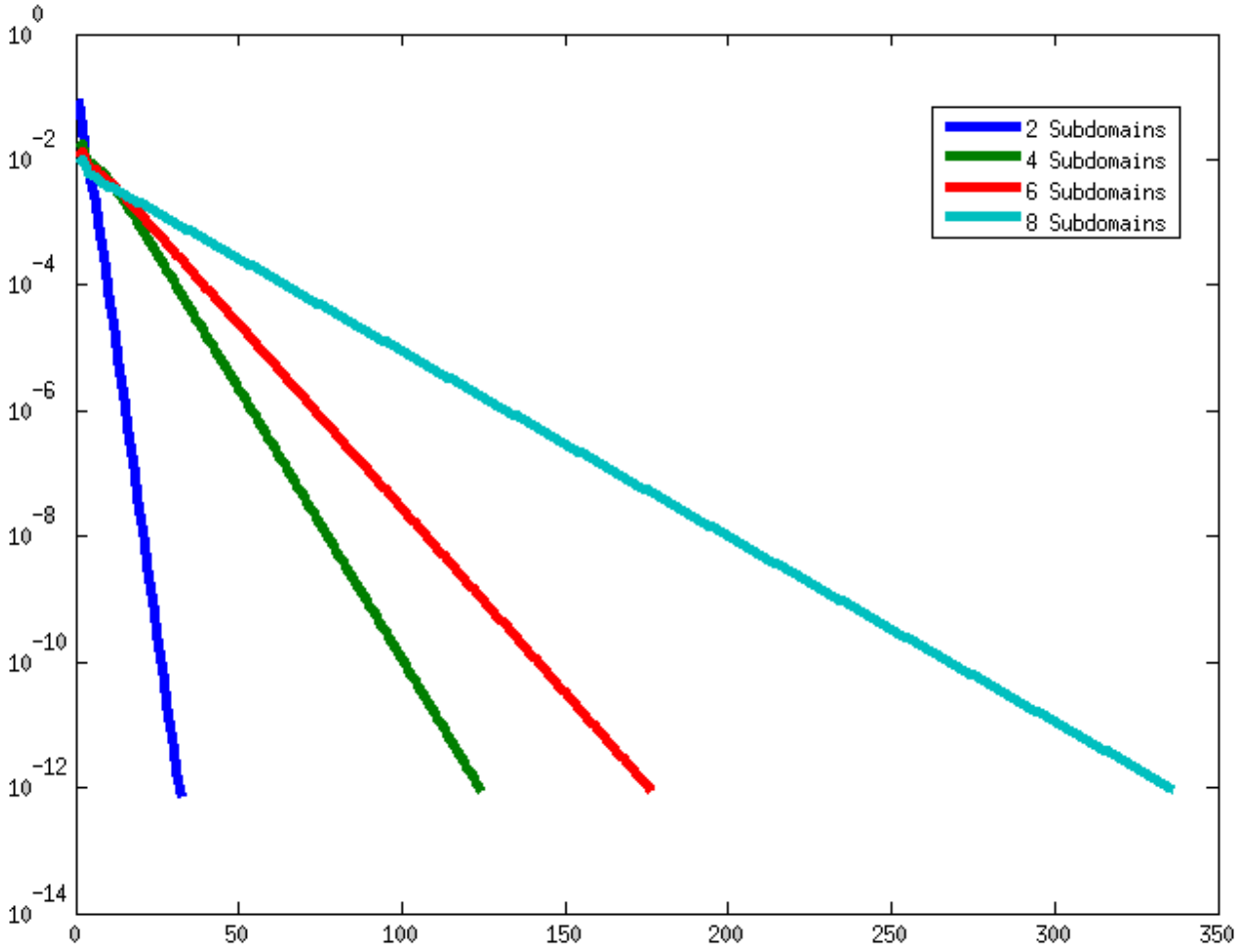


Figure 4.17: Overlay of the Newton step size for different numbers of subdomains vs the iteration number.

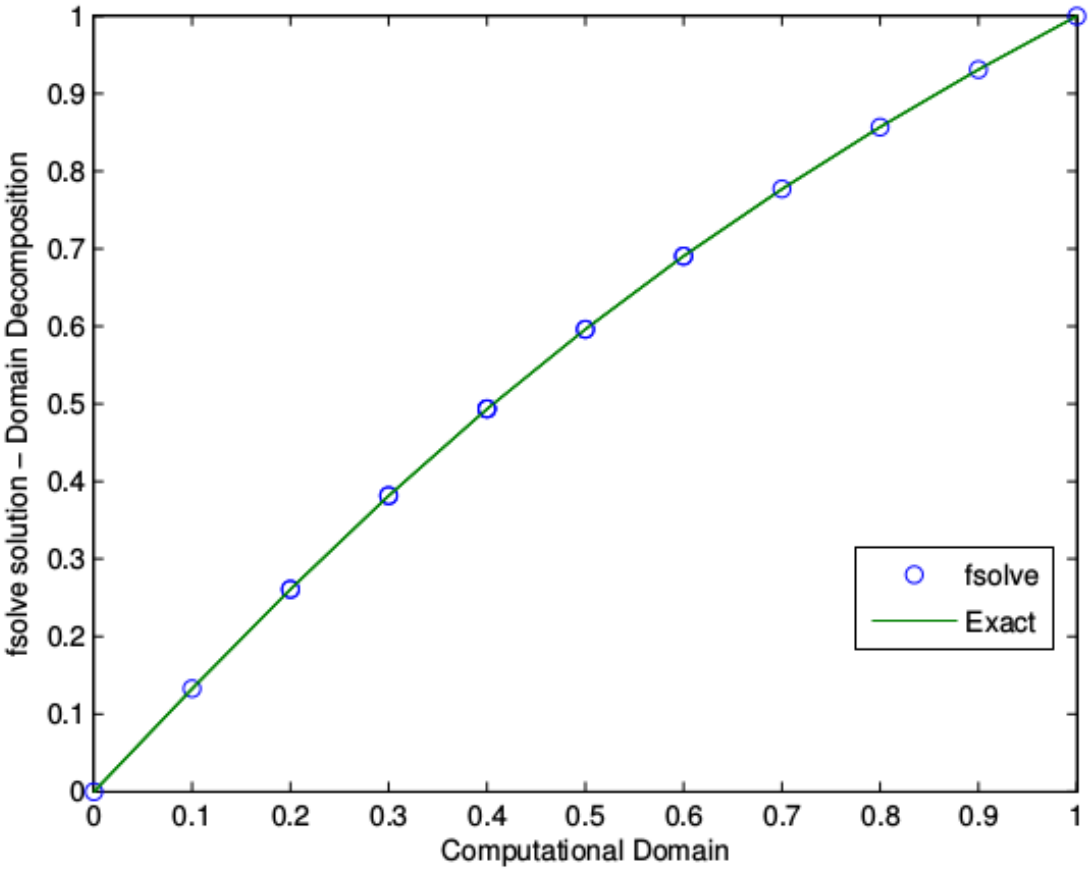


Figure 4.18: Overlay of the `fSolve` results on the known exact solution.

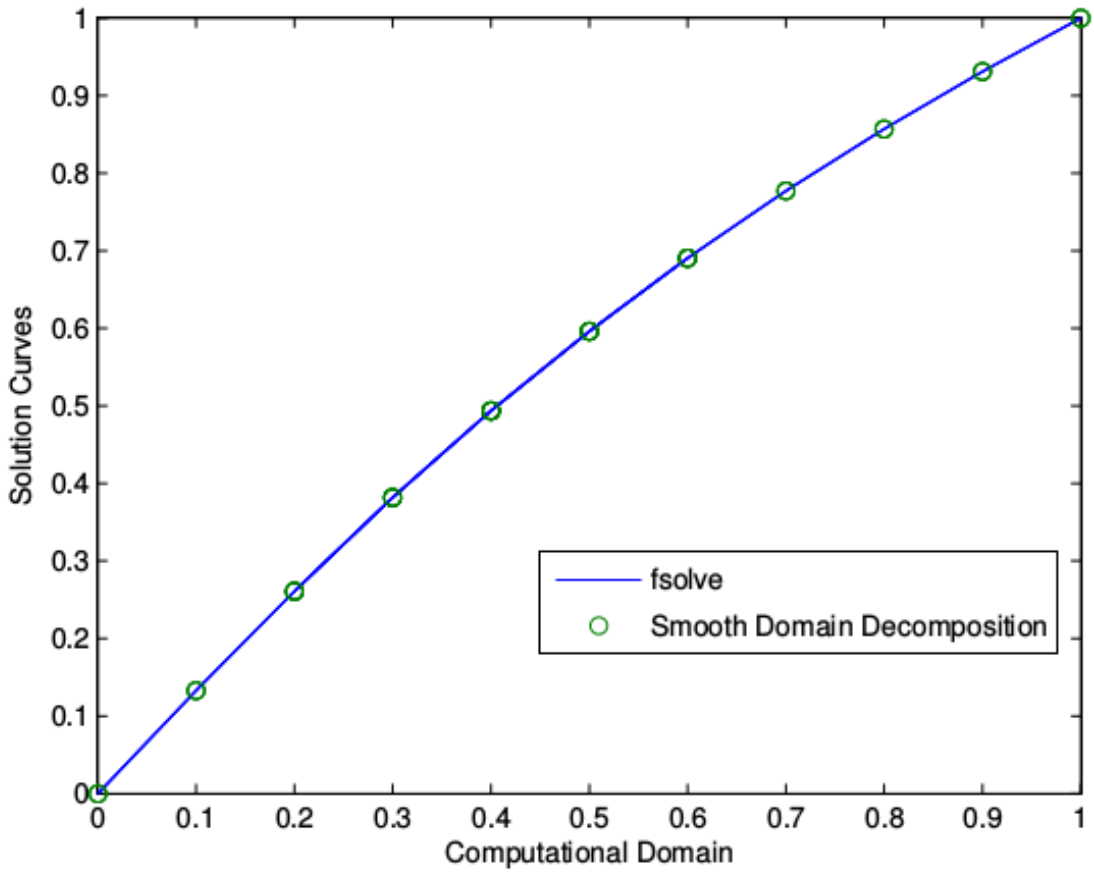


Figure 4.19: Overlay of the `fSolve` results on the Smoothed numerical solution.

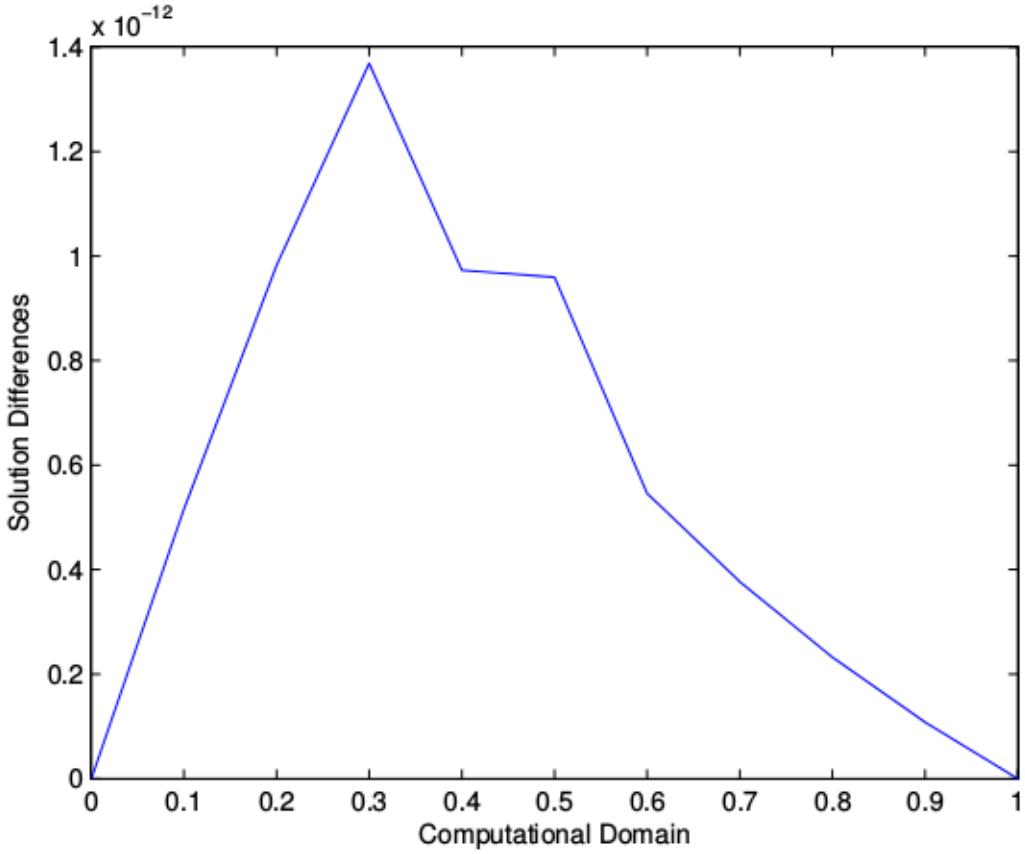


Figure 4.20: Overlay of the `fSolve` results on the known exact solution.

4.3.3 Comparison with More Difficult Monitor Function

We wish to consider a more difficult function in order to prove that the solver and methodology is capable of handling it. We choose the new monitor function,

$$M(x) = 1 + 20(1 - (\tanh(20(x - 0.25)))^2) + 30(1 - (\tanh(30(x - 0.5)))^2) + 10(1 - (\tanh(10(x - 0.75)))^2) \quad (4.1)$$

as an example of a more complicated function. This example is given in [9] as a similarly defined complex problem.

We use MATLAB's own built in solver, `fSolve`, once again and obtain the following results with a discretization of $\delta\xi = 0.025$.

When comparing Figures 4.22 and 4.23 we see that there is a much more regular distribution of mesh points now. This provides a solution whose discretization is more inline with a standard uniform mesh while at the same time providing a higher density near the points of interest.

In Figures 4.24 and 4.25 we once again see the benefit of smoothing. In 4.24 there are gaps near the points of difficulty, such as near $\xi = 0.35$. While the smoothed solver obtains a more consistent result and captures points in this region. This provides a

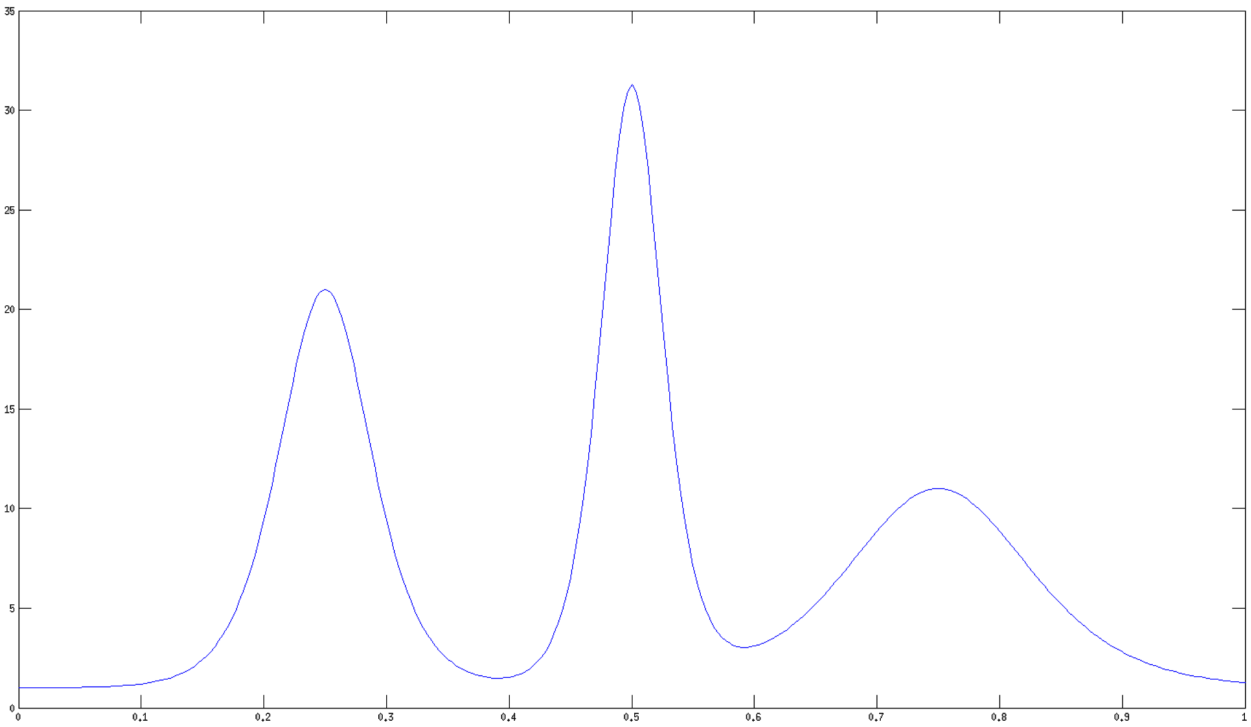


Figure 4.21: Monitor function being considered.

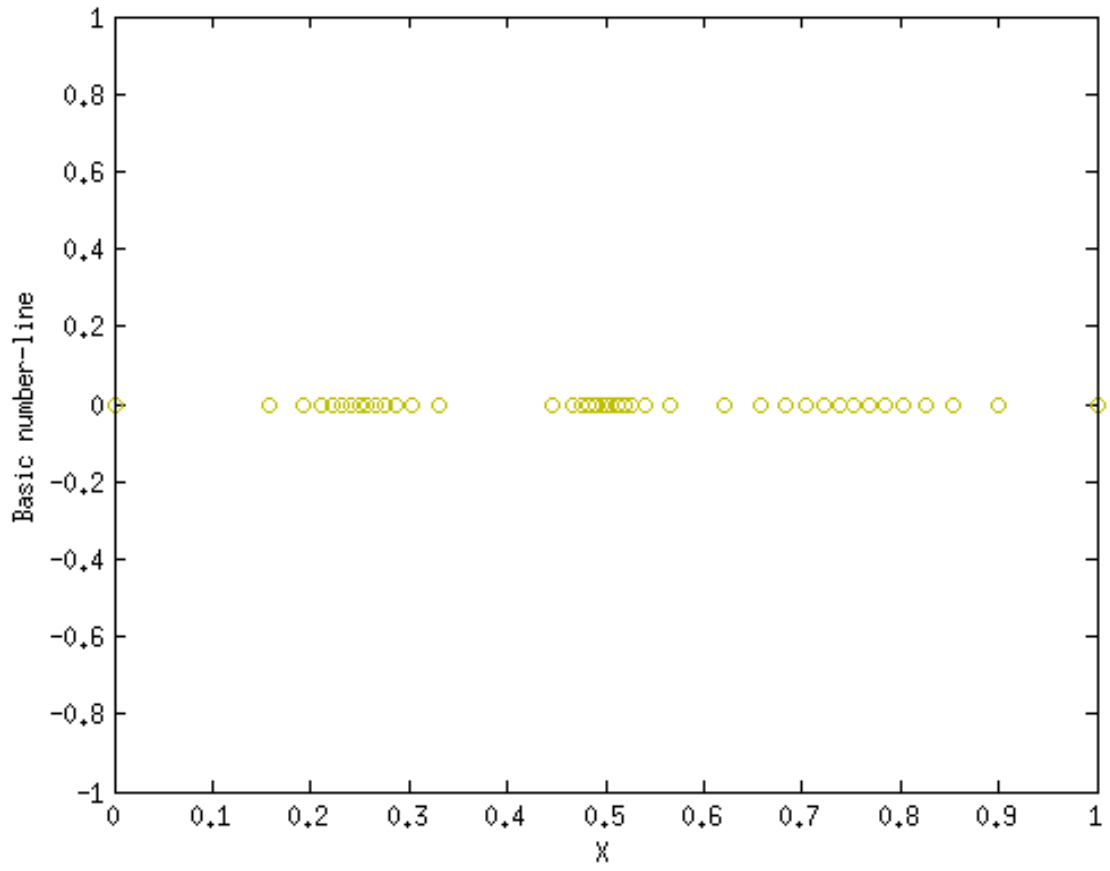


Figure 4.22: Density plot of the unsmoothed `fSolve` solution on the number line.

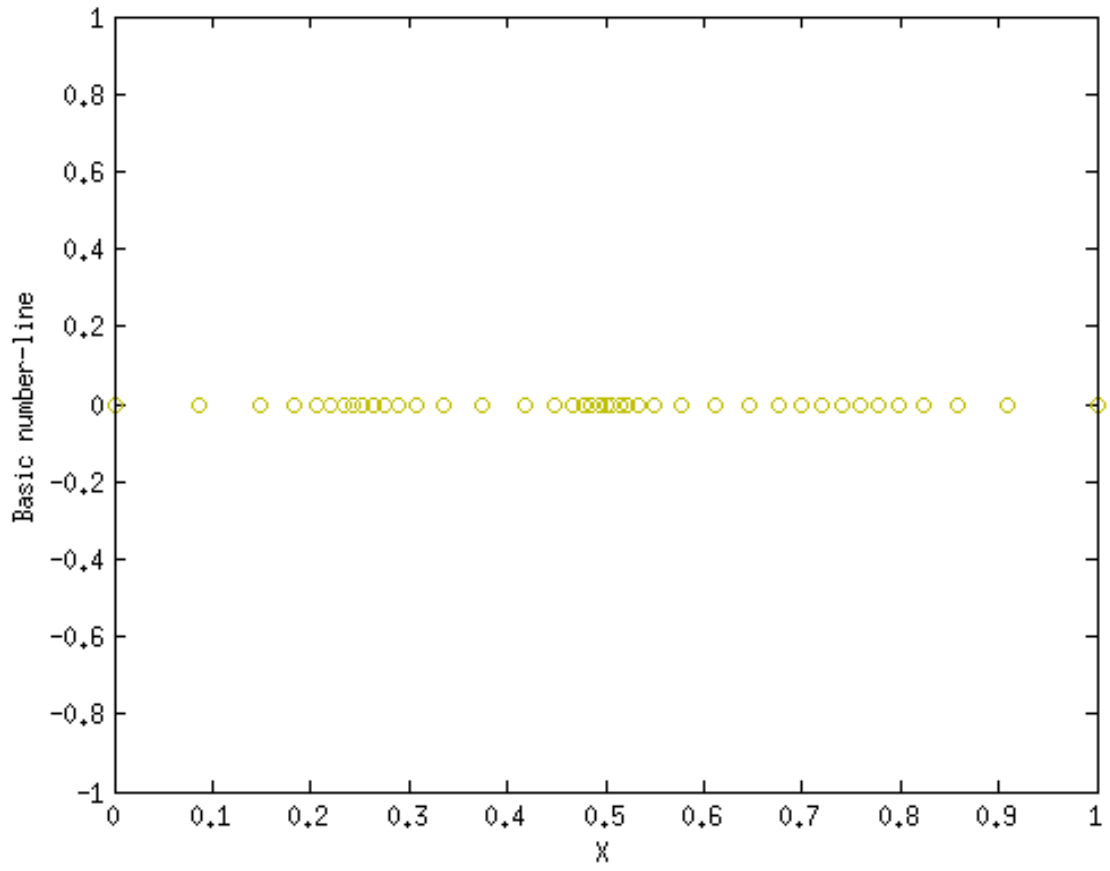


Figure 4.23: Density plot of the smoothed `fSolve` solution on the number line.

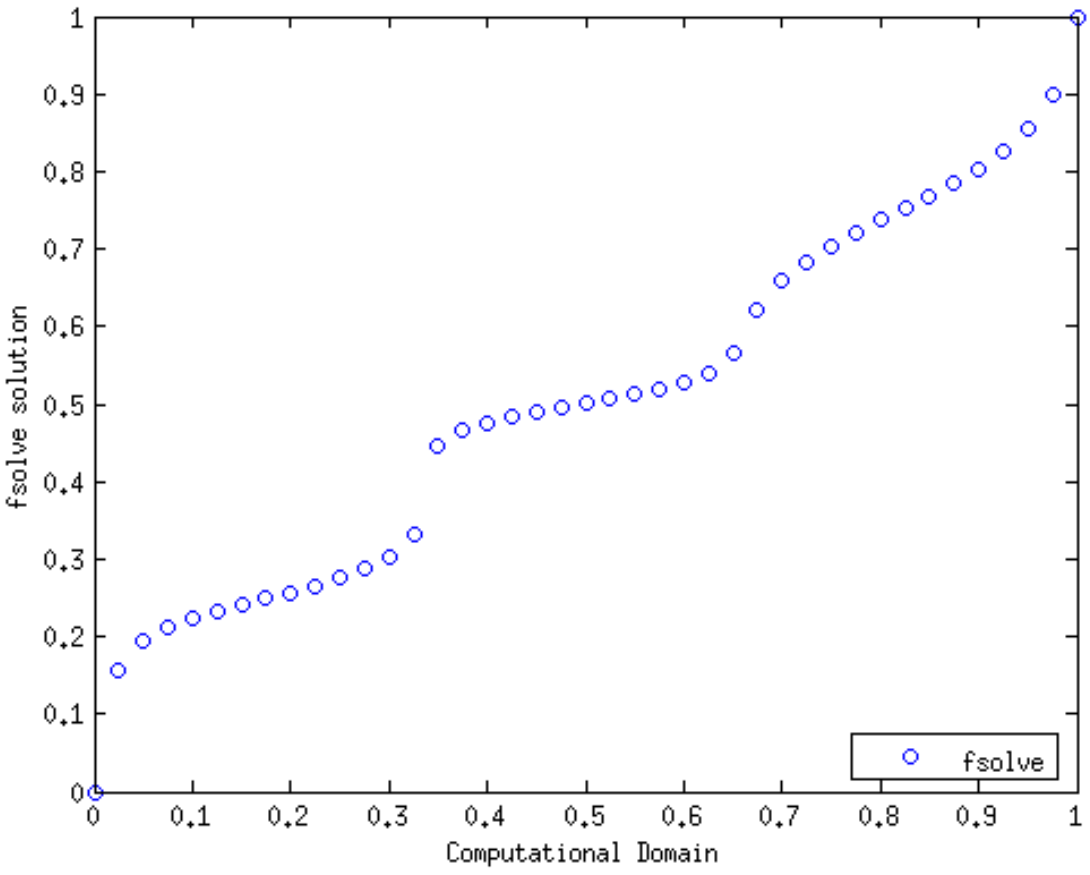


Figure 4.24: Plot of the unsmoothed `fSolve` solution.

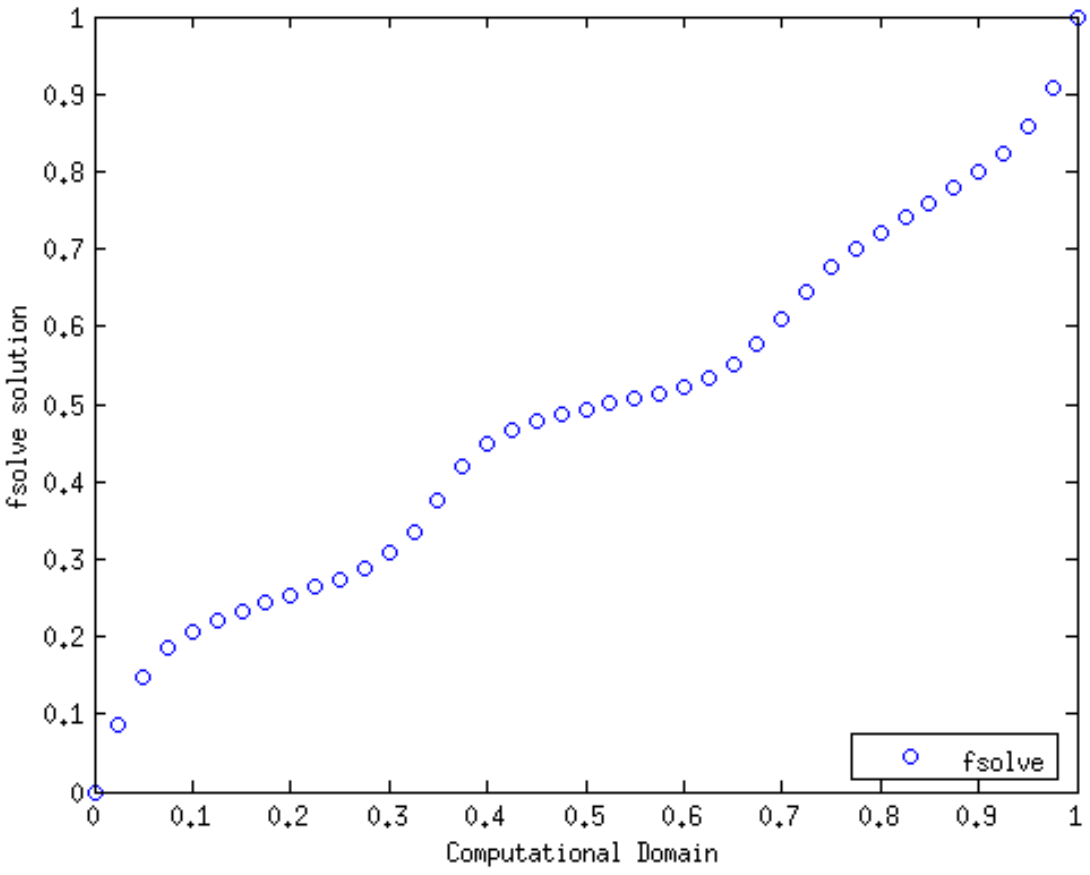


Figure 4.25: Plot of the smoothed `fSolve` solution.

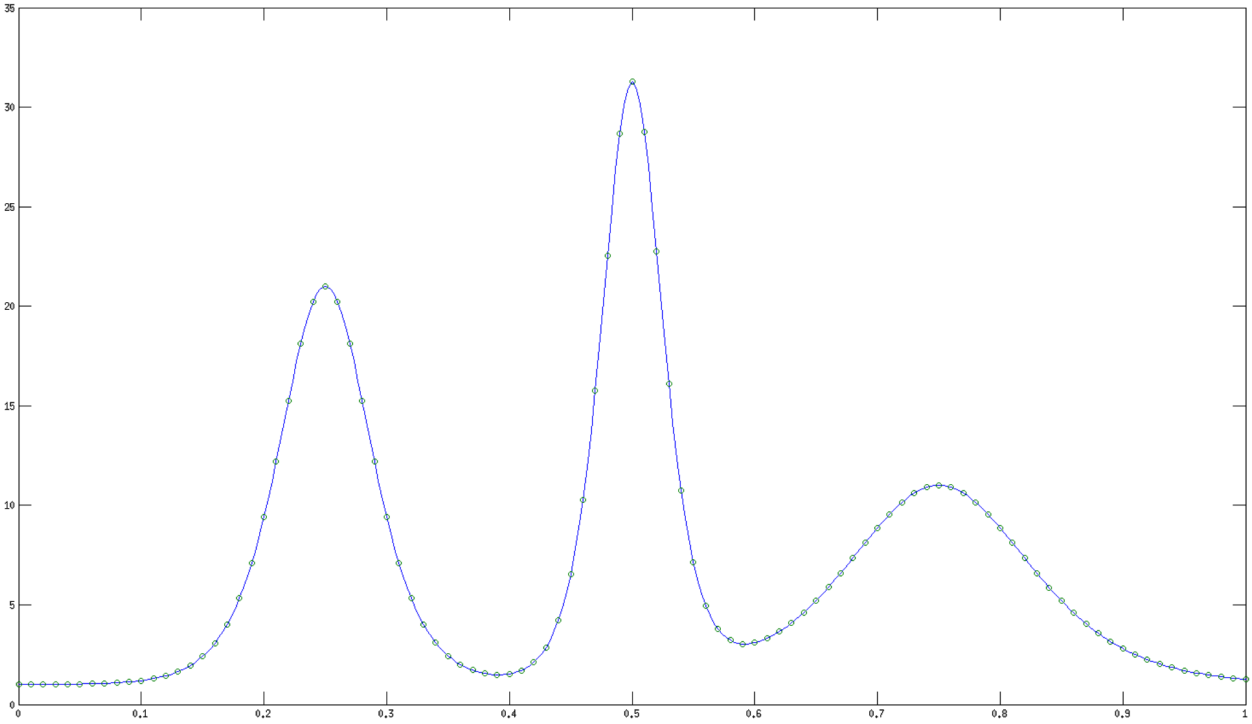


Figure 4.26: M(x) evaluated on a uniform set of nodes.

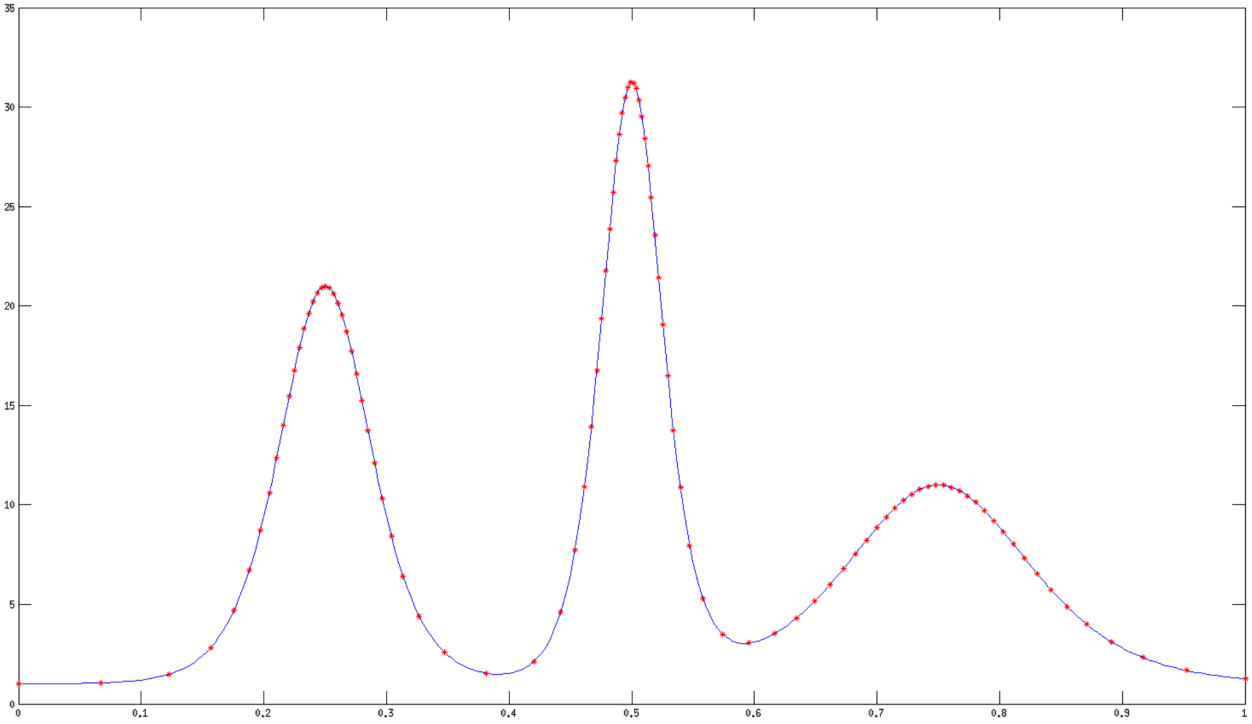


Figure 4.27: Smoothed adaptive mesh applied to monitor function for smoothing parameter $\beta = 250$.

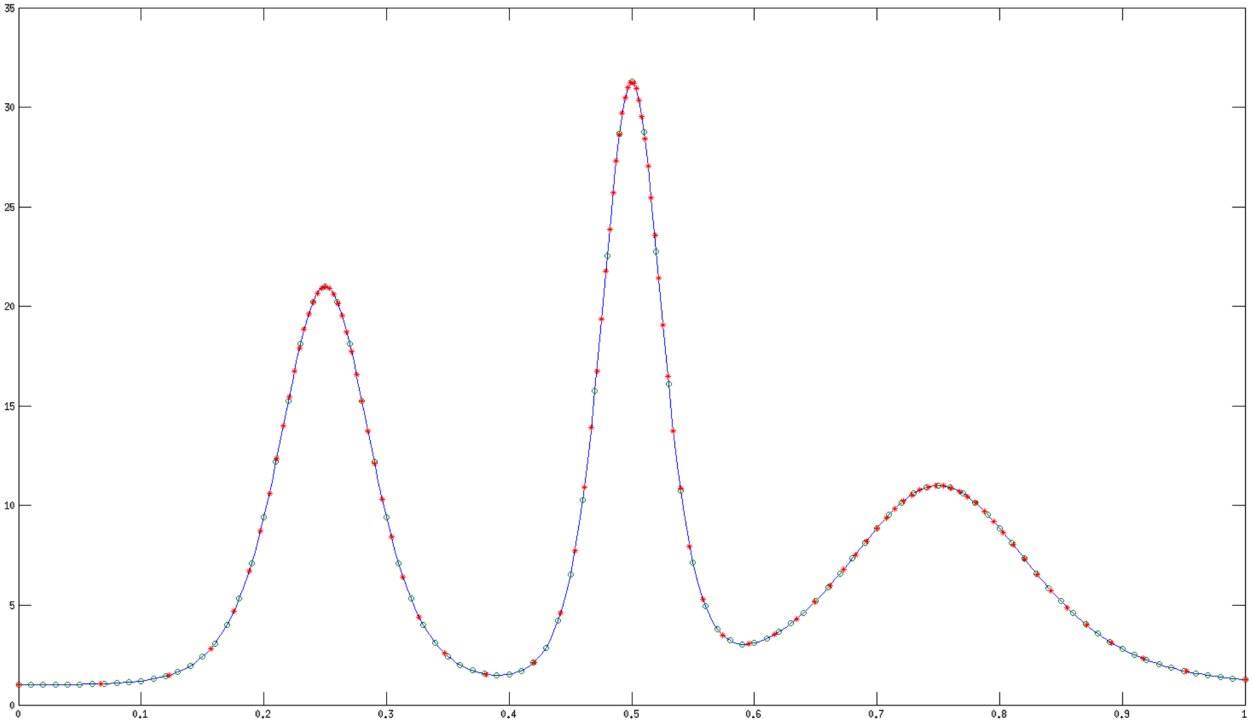


Figure 4.28: Overlay of the uniform grid and adaptive mesh on the monitor function.

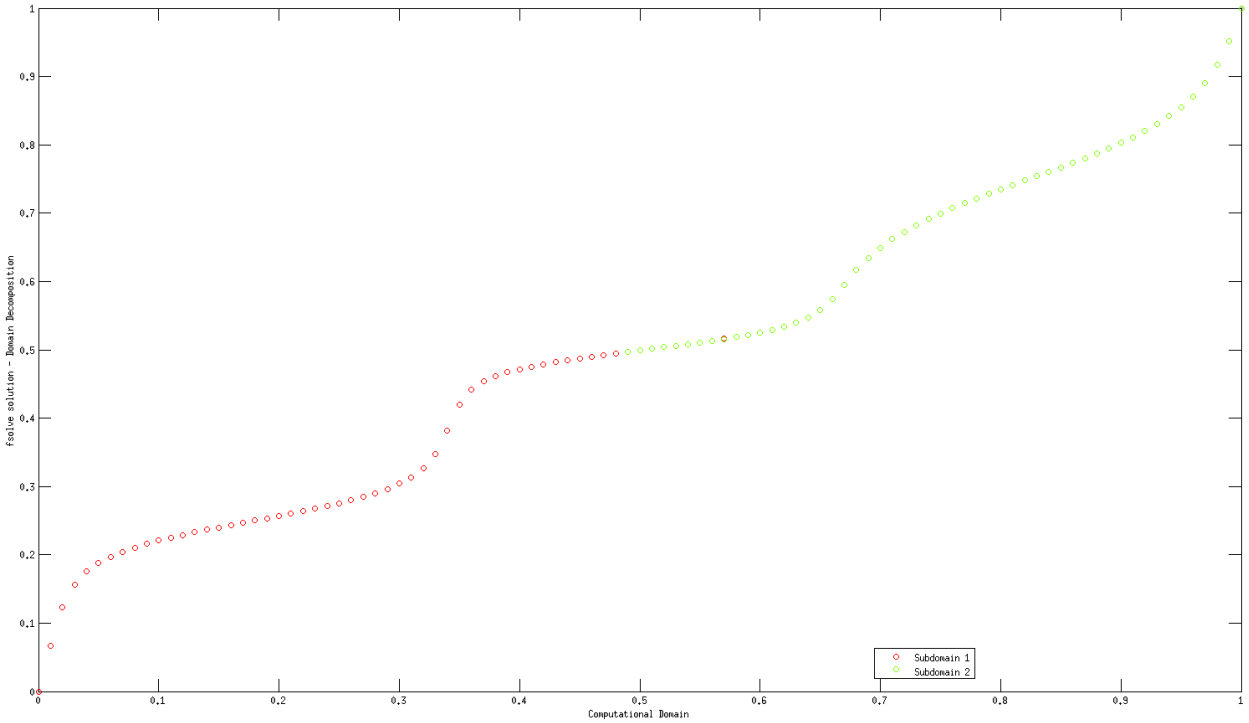


Figure 4.29: Plot of the smoothed `fSolve` solution on two subdomains for $\beta = 250$.

solution which adds points where they are needed most, at points of inflection.

Thanks to 4.21,4.26,4.27, and 4.28 we can see the effect of mesh adaptivity applied to our more complicated function. With reference to [11], we expect these meshes to provide better numerical results when used. The dense mesh points located around the inflection points, coupled with the sparsely populated near-linear points demonstrate the effective use of mesh clustering.

Finally, 4.29 is an excellent demonstration of the modified smoothed monitor function's ability to apply domain decomposition for extremely complicated functions.

The results of our standard methodology, coupled with the results of the smoothed mesh method suggest that this is an excellent candidate for regular use in numerical mesh generation. Thanks to Pryce [11], we know that smoother functions will generate superior meshes in terms of the truncation error estimates. Thus, through the use of the above technique we have generated a mesh with highly desirable properties. We also see that the dependence of β will effect the rate of convergence independent of the other aspects of the system.

In the following chapter we will outline some suggestions for possible future work to be performed in terms of mesh smoothing. These are known techniques which are excellent candidates for this testing as they are well understood and well behaved methods.

Chapter 5

Conclusions and Future Work

In Chapter 5, we outline the results and conclusions which can be obtained from both the analysis of aforementioned methodologies and from numerical results obtained through direct implementation. As well, we briefly describe the possible future work which may be performed utilizing these methods. It is known that each of these methods provide good applications of domain decomposition and are thus good candidates for studying mesh smoothing.

5.1 Conclusions

In these experiments we have been able to demonstrate that a domain decomposition method can be applied to a differential equation in an efficient and effective manner. This produces high quality results which may be used for domain discretization in more complex parent differential equations.

The organization and configuration of the domains plays a large role in the converge of the system with regard to its speed. Few subdomains, which are highly overlapping, provide the best solutions. This tendency suggests that the best solution is found via a single domain evaluated directly. However, discretization of the domain allows for parallelization in code implementation which may be worth the decrease in computational time if employed properly.

As well, thanks to the work of Pryce [11], we know that "smoother" meshes provide a better way to numerical study differential equations. With our studies in this work, we know that "smooth" meshes can be efficiently and accurately created. These can be constructed using domain decomposition techniques as well and experience the same benefits and difficulties as the standard mesh generation technique.

5.2 Future Work

5.2.1 Linearized Schwarz Method

When we consider the use of the solver, it is computationally exhausting to calculate the Jacobian matrix and proceed through a full Newton iteration for each step. Therefore, we can use a modified version of the Schwarz iteration which will "linearize" our system. First, let us remind ourselves of the discretization of the discrete form of the differential equation,

$$M\left(\frac{x_{i+1}^k + x_i^k}{2}\right)\frac{x_{i+1}^k - x_i^k}{\Delta\xi} - M\left(\frac{x_i^k + x_{i-1}^k}{2}\right)\frac{x_i^k - x_{i-1}^k}{\Delta\xi} = 0.$$

Next, we approximate the mesh generating function $M^k = M\left(\frac{x_{i+1}^k + x_i^k}{2}\right)$ by evaluating it at the the previous Newton iteration. This is expressed as

$$M\left(\frac{x_{i+1}^{k-1} + x_i^{k-1}}{2}\right)\frac{x_{i+1}^k - x_i^k}{\Delta\xi} - M\left(\frac{x_i^{k-1} + x_{i-1}^{k-1}}{2}\right)\frac{x_i^k - x_{i-1}^k}{\Delta\xi} = 0.$$

This new form is clearly a system of simple linear solves as the M function is now a constant value coefficient[D[D[D[D[i at the k^{th} iteration. This process is known as a Linearized Schwarz [6] method. The clear benefit of this process is that it reduces the complex non-linear system of equations into a straightforward linear solve which

can be solved much easier. The downside to this process is that the mesh function M is somewhat inaccurately evaluated. By considering it at the previous iteration we may have "outdated" information used. This is particularly relevant in the early stages where the solver approximations x^k and x^{k+1} can differ greatly.

Thus, we have traded numerical accuracy in our function evaluation for a simplified system solve which can be evaluated easily. Thus for a good initial guess, x^0, this linearized version can be a very effective method.

The "smoothed" mesh generating function possesses non-linear terms in the standard differential form. This suggests the fact that a linearized version would be non-trivial to evaluate. Specifically, it seems to suggest that solving for the linear $y_{j+\frac{1}{2}}$ and then solving for the x values from here. This would required two different solver stages and makes it a prime candidate for smoothing if the x solve stage is not too slow to offset the benefit of numerical simplicity.

5.2.2 Optimized Schwarz Method

In the Optimized Schwarz Method [4] we consider an alteration to the transmission conditions being used. Under the standard second order boundary value problem, Dirichlet transmission conditions are used. This is to say that we match function values at the subdomain boundary overlap point. Explicitly, this is written for the

two subdomain case as,

$$v^k(\beta) = w^{k-1}(\beta)$$

$$w^k(\alpha) = v^{k-1}(\alpha)$$

where v is the solution on the first subdomain, and w is the solution on the second.

Optimized Schwarz technique uses Robin transmission conditions which are a mix of Dirichlet and Neumann (derivative matching) conditions. In general, for the two subdomain case we can express this as

$$av^k(\beta) + \frac{v^k(\beta)}{\partial\xi} = aw^{k-1}(\beta) - \frac{w^{k-1}(\beta)}{\partial\xi}$$
$$aw^k(\alpha) + \frac{w^k(\alpha)}{\partial\xi} = av^{k-1}(\alpha) - \frac{v^{k-1}(\alpha)}{\partial\xi}$$

for some constant a to be determined by the properties of the equation.

Since the "Smoothed" mesh function suggests that our governing equation is 4^{th}-order equation then this suggests that this equation needs additional transmission conditions above the standard Dirichlet terms. If we impose both Dirichlet and Robin conditions then we simply define Dirichlet and Neumann conditions implicitly. Moving forward with future studies of Optimized Schwarz techniques on the "Smoothed" mesh function requires constructing modified transmission conditions which can incorporate the higher order derivative terms.

5.2.3 Smoothed de Boor's Algorithm

As was previously discussed, the de Boor's algorithm is the original motivation from the equidistribution principle and the adaptive mesh that it generates. This leads us to ask the question, can the de Boor algorithm be "smoothed" in a manner similar to the way we smoothed the mesh generating function for the boundary value problem?

Let us restate the de Boor algorithm:

$$x_j^h = x_{k-1}^{h-1} + \frac{2(\frac{j-1}{N-1}P(1) - P(x_{k-1}^{h-1})}{\rho(x_{k-1}^{h-1}) + \rho(x_k^{h-1})}.$$

where,

$$P(x_j^o) = \sum_{i=1}^{j}(x_i^o - x_{i-1}^o)\frac{\rho(x_i^o) - \rho(x_{i-1}^o)}{2}$$

for a given mesh density function $\rho(x)$.

Since this algorithm is implicitly dependant on only the mesh function, $\rho(x)$, then this suggests we may be able to do the same "smoothing" operation to this algorithm as before. Again, this process is an excellent candidate for "smoothing" the mesh function.

Appendix A

Primary Code

```
function dd(h,method)

format long

time_start = tic;



%%%%%%%%%%%%%% Description %%%%%%%%%%%%%%%%%

% This code will construct the adaptive

% mesh we desire. It makes use of domain

% decomposition techniques and optimized

% methods for solving linear systems.

%%%%%%%%%%%%%% Description %%%%%%%%%%%%%%%%%
```

```
[alpha,beta,A,B,tol,len,k,m,it,maxit,siz,siz2,x,store,e,space,avg,errsav
    ,no_sub,e_dom] = params_multi(h);    % Predeclare the variables for
    the domain and the boundaries

sizsave=siz;
[sol] = exact(e,siz);                    % Construct the exact solution
    over the domain

for iter = 1:no_sub                      % Need to create the vectors for
     each of the subdomains

    dom{iter} = x(alpha(iter):beta(iter));

end



while (m>tol)&&(k<maxit)                 % This is the while loop which
    we will be iterating over with a maximum number of iterations and a
    tolerance condition to be met
```

```
for iter = 1:no_sub



    x = dom{iter};


    siz = length(x);


    M = zeros(1,siz);                          % Preallocating the Newton
        step matrix for storage space


    if(method == 0)                            % This is where we construct
         the Jacobian and G terms to use for the Newton Steps
         [G,J] = jacobian_trap(x,siz);         % Construct the Solver
            Matrix G and the Jacobian J at the same time for simplicity
            via a Trapazoidal Method
         errsav = strcat(errsav,'_trap');
    elseif (method==1)
         [G,J] = jacobian_mid(x,siz);              % Construct the Solver
            Matrix G and the Jacobian J at the same time for simplicity
            via a Midpoint Method
         errsav = strcat(errsav,'_mid');
```

```
else
    fprintf('Please chose a valid method for testing. Current
       methods are: \n 0 - Trapazoidal. \n 1 - Midpoint.\n');
    return;
end

[x,m] = solver2(x,G,J,siz);     % Solver Stage is called and replies
    with the needed tolerance conditions

dom{iter} = x;                          % Place the new calculated
   solution into the appropriate subdomain cell location


if (iter==1)                            % This stage calculates the
   size of the overlap in order to determine the location of the
   domain node points
    olap = 1+(beta(iter) - alpha(iter+1));          % Overlap
       between First and Second subdomains
elseif (iter == no_sub)
    olap = 1+(beta(iter-1) - alpha(iter));          % Overlap
       between the Last and Second-last subdomains
else
```

```
    olap1 = 1+(beta(iter-1) - alpha(iter));          % Overlap
       between the current subdomain and the previous subdomain
    olap2 = 1+(beta(iter) - alpha(iter+1));          % Overlap
       between the current subdomain and the next subdomain
end




if (iter == 1)                              % This stage will be used to
    replace the nodes with specific interior points of the adjacent
    subdomains
    dom{2}(1) = x(length(dom{1})+1-olap);            % Second
       subdomain
elseif(iter == no_sub)
    dom{no_sub-1}(length(dom{no_sub-1})) = x(olap);      % Second-
       last subdomain
else
    dom{iter-1}(length(dom{iter-1})) = x(olap1);         % Previous
       subdomain
    dom{iter+1}(1) = x(length(dom{iter})+1-olap2);
       % Next subdomain
end
```

```
m_sub(iter) = m;


end         % End of for loop for subdomain calls
m = max(m_sub);
k = k+1;
kcount(k) = k;


[e_dom] = error_domain(e_dom,dom,sol,alpha,beta,k);


        xdomsmooth = dom{1};


        for w = 2:no_sub
            for s = alpha(w):beta(w-1)
                xdomsmooth(s) = (xdomsmooth(s)+dom{w}(s+1-alpha(w)))/2.0
                    ;
            end
            xdomsmooth(beta(w-1):beta(w)) = dom{w}((beta(w-1)+1-alpha(w)
                ):length(dom{w}));
        end
```

```
            xdomsmooth(beta(no_sub-1):sizsave) = dom{no_sub}((beta(
                no_sub-1)+1-alpha(no_sub)):length(dom{no_sub}));



xdom{k}  = xdomsmooth;



end          % End of while loop for stopping criteria


if(k>maxit)||(m>tol)
    [dm] = broken(tol,h,m); % Kills the process if maximum number of
       iterations is exceeded
    return;
else
    [m,k,num_it,dm] = ender_dom(alpha,beta,m,k,tol,h,e,dom,sol,no_sub);
       % Prints the plots 1 & 2 if it converged in time
    %[errsav] = saver(x,h,dm,errsav); % Saves solution to external file
end
x = xdom{length(xdom)};


subdomain_err(e_dom,x,xdom);
```

```
testone(time_start,h);


end              % End of program
```

Appendix B

Smoothed Code

```
function multidomain(delex,method,number)

%%%%% Description %%%%%

% This code will solve the boundary

% value problem for the modified

% smooth BVP. It makes use of

% optimization techniques for solving

% linear systems. It is also

% designed to utilize domain

% decomposition techniques.

%%%%% Description %%%%%
```

```
hold off
format long
time_start = tic;
global bet h siz no_sub;
h = delex;
[alpha,beta,A,B,tol,len,k,m,it,maxit,siz,siz2,x,store,e,space,avg,errsav
   ,no_sub,olap,e_dom,bet_array] = params_multi_smooth(h,number);  %
   Predeclare the variables for the domain and the boundaries

[sol] = exact_smooth(e,siz);                    % Construct the exact
   solution over the domain
sizsave=siz;
for iter = 1:no_sub                      % Need to create the vectors for
    each of the subdomains

    dom{iter} = x(alpha(iter):beta(iter));

end

for b =1:length(bet_array)
```

```
bet = bet_array(b);

if (no_sub == 1)                    % This section is optimized for
   single domain solves

    fprintf('This is a single domain system.\nThis will be solved using
       a single domain solver.\nThis is the entire 0 to 1 domain.\n')

    M = zeros(1,siz);

    [sol] = exact_smooth(e,siz);
    strk = 'k';
fprintf('    %4s    |   Stopping Condition  |    Beta    \n',strk)
fprintf('============================================\n')
    while (m>tol)&&(k<maxit) % This is the while loop which we will be
       iterating over with a maximum number of iterations and a
       tolerance condition to be met

    if(method == 0)
```

```
        fprintf('Currently, the Trapazoidal Method is not implimented.\
            nPlease choose Method = 1, the Midpoint Method.\nSorry for
            the inconvenience.\n');
        return;
        [J,G] = jac_smooth_trap(x,siz,bet,h);       % Construct the
            Solver Matrix G and the Jacobian J at the same time for
            simplicity via a Trapazoidal Method
        errsav = strcat(errsav,'_trap');
    elseif (method==1)
        %[J,G] = jac_smooth_mid(x,siz,bet,h);       % Construct the
            Solver Matrix G and the Jacobian J at the same time for
            simplicity via a Midpoint Method
        [J,G] = jac_smooth_mid(x,siz,bet,h);
        errsav = strcat(errsav,'_mid');
    else
        fprintf('Please chose a valid method for testing. Current
            methods are: \n 0 - Trapazoidal. \n 1 - Midpoint.\n');
        return;
    end
    [x,m,k,store] = solver_smooth(x,J,G,k,siz,M,store); % Solver Stage


    store(:,k) = x;
```

```
    fprintf('    %4.0f     |      %14e     | %6.0f\n',k,m,bet)

    end          % End of while loop for stopping criteria

    if(k>maxit)||(m>tol)
        [dm] = broken_smooth(tol,h,m);       % Kills the process if
           maximum number of iterations is exceeded
        [errsav] = saver_smooth(x,h,dm,errsav); % Saves solution to
           external file
        return;
    else
        [m,k,num_it,dm] = ender_smooth(m,k,tol,h,e,x,sol);  % Prints the
            plots 1 & 2 if it converged in time
        [errsav] = saver_smooth(x,h,dm,errsav); % Saves solution to
           external file
    end

    error_sec_smooth(x,store,num_it);

xsmooth = x;
```

```
else                                    % This section is optimized for
   multidomain solvers
[alpha,beta,A,B,tol,len,k,m,it,maxit,siz,siz2,x,store,e,space,avg,errsav
   ,no_sub,olap,e_dom,bet_array] = params_multi_smooth(h,number);
    fprintf('This is a multidomain system.\nThis will be sovled using a
       multidomain solver.\nThere are %d subdomains.\n',no_sub)
    fprintf('The subdomains are: \n')
    for iter = 1:no_sub                     % Need to create the vectors
        for each of the subdomains

        dom{iter} = x(alpha(iter):beta(iter));      % Creates the cells
           and arrays for each subdomain

        fprintf('%g to %g \n',h*(alpha(iter)-1),h*(beta(iter)-1))   %
           Prints a list of the subdomains to the user for reference (
           Converted to the domain [0,1])
    end
        strk = 'k';
    fprintf('    %4s    |   Stopping Condition  |    Beta    \n',strk)
    fprintf('=============================================\n')
```

```
while (m>tol)&&(k<maxit)                % This is the while loop
    which we will be iterating over with a maximum number of
    iterations and a tolerance condition to be met

     for iter = 1:no_sub



         x = dom{iter};


         siz = length(x);


         M = zeros(1,siz);                      % Preallocating the
             Newton step matrix for storage space


         if(method == 0)                        % This is where we
             construct the Jacobian and G terms to use for the Newton
              Steps
              [J,G] = jac_smooth_trap(x,siz,bet,h);       % Construct
                 the Solver Matrix G and the Jacobian J at the same
                 time for simplicity via a Trapazoidal Method
              errsav = strcat(errsav,'_trap');
         elseif (method==1)
```

```
        [J,G] = jac_smooth_mid(x,siz,bet,h);            %
            Construct the Solver Matrix G and the Jacobian J at
            the same time for simplicity via a Midpoint Method
        errsav = strcat(errsav,'_mid');
    else
        fprintf('Please chose a valid method for testing.
            Current methods are: \n 0 - Trapazoidal. \n 1 -
            Midpoint.\n');
        return;
    end

    [x,m,k,store] = solver_smooth_dom(x,J,G,k,siz,M,store);
        % Solver Stage is called and replies with the needed
        tolerance conditions

    dom{iter} = x;                              % Place the new
        calculated solution into the appropriate subdomain cell
        location


    if (iter==1)                                % This stage
        calculates the size of the overlap in order to determine
```

```
        the location of the domain node points
        olap = 1+(beta(iter) - alpha(iter+1));          %
            Overlap between First and Second subdomains
    elseif (iter == no_sub)
        olap = 1+(beta(iter-1) - alpha(iter));          %
            Overlap between the Last and Second-last subdomains
    else
        olap1 = 1+(beta(iter-1) - alpha(iter));         %
            Overlap between the current subdomain and the
            previous subdomain
        olap2 = 1+(beta(iter) - alpha(iter+1));         %
            Overlap between the current subdomain and the next
            subdomain
    end



    if (iter == 1)                          % This stage will be
        used to replace the nodes with specific interior points
        of the adjacent subdomains
        dom{2}(1) = x(length(dom{1})+1-olap);           % Second
            subdomain
```

```
        elseif(iter == no_sub)
            dom{no_sub-1}(length(dom{no_sub-1})) = x(olap);      %
               Second-last subdomain
        else
            dom{iter-1}(length(dom{iter-1})) = x(olap1);         %
               Previous subdomain
            dom{iter+1}(1) = x(length(dom{iter})+1-olap2);
                    % Next subdomain
        end




        m_sub(iter) = m;


    end          % End of for loop for subdomain calls
    m = max(m_sub);
    k = k+1;
    kcount(k) = k;
fprintf('    %4.0f     |      %14e     | %6.0f\n',k,m,bet)
    [e_dom] = error_domain_smooth(e_dom,dom,sol,alpha,beta,k);


end          % End of while loop for stopping criteria
```

```
    domsmooth = dom;
%%%%%%% This section might need to be moved to after the end term
    xsave(:,b) = x;

    if (no_sub==1)
        error_sec_smooth(x,store,num_it,space,avg)

        evh_smooth(errsav,method);


        testone_smooth(time_start,h);
    else
        if(k>maxit)||(m>tol)
            [dm] = broken_smooth(tol,h,m);      % Kills the process if
               maximum number of iterations is exceeded
            %[errsav] = saver(x,h,dm,errsav); % Saves solution to
               external file
            return;
        else
            [m,k,num_it,dm] = ender_dom_smooth(alpha,beta,m,k,tol,h,e,
               dom,sol,no_sub);  % Prints the plots 1 & 2 if it
```

```
                converged in time
            %[errsav] = saver(x,h,dm,errsav); % Saves solution to
                external file
        end

        testone_smooth(time_start,h);
    end



end % End of if for number of subdomains



end % End of various Smoothing Coeffecient sections



%%%%%%%%%%%%Solver for non-smoothed%%%%%%%%%%%%%%%

if (no_sub ==1) % Single domain solver for the non-smoothed system
```

```
[alpha,beta,A,B,tol,len,k,m,it,maxit,siz,siz2,x,store,e,space,avg,
   errsav,no_sub,olap,e_dom,bet_array] = params_multi_smooth(h,
   number);


for iter = 1:len


    [sol] = exact_smooth(e,siz);


    while (m>tol)&&(k<maxit) % This is the while loop which we will
       be iterating over with a maximum number of iterations and a
       tolerance condition to be met


        if(method == 0)
            [a,b,c,d] = jacobian_trap_thomas(x,siz);        %
               Construct the Solver Matrix G and the Jacobian J at
               the same time for simplicity via a Trapazoidal
               Method
        elseif (method==1)
            [a,b,c,d] = jacobian_mid_thomas(x,siz);     % Construct
               the Solver Matrix G and the Jacobian J at the same
               time for simplicity via a Midpoint Method
        else
```

```
            fprintf('Please chose a valid method for testing.
                Current methods are: \n 0 - Trapazoidal. \n 1 -
                Midpoint.\n');
            return;
        end

        [x,m,k,store] = solver_thomas(x,a,b,c,d,k,siz,M,store); %
            Solver Stage

    end             % End of while loop for stopping criteria

    if(k>maxit)||(m>tol)
        [dm] = broken(tol,h,m);     % Kills the process if maximum
            number of iterations is exceeded
        return;
    else
        [m,k,num_it,dm] = ender(m,k,tol,h,e,x,sol); % Prints the
            plots 1 & 2 if it converged in time
    end

end
```

```
else % This is the section for the multidomain non-smoothed

    [alpha,beta,A,B,tol,len,k,m,it,maxit,siz,siz2,x,store,e,space,avg,
       errsav,no_sub,olap,e_dom,bet_array] = params_multi_smooth(h,
       number);
        for iter = 1:no_sub                        % Need to create the
           vectors for each of the subdomains
            dom{iter} = x(alpha(iter):beta(iter));
        end


    while (m>tol)&&(k<maxit)                  % This is the while loop
       which we will be iterating over with a maximum number of
       iterations and a tolerance condition to be met


        for iter = 1:no_sub


            x = dom{iter};


            siz = length(x);
```

```
M = zeros(1,siz);                          % Preallocating the
    Newton step matrix for storage space


if(method == 0)                            % This is where we
    construct the Jacobian and G terms to use for the Newton
     Steps
     [a,b,c,d] = jacobian_trap_thomas(x,siz);        %
        Construct the Solver Matrix G and the Jacobian J at
        the same time for simplicity via a Trapazoidal
        Method
     errsav = strcat(errsav,'_trap');
elseif (method==1)
     [a,b,c,d] = jacobian_mid_thomas(x,siz);         %
        Construct the Solver Matrix G and the Jacobian J at
        the same time for simplicity via a Midpoint Method
     errsav = strcat(errsav,'_mid');
else
     fprintf('Please chose a valid method for testing.
        Current methods are: \n 0 - Trapazoidal. \n 1 -
        Midpoint.\n');
     return;
```

```
        end

        [x,m,k,store] = solver_thomas_dom(x,a,b,c,d,k,siz,M,store);
               % Solver Stage is called and replies with the needed
            tolerance conditions

        dom{iter} = x;                          % Place the new
           calculated solution into the appropriate subdomain cell
           location



        if (iter==1)                            % This stage
           calculates the size of the overlap in order to determine
            the location of the domain node points
            olap = 1+(beta(iter) - alpha(iter+1));          %
               Overlap between First and Second subdomains
        elseif (iter == no_sub)
            olap = 1+(beta(iter-1) - alpha(iter));          %
               Overlap between the Last and Second-last subdomains
        else
            olap1 = 1+(beta(iter-1) - alpha(iter));         %
               Overlap between the current subdomain and the
```

```
            previous subdomain
        olap2 = 1+(beta(iter) - alpha(iter+1));          %
            Overlap between the current subdomain and the next
            subdomain
    end




    if (iter == 1)                              % This stage will be
        used to replace the nodes with specific interior points
        of the adjacent subdomains
        dom{2}(1) = x(length(dom{1})+1-olap);            % Second
            subdomain
    elseif(iter == no_sub)
        dom{no_sub-1}(length(dom{no_sub-1})) = x(olap);      %
            Second-last subdomain
    else
        dom{iter-1}(length(dom{iter-1})) = x(olap1);         %
            Previous subdomain
        dom{iter+1}(1) = x(length(dom{iter})+1-olap2);
                    % Next subdomain
    end
```

```
    m_sub(iter) = m;

    end         % End of for loop for subdomain calls

    m = max(m_sub);
    k = k+1;
    kcount(k) = k;


end


if(k>maxit)||(m>tol)
    [dm] = broken(tol,h,m);
    return;
else
    [m,k,num_it,dm] = ender_dom(alpha,beta,m,k,tol,h,e,dom,sol,
       no_sub);
end
```

```
end


%%%%%%%%%%%%%%%%%%%


    if (no_sub==1)

        %%%%%FSolve%%%%%

        [alpha,beta,A,B,tol,len,k,m,it,maxit,siz,siz2,x0,store,e,space,
           avg,errsav,no_sub,olap,e_dom,bet_array] =
           params_multi_smooth(h,number);


        options = optimoptions('fsolve','Display','off','TolFun',1e-12);
             % Turn off display

        global left right
```

```
left = x0(1);

right = x0(siz);









[xF,Fval,exitflag,Iter] = fsolve(@fsol,x0,options);









fsolnorm = norm(xF-sol,Inf);


dum_vec = xF-sol;

dum = 100000000000;

w=0;

    while (fsolnorm ~= dum)

        w = w+1;

        dum = abs(dum_vec(w));

    end

rel_err = fsolnorm/sol(w);


figure(11)
```

```
        plot(e,xF,'o',e,sol)
        title('The fsolve results')
        xlabel('Computational Domain')
        ylabel('fsolve solution')
        legend('fsolve','Exact','location','Best')
    %   text(0.3,0.2,{'This is an overlay of the single domain','fsolve
       results and the exact solution.'})


%%%%%

    else

        %%%%%FSolve - Domain Decomp%%%%%

        [alpha,beta,A,B,tol,len,k,m,it,maxit,siz,siz2,x0,store,e,space,
           avg,errsav,no_sub,olap,e_dom,bet_array] =
           params_multi_smooth(h,number);


        dom = domsmooth;
```

```
x_dum = x;
options = optimoptions('fsolve','Display','off','TolFun',1e-12);
     % Turn off display
tmp = 100;
tmpdiff = 100;


lefta(1) = x0(1);
righta(1) = x0(beta(1));
righta(no_sub) = x0(siz);
q=1;


while (tmpdiff > 10^(-18))
    for k = 1:no_sub
        iter = k;
        global left right
        left = lefta(iter);
        right = righta(iter);



        if (iter==1)                          % This stage
           calculates the size of the overlap in order to
           determine the location of the domain node points
```

```
        olap = 1+(beta(iter) - alpha(iter+1));      %
            Overlap between First and Second subdomains
    elseif (iter == no_sub)
        olap = 1+(beta(iter-1) - alpha(iter));      %
            Overlap between the Last and Second-last
            subdomains
    else
        olap1 = 1+(beta(iter-1) - alpha(iter));         %
            Overlap between the current subdomain and the
            previous subdomain
        olap2 = 1+(beta(iter) - alpha(iter+1));         %
            Overlap between the current subdomain and the
            next subdomain
    end


    if(tmp ==100)
        xdum = x0(alpha(k):beta(k));
    else


        xdum = x0(alpha(k):beta(k));
    end
```

```
global siz
siz = length(x0(alpha(k):beta(k)));
[xFdom{k},Fval,exitflag] = fsolve(@fsol,x0(alpha(k):beta
    (k)),options);
x = xFdom{iter};


if (iter == 1)                          % This stage will be
     used to replace the nodes with specific interior
    points of the adjacent subdomains
     lefta(2) = x(length(dom{1})+1-olap);             %
        Second subdomain
elseif(iter == no_sub)
     righta(no_sub-1) = x(olap);      % Second-last
        subdomain
else
     righta(iter-1) = x(olap1);       % Previous subdomain
     lefta(iter+1) = x(length(dom{iter})+1-olap2);    %
        Next subdomain
end


tmp1(k) = norm(xdum-xFdom{iter},Inf);
x0(alpha(k):beta(k)) = xFdom{iter};
```

```
            xFdom{iter};
        end
        tmp2 = max(tmp1);
        tmpdiff= tmp-tmp2;
        tmp = tmp2;
        tmparay(q)=tmp;
        q = q+1;
    end



    figure(16)
    hold off;
    loglog(linspace(1,q-1,q-1),tmparay)
    title('Maximum difference between consecutive solvers')
    xlabel('Iteration')
    ylabel('Difference')



    for k = 1:no_sub
        hold on;
        figure(11)
        plot(e(alpha(k):beta(k)),xFdom{k},'o',e,sol)
```

```
            title('The fsolve results over the Exact solution')
            xlabel('Computational Domain')
            ylabel('fsolve solution - Domain Decomposition')
            legend('fsolve','Exact','location','Best')

            hold off;
        end

        x = x_dum;
        %%%%%

    end

%%%%%%%%%%

siz = sizsave;



if (no_sub == 1)
    figure(13)
    plot(e,xF,e,xsmooth,'o')
    title('Overlay of fsolve and Smoothed Solution')
```

```
xlabel('Computational Domain')
ylabel('Solution Curves')
legend('fsolve','Smooth','location','Best')


figure(14)
plot(e,abs(xF-xsmooth))
title('Difference of fsolve and Smoothed Solution')
xlabel('Computational Domain')
ylabel('Solution Differences')



smoothnorm = norm(xF-xsmooth,Inf);
expon = floor(log10(smoothnorm));


fprintf('The norm of the difference between fsolve and the Smoothed
   Solver is %g\n',smoothnorm)
fprintf('The difference between solvers is O(10^%d)\n',expon)
if (expon<=-10)
    fprintf('This suggests agreement beteen solvers.\n')
else
    fprintf('This does not suggest agreement.\n')
end
```

```
else    % This is the section for figures containing the subdomain
   solutions

    for w = 1:no_sub
        hold on;
        figure(13)
        plot(e(alpha(w):beta(w)),xFdom{w},e(alpha(w):beta(w)),dom{w},'o'
            )
        title('Overlay of fsolve and Smoothed Solution')
        xlabel('Computational Domain')
        ylabel('Solution Curves')
        legend('fsolve','Smooth Domain Decomposition','location','Best')
        hold off;


        errfsol{w} = abs(xFdom{w}-dom{w});


        hold on;
        figure(14)
```

```
        plot(e(alpha(w):beta(w)),abs(xFdom{w}-dom{w}))
        title('Difference of fsolve and Smoothed Solution')
        xlabel('Computational Domain')
        ylabel('Solution Differences')
        hold off;
    end


    errarray = errfsol{1};


    for w = 2:no_sub
        for s = alpha(w):beta(w-1)
            errarray(s) = max(errarray(s),errfsol{w}(s+1-alpha(w)));
        end
        errarray(beta(w-1):beta(w)) = errfsol{w}((beta(w-1)+1-alpha(w)):
            length(xFdom{w}));
    end
        errarray(beta(no_sub-1):siz) = errfsol{no_sub}((beta(no_sub-1)
            +1-alpha(no_sub)):length(xFdom{no_sub}));


        hold on;
        figure(17)
        plot(e,errarray)
```

```
    title('Maximum difference of fsolve and Smoothed Solution')
    xlabel('Computational Domain')
    ylabel('Solution Differences')
    hold off;

    xFdom{w};
    dom{w};
    smoothnor(w) = norm(xFdom{w}-dom{w},Inf);



smoothnorm = norm(smoothnor,Inf);
expon = floor(log10(smoothnorm));

fprintf('The norm of the difference between fsolve and the Smoothed
   Solver is %g\n',smoothnorm)
fprintf('The difference between solvers is O(10^%d)\n',expon)
if (expon<=-10)
    fprintf('This suggests agreement beteen solvers.\n')
else
    fprintf('This does not suggest agreement.\n')
end
```

```
end
hold off;


    if(no_sub==1)
        Solution2 = geval(xF);
        Solution2 = transpose(Solution2);
        normsol2 = norm(Solution2,Inf);
        fprintf('The infinity norm of the G vector evaluated at the
           smoothed fsolve solution is %g\n',normsol2)
        transpose(xsmooth);
        Solution = geval(xsmooth);
        Solution = transpose(Solution);
        normsol = norm(Solution,Inf);
        fprintf('The infinity norm of the G vector evaluated at the
           smoothed Newton solution is %g\n',normsol)
    else

        %%%%%%%This section is for solutions to be averaged at the
           interior points%%%%%%%
        xfdomsmooth = dom{1};
```

```
for w = 2:no_sub
    for s = alpha(w):beta(w-1)
        xfdomsmooth(s) = (xfdomsmooth(s)+xFdom{w}(s+1-alpha(w)))
            /2.0;
    end
    xfdomsmooth(beta(w-1):beta(w)) = xFdom{w}((beta(w-1)+1-alpha
        (w)):length(xFdom{w}));
end
    xfdomsmooth(beta(no_sub-1):sizsave) = xFdom{no_sub}((beta(
        no_sub-1)+1-alpha(no_sub)):length(xFdom{no_sub}));



Solution2 = geval(xfdomsmooth);
Solution2 = transpose(Solution2);
normsol2 = norm(Solution2,Inf);
fprintf('The infinity norm of the G vector evaluated at the
    average smoothed fsolve solution is %g\n',normsol2)



xdomsmooth = dom{1};


for w = 2:no_sub
```

```
        for s = alpha(w):beta(w-1)
            xdomsmooth(s) = (xdomsmooth(s)+dom{w}(s+1-alpha(w)))/2.0
                ;
        end
        xdomsmooth(beta(w-1):beta(w)) = dom{w}((beta(w-1)+1-alpha(w)
            ):length(xFdom{w}));
    end
        xdomsmooth(beta(no_sub-1):sizsave) = dom{no_sub}((beta(
            no_sub-1)+1-alpha(no_sub)):length(xFdom{no_sub}));

    transpose(xdomsmooth);
    Solution = geval(xdomsmooth);
    Solution = transpose(Solution);
    normsol = norm(Solution,Inf);
    fprintf('The infinity norm of the G vector evaluated at the
        average smoothed Newton solution is %g\n',normsol)
    %%%%%



    %%%%%%%This section is for solutions to be exactly evaulated and
        the maximum value taken%%%%%%%
```

```
for w = 1:no_sub
    del2 = sort(abs(geval(xFdom{w})));
    if(w==1||w==no_sub)
        Gsol2(w) = abs(del2(length(xFdom{w})-1));
    else
        Gsol2(w) = abs(del2(length(xFdom{w})-2));
    end
end

normsol2 = norm(Gsol2,Inf);
fprintf('The infinity norm of the G vector evaluated at the
   unique smoothed fsolve solution is %g\n',normsol2)

for w = 1:no_sub
    del = sort(abs(geval(domsmooth{w})));
    if(w==1||w==no_sub)
        Gsol(w) = abs(del(length(domsmooth{w})-1));
    else
        Gsol(w) = abs(del(length(domsmooth{w})-2));
    end
end
```

```
        normsol = norm(Gsol,Inf);
        fprintf('The infinity norm of the G vector evaluated at the
           unique smoothed Newton solution is %g\n',normsol)
        %%%%%









    end



%%%%%

    if (length(bet_array) == 1)
        return;
    else
        for r = 1:length(bet_array)
            betanorm(r) = norm(transpose(x)-xsave(:,r),Inf);
        end
```

```
            figure(9)
            semilogx(bet_array,betanorm)
            title('Plot of norms of difference between non-smoothed
                solution and solutions for various smoothing
                coeffecients, Beta');
            xlabel('Computational Domain');
            ylabel('Solution');

    end

hold off

end             % End of program
```

Appendix C

Smoothed System and Jacobian Constructor

```
function [J_dum,G_dum] = jac_smooth_mid(x,siz,beta,h)


for j = 2:siz-1


    if (j == 2)      % This is the left side boundary section

        G(j) = pinv((x(j)+x(j+1))/2)*(h/(x(j+1)-x(j)) - (1/(beta*beta*h*

           h))*(h/(x(j+2)-x(j+1)) - 2*h/(x(j+1)-x(j)) + h/(x(j)-x(j-1))

           ))-pinv((x(j)+x(j-1))/2)*(h/(x(j)-x(j-1)) - (1/(beta*beta*h*
```

```
    h))*(h/(x(j+1)-x(j)) - h/(x(j)-x(j-1))));                % Left boundary of solver G

J(j,j) = 0.5*pinvprime((x(j)+x(j+1))/2)*(h/(x(j+1)-x(j)) - (1/(beta*beta*h*h))*(h/(x(j+2)-x(j+1)) - 2*h/(x(j+1)-x(j)) + h/(x(j)-x(j-1))))-0.5*pinvprime((x(j)+x(j-1))/2)*(h/(x(j)-x(j-1)) - (1/(beta*beta*h*h))*(h/(x(j+1)-x(j)) - h/(x(j)-x(j-1))))+pinv((x(j)+x(j+1))/2)*(h/((x(j+1)-x(j))^2) - (1/(beta*beta*h*h))*((-2)*h/((x(j+1)-x(j))^2) - h/((x(j)-x(j-1))^2)))-pinv((x(j)+x(j-1))/2)*((-1)*h/((x(j)-x(j-1))^2) - (1/(beta*beta*h*h))*(h/((x(j+1)-x(j))^2) + h/((x(j)-x(j-1))^2)));
                % Centre Element of Jacobian

J(j,j+1) = 0.5*pinvprime((x(j)+x(j+1))/2)*(h/(x(j+1)-x(j)) - (1/(beta*beta*h*h))*(h/(x(j+2)-x(j+1)) - 2*h/(x(j+1)-x(j)) + h/(x(j)-x(j-1))))+pinv((x(j)+x(j+1))/2)*((-1*h)/((x(j+1)-x(j))^2) - (1/(beta*beta*h*h))*(h/((x(j+2)-x(j+1))^2) + 2*h/((x(j+1)-x(j))^2)))-pinv((x(j)+x(j-1))/2)*((-1/(beta*beta*h*h))*((-1)*h/((x(j+1)-x(j))^2)));     % Forward Element of Jacobian
```

```
        J(j,j+2) = pinv((x(j)+x(j+1))/2)*(1/(beta*beta*h*h))*(h/((x(j+2)
            -x(j+1))^2));       % 2nd Forward Element of Jacobian


    elseif (j==siz-1)   % This is the right side boundary section
        G(j) = pinv((x(j)+x(j+1))/2)*(h/(x(j+1)-x(j)) - (1/(beta*beta*h*
            h))*(h/((x(j+1)-x(j)) + h/(x(j)-x(j-1)))))-pinv((x(j)+x(j-1)
            )/2)*(h/(x(j)-x(j-1)) - (1/(beta*beta*h*h))*(h/(x(j+1)-x(j))
             - 2*h/(x(j)-x(j-1)) + h/(x(j-1)-x(j-2))));      % Right
            boundary of solver G


        J(j,j) = 0.5*pinvprime((x(j)+x(j+1))/2)*(h/(x(j+1)-x(j)) - (1/(
            beta*beta*h*h))*(h/((x(j+1)-x(j)) + h/(x(j)-x(j-1)))))-0.5*
            pinvprime((x(j)+x(j-1))/2)*(h/(x(j)-x(j-1)) - (1/(beta*beta*
            h*h))*(h/(x(j+1)-x(j)) - 2*h/(x(j)-x(j-1)) + h/(x(j-1)-x(j
            -2))))+pinv((x(j)+x(j+1))/2)*(h/((x(j+1)-x(j))^2) - (1/(beta
            *beta*h*h))*(h/((x(j+1)-x(j))^2) + (-1)*h/((x(j)-x(j-1))^2))
            )-pinv((x(j)+x(j-1))/2)*((-1)*h/((x(j)-x(j-1))^2) - (1/(beta
            *beta*h*h))*(h/((x(j+1)-x(j))^2) + 2*h/((x(j)-x(j-1))^2)));
                    % Centre Element of Jacobian


        J(j,j-1) = (-0.5)*pinvprime((x(j)+x(j-1))/2)*(h/(x(j)-x(j-1)) -
            (1/(beta*beta*h*h))*(h/(x(j+1)-x(j)) - 2*h/(x(j)-x(j-1)) + h
```

```
        /(x(j-1)-x(j-2))))+pinv((x(j)+x(j+1))/2)*(((-1)/(beta*beta*h
        *h))*(h/((x(j)-x(j-1))^2)))-pinv((x(j)+x(j-1))/2)*(h/((x(j)-
        x(j-1))^2) - (1/(beta*beta*h*h))*((-2)*h/((x(j)-x(j-1))^2) -
         h/((x(j-1)-x(j-2))^2)));   % Backward Element of Jacobian


    J(j,j-2) = (-1)*pinv((x(j)+x(j-1))/2)*(((-1)/(beta*beta*h*h))*(h
        /((x(j-1)-x(j-2))^2))); % 2nd Backward Element of Jacobian


else            % This is the interior points section
    G(j) = pinv((x(j)+x(j+1))/2)*(h/(x(j+1)-x(j)) - (1/(beta*beta*h*
        h))*(h/(x(j+2)-x(j+1)) - 2*h/(x(j+1)-x(j)) + h/(x(j)-x(j-1))
        ))-pinv((x(j)+x(j-1))/2)*(h/(x(j)-x(j-1)) - (1/(beta*beta*h*
        h))*(h/(x(j+1)-x(j)) - 2*h/(x(j)-x(j-1)) + h/(x(j-1)-x(j-2))
        ));     % Interior points of solver G


    J(j,j+1) = 0.5*pinvprime((x(j)+x(j+1))/2)*(h/(x(j+1)-x(j)) -
        (1/(beta*beta*h*h))*(h/(x(j+2)-x(j+1)) - 2*h/(x(j+1)-x(j)) +
         h/(x(j)-x(j-1))))+pinv((x(j)+x(j+1))/2)*((-1*h)/((x(j+1)-x(
        j))^2) - (1/(beta*beta*h*h))*(h/((x(j+2)-x(j+1))^2) + 2*h/((
        x(j+1)-x(j))^2)))-pinv((x(j)+x(j-1))/2)*((-1/(beta*beta*h*h)
        )*((-1)*h/((x(j+1)-x(j))^2)));     % Forward Element of
        Jacobian
```

```
J(j,j) = 0.5*pinvprime((x(j)+x(j+1))/2)*(h/(x(j+1)-x(j)) - (1/(
    beta*beta*h*h))*(h/(x(j+2)-x(j+1)) - 2*h/(x(j+1)-x(j)) + h/(
    x(j)-x(j-1))))-0.5*pinvprime((x(j)+x(j-1))/2)*(h/(x(j)-x(j
    -1)) - (1/(beta*beta*h*h))*(h/(x(j+1)-x(j)) - h/(x(j)-x(j-1)
    )))+pinv((x(j)+x(j+1))/2)*(h/((x(j+1)-x(j))^2) - (1/(beta*
    beta*h*h))*((-2)*h/((x(j+1)-x(j))^2) - h/((x(j)-x(j-1))^2)))
    -pinv((x(j)+x(j-1))/2)*((-1)*h/((x(j)-x(j-1))^2) - (1/(beta*
    beta*h*h))*(h/((x(j+1)-x(j))^2) + h/((x(j)-x(j-1))^2)));
            % Centre Element of Jacobian

J(j,j-1) = (-0.5)*pinvprime((x(j)+x(j-1))/2)*(h/(x(j)-x(j-1)) -
    (1/(beta*beta*h*h))*(h/(x(j+1)-x(j)) - 2*h/(x(j)-x(j-1)) + h
    /(x(j-1)-x(j-2))))+pinv((x(j)+x(j+1))/2)*(((-1)/(beta*beta*h
    *h))*(h/((x(j)-x(j-1))^2)))-pinv((x(j)+x(j-1))/2)*(h/((x(j)-
    x(j-1))^2) - (1/(beta*beta*h*h))*((-2)*h/((x(j)-x(j-1))^2) -
     h/((x(j-1)-x(j-2))^2)));   % Backward Element of Jacobian


    if (j~=3)                                   % Special
       conditions to account for the boundaries
```

```
                J(j,j-2) =(-1)*pinv((x(j)+x(j-1))/2)*(((-1)/(beta*beta*h
                    *h))*(h/((x(j-1)-x(j-2))^2)));  % 2nd Backward
                    Element of Jacobian
            end
            if (j<siz-2)
                J(j,j+2) = pinv((x(j)+x(j+1))/2)*(1/(beta*beta*h*h))*((h
                    )/((x(j+2)-x(j+1))^2));     % 2nd Forward Element of
                     Jacobian
            end

    end

end

for j = 1:siz-2             % This section exists for the purpose of
   indexing properly and then the relevant matrix must be stored here
   to avoid singular matricies

    G_dum(j) = G(j+1);
    J_dum(j,j) = J(j+1,j+1);
    if(j<siz-2)
    J_dum(j,j+1) = J(j+1,j+2);
```

```
    end
    if(j<siz-3)
    J_dum(j,j+2) = J(j+1,j+3);
    end
    if(j>1)
    J_dum(j,j-1) = J(j+1,j);
    end
    if(j>2)
    J_dum(j,j-2) = J(j+1,j-1);
    end
end









end

function y = pinv(x)            % The function call for the actual 1/p
   term
    y = 1/(x^2 + 1);
end
```

```
function y = pinvprime(x)       % The function call for the derivative
   of the 1/p term
    y = (-2)*x/((x^2 + 1)^2);
end
```

Bibliography

[1] U. Ascher and C. Greif. *A First Course on Numerical Methods*. Computational Science and Engineering. Society for Industrial and Applied Mathematics, 2011.

[2] C. de Boor. Good approximation by splines with variable knots ii. *Lecture Notes in Mathematics*, 363:12–20, 1974.

[3] O. Dubois, M. J. Gander, S. Loisel, A. St-Cyr, and D. B. Szyld. The optimized schwarz method with a coarse grid correction. *SIAM Journal on Scientific Computing*, 34(1):A421–A458, 2012.

[4] M. J. Gander and G. H. Golub. A non-overlapping optimized schwarz method which converges with arbitrarily weak dependence on h. 2003.

[5] M. J. Gander and R. D. Haynes. Domain decomposition approaches for mesh generation via the equidistribution principle. *Siam Journal on Numerical Analysis*, 50:2111–2135, 2012.

[6] M. J. Gander, R. D. Haynes, and A. J. M. Howse. Alternating and linearized alternating schwarz methods for equidistributing grids. In *Domain Decomposition Methods in Science and Engineering XX*, pages 395–402. Springer Berlin Heidelberg, 2013.

[7] W. Huang, Y. Ren, and R. D. Russell. Moving mesh partial differential equations (mmpdes) basded on the equidistribution principle. *Siam Journal on Numerical Analysis*, 31:709–730, 1994.

[8] W. Huang and R. D. Russell. Analysis of moving mesh partial differential equations with spatial smoothing. *Siam Journal on Numerical Analysis*, 34:1106–1126, 1997.

[9] W. Huang and R. D. Russell. *Adaptive Moving Mesh Methods*. Springer, USA, 2011.

[10] R. J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM, USA, 2007.

[11] J. Pryce. On the convergence of iterated remeshing. *IMA journal of numerical analysis*, (March 1988):315–335, 1989.

[12] A. Toselli and O. B. Widlund. *Domain Decomposition Methods – Algorithms and Theory*. Springer, Germany, 2005.